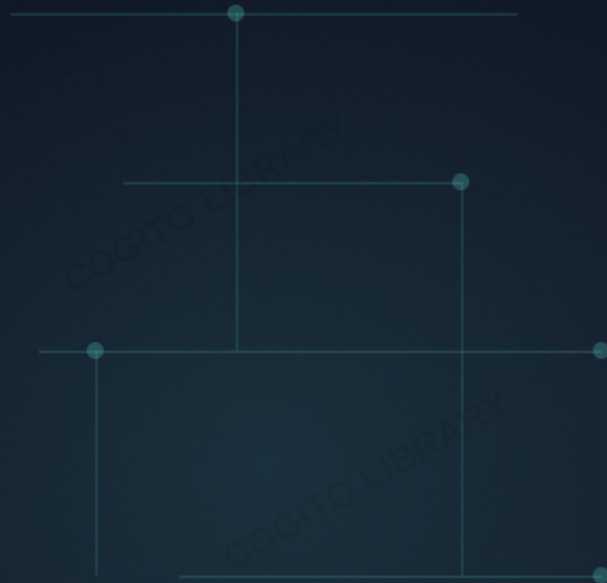




Claude Code로 1인 개발팀 만들기

30개 서비스를 혼자 운영하는
AI 에이전트 워크북

프롤로그: 팀원이 없는 팀

지금 이 순간에도 격차는 벌어지고 있습니다

2026년 1분기, GitHub이 발표한 데이터에 따르면 AI 코딩 도구를 사용하는 개발자의 코드 생산량은 그렇지 않은 개발자의 **3.2배**입니다. 하지만 이건 시작에 불과합니다. AI를 '프롬프트 도구'로 쓰는 사람과 '에이전트 팀'으로 쓰는 사람의 격차는 **10배 이상**으로 벌어집니다.

당신 옆자리 개발자는 이미 에이전트로 3배 빠르게 일하고 있습니다.

같은 연차, 같은 기술 스택. 한 명은 서비스 3개를 야근하며 운영합니다. 다른 한 명은 서비스 10개를 혼자 운영하면서 정시에 퇴근합니다. 차이는 실력이 아닙니다. **AI를 도구로 쓰느냐, 팀으로 쓰느냐**의 차이입니다.

구체적으로 비교해 보겠습니다. 같은 기능을 구현하는 데 한 사람은 프롬프트를 던지고, 결과를 복사하고, 수동으로 테스트하고, 직접 배포합니다. 4시간. 다른 한 사람은 에이전트에게 구현을 지시하고, QA 에이전트가 자동 검증하고, CI/CD가 배포합니다. 40분. **같은 시간에 6배 더 많은 일**을 합니다.

AI 코딩 도구를 '써봤다'와 '팀으로 쓴다'의 차이는 연봉 2천만 원입니다. 이것은 과장이 아니라, 1인 개발로 서비스를 운영해서 발생하는 **실제 수익 차이**입니다.

이 책이 그 격차를 메웁니다. 이 책을 읽기 전의 당신은 AI에게 "이 코드 리팩토링해줘"라고 말하는 사람입니다. 읽고 난 후의 당신은 자는 동안에도 에이전트가 장애를 감지하고, 롤백하고, 보고서를 남기는 시스템을 운영하는 사람입니다.

그 전환이 어떻게 시작됐는지, 새벽 3시의 이야기부터 해보겠습니다.

오전 3시 17분

슬랙 알림이 울렸습니다.

[AGENT] ● 심리테스트 플랫폼 - DB 커넥션 풀 고갈 감지
[AGENT] 롤백 실행 중... (v2.3.1 → v2.2.9)
[AGENT] 롤백 완료. 응답시간 정상화 확인.
[AGENT] 원인 분석 리포트 작성 완료 → #incident 채널

저는 자고 있었습니다.

아무도 깨우지 않았고, 아무도 전화하지 않았습니다. 에이전트가 장애를 감지하고, 롤백을 판단하고, 실행하고, 보고서까지 남겼습니다. 다음날 아침에 일어나서 슬랙을 열었을 때 처음 이 사실을 알았습니다.

2026년 현재, 저는 30개 서비스를 혼자 운영합니다. 심리테스트 플랫폼, 교육 플랫폼, 결제 서비스, 모니터링 봇, 인터뷰 준비 서비스, 일정 관리 툴... 팀원 수는 그대로 1명입니다.

달라진 건 하나입니다.

Claude Code를 '도구'가 아닌 '팀'으로 쓰기 시작한 것.

2년 전의 저

2024년 초, 저는 3개 서비스를 겨우 운영하고 있었습니다.

AI 코딩 도구는 이미 쓰고 있었습니다. “이 함수 리팩토링해줘”, “버그 찾아줘”, “테스트 코드 짜줘”. 똑같이 프롬프트를 던지고, 코드를 복사하고, 붙여넣고, 손으로 고쳤습니다. AI가 저를 도와주는 것처럼 보였지만, 실제로는 AI를 제가 관리하고 있었습니다.

그것도 엄청나게 비효율적으로.

매 요청마다 컨텍스트를 다시 설명해야 했습니다. “이 프로젝트는 Spring Boot야”, “우리는 헥사고날 아키텍처 써”, “결제는 토스 페이먼츠 연동이야”... 같은 말을 매번 반복했습니다. 새벽에 장애가 나면 저만 깨어 있었습니다. QA는 항상 저 혼자였습니다. 배포 후 모니터링도, 코드 리뷰도, 문서 작성도 전부 제가 처리해야 했습니다.

1인 개발자의 한계는 시간이었습니다. 아이디어는 많고, 실행할 손은 하나뿐.

전환점은 생각보다 단순한 질문에서 시작됐습니다.

“왜 나는 매번 똑같은 컨텍스트를 설명하고 있지?”

프롬프트와 에이전트의 차이

많은 분들이 Claude Code를 프롬프트 도구로 씁니다. 당연한 출발점입니다. 하지만 거기서 멈추면, 터미널에서 ChatGPT를 쓰는 것과 크게 다르지 않습니다.

프롬프트와 에이전트의 차이를 한 줄로 요약하면 이렇습니다.

프롬프트는 내가 지시하고 결과를 받는다. 에이전트는 맥락을 기억하고, 스스로 판단하고, 연속으로 실행한다.

Claude Code에는 이 차이를 만드는 세 가지 핵심이 있습니다.

첫째, `CLAUDE.md`. 프로젝트의 규칙, 아키텍처, 금지사항, 배포 방식을 이 파일 하나에 담아두면, Claude는 매번 같은 질문에 같은 원칙으로 답합니다. 이게 없으면 Claude는 대화가 끊길 때마다 기억을 잃는 임시 조수입니다. 이게 있으면 우리 팀의 컨벤션을 아는 동료가 됩니다.

배포 규칙

- 절대 프로덕션에 직접 배포하지 말 것
- 올바른 흐름: 코드 수정 → 커밋 → push → CI/CD 자동 배포
- Docker 이미지 직접 빌드/전송 금지

이것이 `CLAUDE.md`의 핵심 역할입니다. 이 단 세 줄이 있었기에, 수십 번의 배포 요청에서 Claude는 단 한 번도 프로덕션에 직접 배포하지 않았습니다.

둘째, 서브에이전트. 한 Claude가 모든 것을 하게 하면 느리고 실수가 잦습니다. 역할을 쪼개면 달라집니다. QA 전문 에이전트, 비즈니스 점검 에이전트, 장애 대응 에이전트... 각자 자기 영역의 전문가로 만들 수 있습니다.

셋째, 크론. Claude를 크론으로 자동 실행하면 사람이 없어도 작동하는 시스템이 됩니다. 매일 새벽 2시에 서비스 상태를 점검하고, 이상이 있으면 에이전트가 판단하고 행동합니다.

이 세 가지가 맞물리면, 비로소 팀이 완성됩니다.

이 책이 당신에게 필요한지 확인하세요

다음 항목 중 해당되는 것을 체크해 보십시오.

- Claude Code를 설치했다
- “이 코드 리팩토링해줘” 수준으로 사용 중이다
- `CLAUDE.md`가 뭔지 모른다
- 에이전트와 프롬프트의 차이를 설명할 수 없다
- 크론으로 Claude를 자동 실행하는 게 가능한지 몰랐다
- AI 도구를 쓰는데도 여전히 야근한다

3개 이상 체크했다면, 이 책이 당신을 위한 책입니다.

이 책은 Claude Code 설치 방법을 가르쳐주지 않습니다. API 키 발급 방법도 없습니다. 이미 도구는 설치했고, 프롬프트도 써봤지만, 아직 '팀'을 만들지 못한 분들을 위한 책입니다.

이 책에서 얻을 것

이 책을 다 읽고 나면, 다음 세 가지를 실전에서 구현할 수 있습니다.

첫째, 나를 대신해 판단하는 에이전트를 설계할 수 있습니다. 단순 자동화가 아닙니다. “이 PR은 머지해도 되는가?”, “지금 롤백해야 하는가?”를 에이전트가 판단하는 구조를 만드는 법을 배웁니다.

둘째, 자는 동안 작동하는 시스템을 구축할 수 있습니다. 크론 + 에이전트 조합으로, 새벽에도 QA가 돌아가고, 장애 대응이 실행되고, 보고서가 쌓입니다.

셋째, 10개 이상의 서비스를 혼자 관리하는 운영 체계를 갖출 수 있습니다. 정기 점검 루틴, 비즈니스 점검, 기술 점검, 성장 검토까지 — 저자가 실제로 30개 서비스를 운영하며 만든 검증된 체계를 그대로 가져갑니다.

이 책의 여정

이 책은 다섯 개의 단계로 구성됩니다.

Part 1 — 팀의 기반 만들기 (CH01~02)

가장 중요한 두 가지, `CLAUDE.md` 와 서브에이전트를 설계합니다. 여기서 기반이 탄탄하지 않으면 뒤의 모든 자동화가 흔들립니다.

Part 2 — 잠자는 동안 돌아가는 QA (CH03~04)

테스트, 코드 리뷰, E2E 검증을 에이전트에게 위임하는 법을 다룹니다. 사람이 없어도 품질이 유지되는 구조입니다.

Part 3 — 자율 운영 체계 (CH05~06)

크론 스케줄링, 장애 감지, 자동 롤백까지. 운영 부담을 90% 줄이는 자동화 레이어를 만듭니다.

Part 4 — 성장 엔진 (CH07~08)

비즈니스 지표 모니터링, A/B 테스트 자동화, 기능 우선순위 판단까지. Claude를 개발자가 아닌 공동 창업자로 쓰는 법을 다룹니다.

Part 5 — 스케일 (CH09~10)

30개 서비스를 혼자 운영하는 실전 아키텍처. 에이전트 간 협업 구조, 지식 축적 시스템, 그리고 '팀'이 자기 스스로 성장하게 만드는 마지막 단계입니다.

시작하기 전에

한 가지만 당부드립니다.

이 책의 예시는 전부 실제 운영 중인 시스템에서 가져왔습니다. 이상적인 코드가 아니라, 실수하고, 롤백하고, 다시 설계한 흔적이 담긴 실전 코드입니다. 완벽하지 않습니다. 하지만 작동합니다.

Claude Code를 팀으로 만드는 일은, 좋은 동료로 온보딩하는 일과 같습니다. 처음에는 어색하고, 규칙을 정의하는 데 시간이 걸립니다. 하지만 한 번 제대로 세팅되면, 당신이 없어도 일이 돌아가는 날이 옵니다.

그 날이 오면, 오전 3시에 울리는 알람도 더 이상 두렵지 않습니다.

CH01. AI 코딩 시대의 지형도

“Claude Code 잘 쓰고 있어요.”

그 말이 얼마나 위험한지, 이 챕터를 읽고 나면 알게 됩니다.

1.1 당신은 지금 어디에 서 있습니까

2024년 말 기준, 국내 개발자의 80% 이상이 AI 코딩 도구를 사용한다고 답합니다. 그런데 이상한 일이 있습니다. 도구를 쓰는 사람은 늘었는데, “AI 덕분에 팀 하나를 줄였다”거나 “혼자서 10개 서비스를 운영한다”는 사람은 극히 드뭅니다.

왜일까요?

도구를 쓰는 것과, 도구를 제대로 쓰는 것은 다릅니다. 망치를 들고 있다고 해서 목수가 아닌 것처럼.

AI 코딩 도구의 활용에는 명확한 성숙도 단계가 있습니다. 이 챕터에서는 그 5단계를 정의하고, 각 단계에서 실제로 무슨 일이 일어나는지를 코드와 터미널 예시로 보여드립니다. 그리고 마지막에 하나를 묻습니다.

당신은 지금 몇 단계입니까?

1.2 AI 코딩 성숙도 5단계 모델

Level 5	에이전트 시스템	자율 운영 + 크론 + 품질 게이트
Level 4	프로토콜 기반	CLAUDE.md + 컨텍스트 시스템
Level 3	인라인 에이전트	Claude Code/Cursor, 파일 단위
Level 2	채팅	ChatGPT에 코드 붙여넣기
Level 1	자동완성	Copilot, 코드 서제스천

대부분의 개발자는 Level 2~3에 머뭅니다. Level 3에 도달하면 “나는 AI를 잘 쓴다”고 느낍니다. 던닝-크루거의 정점입니다. 실제 생산성 레버리지는 Level 4부터 시작됩니다.

1.3 Level 1 — 자동완성: 도구가 내 손가락을 돕는다

대표 도구: GitHub Copilot, Cursor (기본 모드), JetBrains AI Assistant

작동 방식: 타이핑하면 다음 줄을 예측합니다. Tab을 누르면 수락, 무시하면 계속 씁니다.

```
# 개발자가 타이핑:
def find_active_users():

# Copilot이 제안:
    return user_repo.find_by_status("ACTIVE")
```

진짜 쓰임새: 반복 보일러플레이트 감소. 함수 이름만 쓰면 구현을 제안받습니다.

한계: AI는 지금 커서 위치의 맥락만 봅니다. 프로젝트 전체 아키텍처를 모릅니다. 그래서 제안이 빗나갈 때가 많습니다.

Level 1 개발자의 하루: - 코드 10줄 쓰는 데 Tab을 7번 누릅니다 - 생산성이 올랐다는 느낌이 있습니다 - 하지만 설계 결정, 디버깅, 아키텍처 판단은 여전히 100% 본인 몫입니다

1.4 Level 2 — 채팅: 코드를 붙여넣고 답을 받아온다

대표 도구: ChatGPT, Claude.ai (웹), Gemini

작동 방식: 브라우저 탭을 열고, 에러 메시지나 코드를 복사해서 붙여넣습니다. 답을 받으면 다시 IDE로 돌아옵니다.

전형적인 Level 2 세션:

[개발자 → ChatGPT 웹]

아래 코드에서 `user.profile`이 `null`일 때 에러가 납니다. 원인 알려주세요.

```
function getUsername(userId) {  
  const user = findUserId(userId);  
  return user.profile.name;  
}
```

[ChatGPT 응답]

`user.profile`이 `null`이거나 `undefined`일 때 `.name`에 접근하면 에러가 발생합니다.

옵셔널 체이닝이나 `null` 체크를 추가하세요:

```
function getUsername(userId) {  
  const user = findUserId(userId);  
  return user?.profile?.name ?? "Unknown";  
}
```

Level 2의 문제: 컨텍스트 이동 비용이 큼니다. IDE → 브라우저 → 복사 → 붙여넣기 → 답 확인 → IDE. 이 사이클이 하루에 수십 번 반복됩니다. 그리고 AI는 프로젝트 구조를 전혀 모릅니다. 매번 새로 설명해야 합니다.

Level 2 개발자가 자주 겪는 상황: - “이 코드 어디에 넣으면 돼?” → 직접 판단해야 합니다 - “우리 프로젝트 규칙이 있는데 AI가 무시하고 다른 패턴으로 씁니다” - “매번 배경 설명하는 게 귀찮습니다”

1.5 Level 3 — 인라인 에이전트: IDE 안에서 파일을 고친다

대표 도구: Claude Code, Cursor (Agent 모드), Windsurf

작동 방식: AI가 IDE 안에서 실제 파일을 읽고, 수정하고, 실행합니다. 컨텍스트 이동이 없습니다.

Level 3 터미널 세션 예시:

```
# 프로젝트 루트에서 Claude Code 실행
$ claude

# 대화 시작
> 회원가입 API를 만들어줘. 이메일, 비밀번호, 닉네임을 받아서
  users 테이블에 저장하고, 이메일 중복 체크도 해줘.

# Claude Code가 실제로 하는 일:
# 1. 프로젝트 파일 구조 탐색
# 2. 기존 코드 패턴 파악
# 3. 라우터/컨트롤러 파일 수정
# 4. 서비스 로직 파일 수정
# 5. DB 접근 레이어 수정
# 6. 결과 보고
```

Level 3이 Level 2보다 나은 점: - 파일을 직접 수정합니다. 복붙이 없습니다 - 프로젝트 파일을 읽어서 기존 패턴을 참고합니다 - 여러 파일에 걸친 변경을 한 번에 처리합니다

그런데 Level 3에는 결정적인 약점이 있습니다.

```
# 다음 날 같은 프로젝트에서 다시 요청
> 결제 API를 추가해줘.

# Claude Code는 어제 회원가입 API를 만든 걸 모릅니다.
# 프로젝트 구조를 다시 탐색합니다.
# 어제와 다른 패턴으로 코드를 작성할 수도 있습니다.
# 여러분의 코딩 스타일, 제약사항, 금지 패턴을 모릅니다.
```

Level 3의 핵심 문제: 세션이 끝나면 컨텍스트가 사라집니다. AI에게 프로젝트는 매번 처음 보는 곳입니다.

1.6 Level 4 — 프로토콜 기반: AI가 우리 팀의 규칙을 안다

핵심 도구: CLAUDE.md (컨텍스트 파일) + Claude Code

Level 4는 발상의 전환에서 시작합니다. AI에게 매번 설명하는 게 아니라, AI가 항상 읽는 문서를 만드는 것입니다.

CLAUDE.md 는 Claude Code가 세션 시작 시 자동으로 읽는 파일입니다. 여기에 프로젝트의 모든 규칙을 씁니다.

Level 3 방식 vs Level 4 방식 비교 실험

같은 요청, 두 가지 방식:

Level 3 방식 (CLAUDE.md 없음):

```
$ claude
> 회원가입 API를 만들어줘. 이메일, 비밀번호, 닉네임을 받아서 저장해줘.
```

Claude Code가 만들어낸 코드는 프레임워크 기본 패턴을 따릅니다. 하지만 어제 만든 결제 API는 공통 응답 래퍼(`ApiResponse`)를 사용했는데, 오늘 코드는 단순 문자열을 반환합니다. 일관성이 깨졌습니다.

Level 4 방식 (CLAUDE.md 설정 후):

먼저 프로젝트 루트에 `CLAUDE.md` 를 만듭니다:

```
# 프로젝트 컨텍스트

## 기술 스택
- 백엔드: Python/FastAPI (또는 Spring Boot, Express 등 사용 중인 스택)
- DB: PostgreSQL
- 인증: JWT (Authorization 헤더, Bearer 토큰)

## 응답 형식 규칙
모든 API 응답은 반드시 공통 응답 래퍼로 감싼다:
성공: { "success": true, "data": ... }
실패: { "success": false, "error": ... }
단순 문자열이나 딕셔너리를 직접 반환 금지.
```

이어서 프로젝트 구조와 금지 패턴입니다:

```
## 프로젝트 구조
src/
├── routes/           (HTTP 라우팅만. 비즈니스 로직 금지)
├── services/         (비즈니스 로직)
├── repositories/     (DB 접근 레이어)
├── models/           (도메인 모델)
└── schemas/          (요청/응답 스키마)

## 금지 패턴
- 라우터/컨트롤러에 비즈니스 로직 작성 금지
- DB 조회 실패 시 예외 없이 넘어가기 금지. 명시적 에러 처리 필수
- print/console.log 디버깅 금지. 로거 사용
```

이제 같은 요청을 합니다:

```
$ claude
```

```
> 회원가입 API를 만들어줘. 이메일, 비밀번호, 닉네임을 받아서 저장해줘.
```

이번에 만들어진 코드는 공통 응답 래퍼를 사용하고, 로거로 로깅하고, 입력 검증을 수행하고, 비즈니스 로직을 서비스 레이어에 위임합니다. CLAUDE.md에 쓴 규칙이 모두 반영됩니다.

Level 4의 핵심: AI가 프로젝트 규칙을 “기억”합니다. 정확히는 매 세션마다 자동으로 읽습니다. 결과적으로 같은 효과입니다.

1.7 Level 5 — 에이전트 시스템: AI가 혼자 일한다

Level 5는 사람이 요청하지 않아도 AI가 작업을 실행하는 단계입니다.

핵심 구성요소: 1. **자율 에이전트:** 특정 역할을 부여받은 Claude Code 인스턴스 2. **크론(Cron):** 정해진 시간에 에이전트를 자동 실행 3. **품질 게이트:** 에이전트 결과물을 검증하는 자동화된 기준

실제 예시 — 코드 품질 자동 검수 에이전트:

```
# /ops/agents/code-reviewer.sh
#!/bin/bash

# 매일 오전 9시 자동 실행
# 어제 커밋된 코드를 검토하고 리포트 생성

PROJECT_DIR="/home/deploy/my-service"
REPORT_DIR="/home/deploy/reports/${date +%Y-%m-%d}"
mkdir -p "$REPORT_DIR"

cd "$PROJECT_DIR"

# 어제 변경된 파일 목록 추출
CHANGED_FILES=$(git diff --name-only HEAD~1 HEAD -- '*.java' '*.kt')
```

변경 파일이 있으면 Claude로 코드 리뷰를 실행합니다:

```

if [ -z "$CHANGED_FILES" ]; then
    echo "어제 변경된 파일 없음. 종료."
    exit 0
fi

# Claude Code로 코드 리뷰 실행
claude --print \
    "다음 파일들을 코드 리뷰해줘. CLAUDE.md의 규칙 기준으로 위반 사항을 찾고,
    심각도(CRITICAL/MAJOR/MINOR)와 함께 리포트를 만들어줘.
    검토 파일: $CHANGED_FILES
    리포트 파일로 저장: $REPORT_DIR/review.md" \
    > "$REPORT_DIR/agent-log.txt" 2>&1

echo "코드 리뷰 완료: $REPORT_DIR/review.md"

```

```

# crontab -e 로 등록
0 9 * * 1-5 /home/deploy/ops/agents/code-reviewer.sh >> /var/log/agent-cron.log 2>&1

```

이 스크립트가 평일 오전 9시마다 자동으로 실행됩니다. 개발자가 자는 동안에도.

실제 품질 게이트 예시 — PR 병합 전 자동 검증:

```

# .github/hooks/pre-merge-check.sh
#!/bin/bash

echo "=== AI 품질 게이트 시작 ==="

# 1. 유닛 테스트 실행
./gradlew test --quiet
if [ $? -ne 0 ]; then
    echo "FAIL: 테스트 실패. 병합 거부."
    exit 1
fi

```

이어서 Claude로 보안 취약점을 스캔합니다:

```
# 2. Claude Code로 보안 취약점 스캔
SECURITY_RESULT=$(claude --print \
  "변경된 파일에서 SQL Injection, XSS, 하드코딩된 시크릿을 스캔해줘.
  발견 시 SECURITY_ISSUE:true를 첫 줄에 출력해줘.")

if echo "$SECURITY_RESULT" | grep -q "SECURITY_ISSUE:true"; then
  echo "FAIL: 보안 이슈 발견. 병합 거부."
  echo "$SECURITY_RESULT"
  exit 1
fi

echo "PASS: 품질 게이트 통과."
exit 0
```

Level 5 개발자의 하루: 아침에 출근하면 어젯밤 에이전트들이 남긴 리포트가 쌓여 있습니다. 코드 리뷰, 에러 로그 분석, 신규 사용자 온보딩 현황, 결제 실패 건. 판단만 합니다. 수집과 분석은 에이전트가 했습니다.

1.8 도구 간 핵심 차이

같은 태스크 — “우리 프로젝트에 맞는 결제 API를 추가해줘”

이 요청을 도구별로 실행하면 결과가 극적으로 달라집니다. 가장 큰 차이를 만드는 세 가지 기준이 있습니다.

기존 패턴 준수: ChatGPT나 Copilot은 프로젝트의 기존 코드 패턴을 모릅니다. Claude Code도 CLAUDE.md 없이 쓰면(Level 3) 매번 다른 패턴으로 코드를 생성합니다. CLAUDE.md가 있을 때(Level 4+) 비로소 기존 패턴을 일관되게 따릅니다.

세션 간 규칙 유지: 대부분의 도구는 세션이 끝나면 규칙을 잊습니다. CLAUDE.md가 있는 Claude Code만 세션이 바뀌어도 동일한 규칙을 유지합니다.

자율 실행: 크론이나 CI/CD로 도구를 사람 없이 실행할 수 있는 것은 Claude Code뿐입니다. 이것이 Level 5의 핵심입니다.

도구를 바꾸는 게 아니라, 같은 도구를 어떻게 쓰느냐가 레벨을 결정합니다.

1.9 자가진단: 당신은 지금 몇 단계입니까?

아래 체크리스트를 확인하세요. 솔직하게.

Level 1 체크

- GitHub Copilot이나 JetBrains AI의 Tab 제안을 주로 씁니다
- AI 도움은 “코드 한 줄씩 제안받기” 수준입니다
- 버그 원인 분석은 여전히 직접 합니다

Level 2 체크

- ChatGPT나 Claude 웹에서 코드를 붙여넣고 답을 받아옵니다
- “이 에러 고쳐줘”, “이 코드 설명해줘” 수준의 요청을 합니다
- AI 답변을 직접 IDE에 옮겨 붙입니다

Level 3 체크

- Claude Code나 Cursor를 터미널/IDE에서 직접 실행합니다
- AI가 파일을 직접 수정하도록 허용합니다
- “이 기능 전체 만들어줘” 같은 작업 단위 요청을 합니다

Level 4 체크

- `CLAUDE.md` 가 프로젝트 루트에 있고, 실제 규칙이 담겨 있습니다
- AI가 만든 코드가 기존 패턴과 일관성 있게 나옵니다
- 글로벌 `CLAUDE.md` 와 프로젝트별 `CLAUDE.md` 를 분리해서 씁니다

Level 5 체크

- 크론이나 CI/CD로 Claude Code를 자동 실행하는 파이프라인이 있습니다
- AI가 사람 없이 특정 작업(리뷰, 모니터링, 분석)을 수행합니다
- AI 작업 결과를 검증하는 품질 게이트가 있습니다

진단 결과 해석:

Level 1~2에 해당한다면 CH02부터 차근차근 따라오시면 됩니다. Level 3이라면 CH03(CLAUDE.md 설계)부터가 핵심입니다. Level 4라면 CH05(크론 자동화)에서 자율 운영으로 도약할 수 있습니다. 전부 해당한다면 CH07(30개 서비스 운영)에서 확장 전략을 가져가십시오.

1.10 비교 실험: Level 3과 Level 4의 실제 차이

직접 해보세요. 10분이면 됩니다.

준비: 1. 아무 프로젝트나 열어도 됩니다. 언어와 프레임워크는 무엇이든 상관없습니다. 2. 프로젝트 루트에 `CLAUDE.md` 가 없는지 확인합니다.

실험 A — Level 3 방식 (CLAUDE.md 없음):

```
$ cd /path/to/your-project
$ claude

> 사용자 목록을 페이지네이션으로 조회하는 API를 만들어줘.
page와 size 파라미터를 받아서.
```

결과를 저장해둡니다.

실험 B — Level 4 방식 (CLAUDE.md 작성 후):

```
# 먼저 CLAUDE.md 작성
$ cat > CLAUDE.md << 'EOF'
# 프로젝트 규칙

## 기술 스택
[여기에 실제 스택 기입. 예: FastAPI + SQLAlchemy, Express + Prisma 등]

## 응답 규칙
[예: 모든 응답은 공통 래퍼로 감싼다]

## 페이지네이션 규칙
[예: 페이지는 0-indexed, 기본 size는 20, 최대 100]

## 네이밍 규칙
[예: 함수명은 snake_case, URL은 kebab-case]
EOF

$ claude

> 사용자 목록을 페이지네이션으로 조회하는 API를 만들어줘.
page와 size 파라미터를 받아서.
```

비교 포인트: - 응답 형식이 기존 코드와 일관성이 있는지 - 페이지네이션 구현 방식이 프로젝트 규칙을 따르는지 - 추가 설명 없이도 규칙이 적용되는지

대부분의 경우, 실험 B의 결과물이 바로 PR에 올릴 수 있는 수준으로 나옵니다. 실험 A는 수정이 필요한 경우가 많습니다.

1.11 왜 대부분이 Level 3에서 멈추는가

세 가지 이유가 있습니다.

첫째, Level 3도 충분히 편합니다. “파일을 직접 고쳐주다니!” 감동을 경험하면 여기서 만족합니다. 더 갈 수 있다는 걸 모릅니다.

둘째, Level 4는 선행 투자가 필요합니다. CLAUDE.md를 잘 쓰려면 30분~1시간이 필요합니다. 당장 바쁠 때 이걸 하기가 어렵습니다. 하지만 이 투자는 이후 수십 시간을 돌려줍니다.

셋째, Level 5는 개념 자체를 모릅니다. “Claude Code를 크론으로 자동 실행한다”는 발상 자체가 낮습니다. 가능한지도 몰랐던 것입니다.

이 책은 그 세 장벽을 하나씩 넘어가는 로드맵입니다.

1.12 이 책의 로드맵

챕터별로 레벨을 올려갑니다.

CH02. 개발 환경 세팅	→ Level 3 진입 (Claude Code 제대로 설치)
CH03. 프롬프트 말고 프로토콜	→ Level 4 진입 (CLAUDE.md 설계)
CH04. 에이전트 아키텍처	→ Level 4 심화 (멀티 에이전트 설계)
CH05. 자동화 인프라	→ Level 5 진입 (크론 + 파이프라인)
CH06. 품질 게이트	→ Level 5 심화 (자율 운영의 안전장치)
CH07. 30개 서비스 운영	→ Level 5 실전 (실제 운영 패턴)

지금 당장 Level 5를 목표로 할 필요는 없습니다. 하지만 Level 5가 어떤 모습인지 알고 나서, Level 3에 머무르는 것은 다릅니다.

핵심 요약

- AI 코딩 도구 활용에는 명확한 5단계 성숙도가 있습니다
- 대부분의 개발자는 Level 2~3에서 “잘 쓰고 있다”고 느끼며 멈춥니다
- Level 4의 핵심은 CLAUDE.md — AI가 항상 읽는 규칙 문서입니다
- Level 5는 AI가 자율적으로 작동하는 에이전트 시스템입니다
- 레벨을 올리는 건 도구 교체가 아니라 사용 방식의 전환입니다

다음 챕터에서는 Claude Code를 처음부터 Level 4를 향해 세팅하는 방법을 다룹니다. 설치 방법부터 글로벌/프로젝트 CLAUDE.md 분리, MCP 서버 연결, 모델 선택 전략까지. 이미 설치했어도 다시 보세요. 세팅이 달라 집니다.

CH02. 개발 환경 세팅: 5분 만에 Claude Code를 Level 4로 올리는 법

이 챕터를 마치면: 글로벌/프로젝트 CLAUDE.md 2개를 작성하고, MCP 서버 1개를 연결하고, 커스텀 alias와 hooks를 갖춘 Level 4 환경을 완성합니다.

대부분의 개발자는 Claude Code를 설치한 뒤 이렇게 사용합니다.

```
$ claude
> 이 함수 버그 찾아줘
> 이 파일 리팩토링해줘
> 유닛 테스트 작성해줘
```

이것은 “Level 1 — 비서 수준”입니다. Claude Code가 내 코드베이스를 모르고, 내 프로젝트 규칙을 모르고, 외부 도구와 연결도 안 된 상태입니다. 매번 컨텍스트를 설명해야 하고, 잘못된 패턴으로 코드를 생성할 때마다 수정해야 합니다.

이 챕터에서는 총 5단계 실습으로 Level 1에서 Level 4까지 끌어올립니다. 총 소요 시간은 약 30분입니다.

Claude Code 환경 레벨 정의

이 챕터에서는 환경 설정을 4단계로 나눠서 진행합니다. **Level 1**(비서 수준)은 프롬프트만 치는 상태입니다. **Level 2**는 CLAUDE.md를 작성해서 프로젝트 규칙을 인식시킨 상태. **Level 3**은 MCP로 외부 시스템(DB, 브라우저)에 직접 접근하는 상태. **Level 4**는 hooks, alias, 모델 전략까지 갖춘 완성된 환경입니다.

CLAUDE.md란 무엇인가

Claude Code는 실행 시 두 곳에서 지침 파일을 자동으로 읽습니다.

1. `~/.claude/CLAUDE.md` — **글로벌 지침**: 모든 프로젝트에서 공통으로 적용
2. `{프로젝트 루트}/CLAUDE.md` — **프로젝트 지침**: 해당 프로젝트에서만 적용

두 파일이 모두 존재하면 글로벌 지침이 먼저 적용되고, 프로젝트 지침이 그 위에 덮어씁니다. 프로젝트 지침이 글로벌 지침보다 높은 우선순위를 갖습니다.

이 구조 덕분에 “모든 프로젝트에서 지켜야 할 나만의 원칙”과 “이 프로젝트에서만 필요한 도메인 규칙”을 분리해서 관리할 수 있습니다.

실습 1: 글로벌 CLAUDE.md 작성 (5분)

글로벌 CLAUDE.md는 내 개발 철학과 전역 규칙을 담는 파일입니다. 한 번 잘 작성해 두면 모든 프로젝트에서 동일하게 적용됩니다.

```
# 파일이 없으면 디렉토리 먼저 생성
mkdir -p ~/.claude
touch ~/.claude/CLAUDE.md
```

다음은 실전에서 검증된 글로벌 CLAUDE.md 예시입니다.

```
# 글로벌 개발 원칙

## 커밋 규칙
- Co-Authored-By 서명 제거 (AI 서명 불필요)
- feat/fix/chore/docs 컨벤션 준수
- 커밋 메시지는 한국어 또는 영어 중 프로젝트 규칙 따름

## 코드 스타일
- 함수는 단일 책임 원칙 준수
- 주석은 "왜"를 설명, "무엇"은 코드가 이미 설명함
- 매직 넘버는 상수로 추출
```

이어서 금지 규칙, 배포 규칙, 검색 전략입니다:

절대 금지

- 프로덕션 환경에 직접 배포 (Docker 이미지 직접 전송 등)
- 비밀번호/API 키 임의 생성 (반드시 사용자에게 받을 것)
- .env 파일 커밋

배포 규칙

- 올바른 흐름: 코드 수정 → 커밋 → push → CI/CD 자동 배포
- SSH 직접 접근이 필요하면 먼저 묻기

검색 전략

- 정확한 키워드를 아는 경우 → Grep, Glob 사용
- 개념/기능으로 찾는 경우 → semantic_search 사용
- Grep으로 못 찾을 경우 → semantic_search로 재시도

포인트: 글로벌 CLAUDE.md는 “내가 어떤 개발자인가”를 Claude에게 가르치는 파일입니다. 규칙이 많을수록 좋은 것이 아니라, 실제로 지키는 규칙만 넣어야 합니다. 예외 없이 지키지 못할 규칙을 넣으면 Claude가 혼란을 겪습니다.

실습 2: 프로젝트 CLAUDE.md 작성 (10분)

프로젝트 CLAUDE.md는 해당 프로젝트의 도메인 지식과 기술 스택 규칙을 담습니다. 새 프로젝트를 시작할 때마다 작성합니다.

```
# 프로젝트 루트에서
touch CLAUDE.md
```

다음은 Spring Boot + React 기반 교육 플랫폼 프로젝트의 예시입니다.

교육 플랫폼 프로젝트

기술 스택

- Backend: Spring Boot 3.3, Java 21
- Frontend: React 18, TypeScript, TailwindCSS
- DB: MySQL 8.0 (Hexagonal Architecture)
- 인프라: Docker, GitHub Actions CI/CD

아키텍처 규칙

- Hexagonal Architecture 준수: domain → application → infrastructure 방향
- 도메인 레이어에서 외부 의존성 절대 금지 (Spring, JPA 어노테이션 불가)
- 포트(Port)와 어댑터(Adapter) 명확히 분리

API 규칙

- 모든 API는 /api/v1/ prefix 사용
- Gateway 경유 시 /api/{서비스명}/ 경로 사용 (Gateway가 /api/v1/으로 rewrite)
- 인증: Bearer JWT (Authorization 헤더)

이어서 데이터베이스, 테스트, 프로덕션 접근 규칙입니다:

데이터베이스 규칙

- 마이그레이션: Flyway 사용
- 네이밍: snake_case (테이블명, 컬럼명)
- Soft Delete 패턴: deleted_at 컬럼 사용

테스트 규칙

- 단위 테스트: JUnit 5 + Mockito
- 통합 테스트: @SpringBootTest + TestContainers
- 기존 실패 테스트는 수정 없이 그대로 둘 것 (별도 이슈로 관리)

프로덕션 접근

- SSH: ssh -p 2222 deploy@my-server.example.com
- DB 접근: docker exec my-mysql mysql -u root -p

현재 상태

- 결제 모듈: Toss Payments 연동 완료
- 사용자 인증: OAuth2 (Google, Kakao) 완료
- 구독 관리: 진행 중

포인트: 아키텍처 규칙 섹션이 핵심입니다. Claude는 프로젝트 구조를 파악하는 데 상당한 토큰을 소비합니다. CLAUDE.md에 “이 프로젝트는 Hexagonal Architecture를 씁니다”라고 명시해 두면 Claude가 도메인 레이어에 Spring 어노테이션을 추가하는 실수를 하지 않습니다.

실습 3: MCP 서버 연결 (10분)

MCP(Model Context Protocol)는 Claude Code가 외부 시스템에 직접 접근할 수 있게 해주는 연결 프로토콜입니다. MCP 서버를 연결하면 Claude Code는 단순한 코드 생성기를 넘어 실제 시스템을 조작하는 에이전트가 됩니다.

MCP가 가능하게 하는 것

MCP 없이:

"이 쿼리 결과가 어떻게 나오는지 확인해줘"

→ Claude: "SQL 쿼리를 실행해서 결과를 알려드릴 수 없습니다. 직접 실행하신 후..."

MCP 있을 때:

"이 쿼리 결과가 어떻게 나오는지 확인해줘"

→ Claude: (직접 DB에 쿼리 실행) "결과는 총 47건이며, 그 중 30일 이상 지난 레코드가..."

대표적인 MCP 서버

가장 자주 쓰이는 MCP 서버는 **Playwright**(브라우저 자동화, E2E 테스트)와 **MySQL/PostgreSQL**(DB 직접 조회)입니다. 이 두 가지만 연결해도 Claude Code의 활용 범위가 크게 넓어집니다. 그 외에 Notion(문서 관리), GitHub(PR/이슈), Slack(알림) 등도 필요에 따라 추가할 수 있습니다.

MCP 설정 파일 위치

MCP 서버는 `~/.claude.json` (글로벌) 또는 프로젝트 루트의 `.mcp.json` (프로젝트 전용)에 설정합니다.

```
# 글로벌 MCP 설정
cat ~/.claude.json
```

Playwright MCP 연결 실습

웹 기반 프로젝트라면 Playwright MCP가 가장 즉각적인 효과를 냅니다. Claude Code가 직접 브라우저를 열고 UI를 테스트할 수 있게 됩니다.

1단계: Playwright MCP 설치

```
npm install -g @playwright/mcp
npx playwright install chromium
```

2단계: MCP 설정 파일 작성

```
# 글로벌 설정 (모든 프로젝트에서 사용)
cat > ~/.claude.json << 'EOF'
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp", "--headless"]
    }
  }
}
EOF
```

특정 프로젝트에서만 사용하려면 프로젝트 루트에 `.mcp.json` 을 생성합니다.

```
# 프로젝트 전용 설정
cat > .mcp.json << 'EOF'
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp", "--headless"]
    },
    "mysql": {
      "command": "npx",
      "args": ["-y", "@benborla29/mcp-server-mysql"],
      "env": {
        "MYSQL_HOST": "localhost",
        "MYSQL_PORT": "3306",
        "MYSQL_USER": "dev_user",
        "MYSQL_PASS": "dev_password",
        "MYSQL_DB": "myapp_dev"
      }
    }
  }
}
EOF
```

3단계: Claude Code 재시작 후 확인

```
claude
> /mcp
```

MCP 서버 목록에 `playwright` 가 표시되면 연결 성공입니다.

4단계: MCP 동작 확인

> 로컬 서버 `http://localhost:3000`에 접속해서 로그인 페이지가 정상적으로 렌더링되는지 확인해줘

Claude Code가 브라우저를 직접 열고 스크린샷을 찍어 결과를 보고합니다.

MySQL MCP 연결 실습

백엔드 개발에서 가장 자주 쓰는 MCP입니다. Claude Code가 직접 DB를 조회하고 데이터를 확인할 수 있습니다.

```
# MySQL MCP 설치
npm install -g @benborla29/mcp-server-mysql
```

`.mcp.json`에 위의 `mysql` 설정을 추가한 뒤 Claude Code를 재시작하면 됩니다.

주의: 프로덕션 DB가 아닌 개발용 DB만 연결하세요. Claude Code가 실수로 데이터를 수정할 수 있습니다. 프로덕션 연결이 필요하다면 읽기 전용 계정을 생성해서 사용하세요.

```
-- 읽기 전용 계정 생성 예시
CREATE USER 'claude_readonly'@'localhost' IDENTIFIED BY 'password';
GRANT SELECT ON myapp.* TO 'claude_readonly'@'localhost';
FLUSH PRIVILEGES;
```

실습 4: settings.json으로 hooks 설정 (5분)

Claude Code의 `settings.json`은 행동 방식을 세밀하게 제어하는 설정 파일입니다. hooks 기능을 사용하면 특정 이벤트 발생 시 자동으로 스크립트를 실행할 수 있습니다.

settings.json 위치

```
~/.claude/settings.json      # 글로벌 설정
{프로젝트 루트}/.claude/settings.json  # 프로젝트 설정
```

사용 가능한 hooks 종류

Hook	실행 시점
<code>PreToolUse</code>	Claude가 도구를 사용하기 직전
<code>PostToolUse</code>	Claude가 도구를 사용한 직후
<code>PostCompact</code>	컨텍스트가 압축된 직후 (긴 세션에서 유용)
<code>Notification</code>	Claude가 사용자에게 알림을 보낼 때
<code>Stop</code>	Claude Code 세션 종료 시

실전 hooks 예시: 파일 변경 후 자동 포맷

```
mkdir -p ~/.claude
cat > ~/.claude/settings.json << 'EOF'
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "echo '[Hook] 파일이 변경되었습니다: $CLAUDE_TOOL_INPUT_FILE_PATH'"
          }
        ]
      }
    ]
  }
}
EOF
```

실전 hooks 예시: 세션 종료 시 git status 확인

```
cat > .claude/settings.json << 'EOF'
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "cd $CLAUDE_PROJECT_DIR && git status --short && echo '--- 세션
종료: 위 파일들을 확인하세요 ---'"
          }
        ]
      }
    ]
  }
}
EOF
```

이 hook을 설정해두면 Claude Code 세션이 끝날 때마다 변경된 파일 목록이 터미널에 출력됩니다. 실수로 불필요한 파일을 수정했는지 바로 확인할 수 있습니다.

hooks 설정이 복잡하게 느껴진다면, Claude에게 직접 시키는 것이 가장 빠릅니다.

Claude에게 이렇게 시키세요:

"파일 변경 후 자동으로 린트를 실행하는 PostToolUse hook을 settings.json에 추가해줘"

Claude가 프로젝트의 빌드 도구(Gradle, npm 등)를 확인하고 적절한 hooks 설정을 만들어줍니다.

실습 5: 셸 alias 설정 (5분)

자주 쓰는 Claude Code 패턴을 alias로 등록해 두면 타이핑 시간을 절약합니다.

```
# ~/.zshrc 또는 ~/.bashrc에 추가
cat >> ~/.zshrc << 'EOF'

# Claude Code aliases
alias cc="claude"
alias ccp="claude --print"                # 출력만 하고 종료
alias ccm="claude --model opus"           # Opus 모델 강제 사용
alias ccs="claude --model sonnet"         # Sonnet 모델 강제 사용
alias cch="claude --model haiku"          # Haiku 모델 (경량 작업)

# 자주 쓰는 작업 단축키
alias cc-review="claude --print 'PR diff를 검토하고 코드 품질 이슈를 정리해줘'"
alias cc-test="claude --print '변경된 코드에 대한 단위 테스트를 작성해줘'"
alias cc-commit="claude --print '현재 staged 변경사항으로 커밋 메시지를 작성해줘'"
EOF

source ~/.zshrc
```

alias 활용 예시:

```
# 일반 작업
cc

# 빠른 코드 리뷰 (대화 없이 출력만)
git diff | ccp "이 diff에서 버그 가능성이 있는 부분을 찾아줘"

# 커밋 메시지 자동 생성
git diff --cached | cc-commit

# 무거운 설계 작업은 Opus
ccm
```

모델 선택 전략

Claude Code는 여러 모델 중 하나를 선택해 사용합니다. 작업 성격에 따라 적절한 모델을 선택하면 구독 사용량과 속도를 최적화할 수 있습니다.

실전에서는 대부분의 작업을 **Sonnet**(균형 잡힌 속도와 품질)으로 처리합니다. 아키텍처 설계나 복잡한 장애 분석처럼 깊은 추론이 필요한 작업에만 **Opus**를 씁니다. **Haiku**는 매일 밤 실행되는 자동화 스크립트나 단순 변환 작업에 사용합니다.

Pro 플랜(\$20/월)은 대화형 작업에 적합하지만, 크론 자동화를 본격적으로 병행하면 사용량이 부족할 수 있습니다. 자동화를 많이 쓴다면 Max 플랜(\$100/월)도 고려하십시오.

모델 별칭은 `opus`, `sonnet`, `haiku` 처럼 간단하게 쓸 수 있습니다. 정확한 모델명(예: `claude-sonnet-4-5`)을 쓸 수도 있지만, 별칭을 사용하면 최신 버전이 자동으로 선택됩니다.

Before / After 비교

Before: Level 1 환경

```
$ claude
> 이 Spring Boot 컨트롤러에 유효성 검사 추가해줘
Claude: 어떤 유효성 검사가 필요하신가요? 프로젝트에서 어떤 검증 라이브러리를 쓰시나요?
> jakarta.validation 써
Claude: 알겠습니다. 그런데 이 프로젝트의 아키텍처 패턴이 어떻게 되나요?
> Hexagonal Architecture야
Claude: 그렇군요. 도메인 레이어에 검증 로직을 넣어야 하는지 컨트롤러 레이어에 넣어야 하는지...
```

매번 컨텍스트를 설명하는 데 5~10개의 메시지를 소비합니다.

After: Level 4 환경

```
$ cc
> 이 Spring Boot 컨트롤러에 유효성 검사 추가해줘
Claude: (CLAUDE.md에서 Hexagonal Architecture, jakarta.validation 사용 확인)
컨트롤러 레이어에 @Valid 어노테이션을 추가하고, 도메인 모델에는 별도
검증 로직을 유지하는 방식으로 작성하겠습니다.
[바로 코드 작성 시작]
```

CLAUDE.md가 프로젝트 컨텍스트를 미리 전달하므로 첫 메시지부터 작업에 집중합니다.

```
> 로그인 후 대시보드 진입까지 E2E 테스트해줘
Claude: (Playwright MCP로 브라우저 직접 실행)
http://localhost:3000으로 접속했습니다.
로그인 폼에 테스트 계정을 입력하고...
대시보드 진입 확인 완료. 총 3개의 UI 이슈를 발견했습니다.
```

MCP 덕분에 Claude Code가 직접 브라우저를 제어하고 결과를 보고합니다.

체크리스트

실습을 모두 마쳤다면 다음을 확인하세요.

- `~/.claude/CLAUDE.md` 가 존재하고, 전역 규칙이 작성되어 있다
- 현재 프로젝트 루트에 `CLAUDE.md` 가 존재하고, 기술 스택과 아키텍처 규칙이 작성되어 있다
- `claude` 실행 후 `/mcp` 명령으로 MCP 서버가 연결되어 있음을 확인했다
- `~/.claude/settings.json` 에 최소 1개 이상의 hook이 설정되어 있다
- `~/.zshrc` 에 `cc` alias가 설정되어 있다
- `source ~/.zshrc` 로 alias를 활성화했다

자주 하는 실수와 해결법

Q: CLAUDE.md를 작성했는데 Claude가 무시하는 것 같습니다.

A: CLAUDE.md의 내용이 너무 많거나 모순된 규칙이 있을 때 발생합니다. 파일이 너무 길면 Claude가 중간 내용을 놓칩니다. 섹션별로 나누고, 각 규칙은 3줄 이내로 간결하게 작성하세요. 또한 “절대 금지” 항목은 별도 섹션으로 강조해서 배치하면 효과적입니다.

Q: MCP 서버를 설정했는데 /mcp에 표시되지 않습니다.

A: 다음을 순서대로 확인하세요. (1) JSON 문법 오류가 없는지 확인 (`cat ~/.claude.json | python3 -m json.tool`), (2) MCP 패키지가 전역으로 설치되어 있는지 확인 (`which npx`), (3) Claude Code를 완전히 종료 후 재시작.

Q: hooks 설정 후 Claude Code가 매우 느려졌습니다.

A: `PreToolUse` 에 사용자 승인 hooks를 설정했다면, 모든 도구 사용마다 승인을 기다립니다. 해당 hook을 제거하거나 `matcher` 를 좁혀서 특정 도구에만 적용하세요.

Q: 프로젝트 CLAUDE.md와 글로벌 CLAUDE.md가 충돌하면 어떻게 됩니까?

A: 프로젝트 CLAUDE.md가 글로벌 CLAUDE.md를 덮어씁니다. 따라서 프로젝트별로 예외적인 규칙이 있다면 프로젝트 CLAUDE.md에 명시하세요. 글로벌에 “Co-Authored-By 제거”라고 해도, 특정 프로젝트에서 “Co-Authored-By 필수”라고 프로젝트 CLAUDE.md에 쓰면 프로젝트 규칙이 우선합니다.

정리

이 챕터에서 구축한 환경은 다음 챕터들의 기반이 됩니다.

- **글로벌 CLAUDE.md**: 저의 개발 원칙을 Claude가 항상 알고 있습니다.
- **프로젝트 CLAUDE.md**: 프로젝트 진입과 동시에 도메인 지식이 전달됩니다.
- **MCP 서버**: Claude Code가 코드 생성을 넘어 실제 시스템과 상호작용합니다.
- **hooks**: 반복 작업이 자동으로 처리됩니다.
- **alias + 모델 전략**: 작업 성격에 맞는 모델을 빠르게 선택합니다.

Claude Code에는 이 외에도 다양한 실험적 기능이 추가되고 있습니다. 개발 환경이 안정된 후에 하나씩 탐험해 보십시오.

다음 챕터에서는 “프롬프트가 아닌 프로토콜”로 사고방식을 전환하는 방법을 다룹니다. CLAUDE.md를 잘 작성했다면 프롬프트 엔지니어링에 소비하는 시간이 절반 이하로 줄어드는 것을 경험할 것입니다.

CH03. 프로토콜이 프롬프트를 이긴다 — CLAUDE.md 설계

같은 질문, 다른 결과

2024년 말, 처음으로 이 실험을 해봤습니다.

두 개의 Claude Code 세션을 열고, 완전히 동일한 프롬프트를 입력했습니다.

회원가입 API를 만들어줘.

첫 번째 세션은 아무 설정도 없는 빈 상태였습니다.

두 번째 세션에는 프로젝트 `CLAUDE.md` 가 있었습니다.

두 결과를 비교하는 순간, 모든 것이 명확해졌습니다.

첫 번째 세션이 만든 코드는 기교 없는 교과서였습니다. 프레임워크 기본 튜토리얼에서 그대로 옮긴 것 같은 구조. 비밀번호 해싱, DB 엔티티, 기본 응답 포맷. 틀린 건 없었습니다. 하지만 저희 프로젝트에는 쓸 수 없었습니다.

두 번째 세션이 만든 코드는 달랐습니다. 헥사고날 아키텍처로 레이어가 분리되어 있었고, 팀에서 쓰는 에러 응답 포맷을 그대로 썼고, 이미 만들어진 이메일 인증 모듈을 재사용했습니다. 코드를 열어보니 이미 저희 프로젝트에서 쓰던 코드 스타일과 같았습니다.

프롬프트는 한 글자도 다르지 않았습니다.

차이를 만든 건 `CLAUDE.md` 였습니다.

프롬프트 엔지니어링의 한계

Claude Code를 처음 쓰기 시작하면, 누구나 같은 결론에 도달합니다.

“프롬프트를 더 잘 써야겠다.”

틀린 생각은 아닙니다. 명확한 요청이 더 명확한 결과를 가져오는 건 사실입니다. 하지만 이 길에는 눈에 잘 보이지 않는 함정이 있습니다.

프롬프트 최적화는 $O(n)$ 비용입니다.

요청이 늘어날수록, 프롬프트를 개선하는 노력도 비례해서 늘어납니다. 새로운 기능을 만들 때마다, 버그를 고칠 때마다, 코드 리뷰를 요청할 때마다 — 매번 컨텍스트를 다시 설명해야 합니다.

```
# 매번 이 내용을 프롬프트에 넣고 있었다
"우리 프로젝트는 hexagon 아키텍처를 써.
에러 응답은 ErrorResponse 클래스로 통일해.
CI/CD는 GitHub Actions으로 설정되어 있어서 직접 배포하면 안 돼.
테스트는 Mockito 사용하고, 통합 테스트는 @SpringBootTest로 써."
```

저도 처음에는 이걸 매번 붙여넣었습니다. 며칠이 지나자 이 프롬프트 앞부분이 제 클립보드의 고정 아이템이 되어 있었습니다.

어느 날 문득 깨달았습니다.

나는 지금 AI에게 매번 온보딩을 시키고 있다.

새 팀원이 입사할 때마다 같은 온보딩 교육을 반복하는 회사를 상상해보십시오. 이들을 함께 일해도 그 팀원이 회사 규칙을 모르고, 오늘도 처음 만난 사람처럼 대해야 한다면. 그건 팀이 아닙니다.

CLAUDE.md 는 그 반복을 끊는 도구입니다. 한 번 작성해두면 Claude는 프로젝트를 열 때마다 그 파일을 먼저 읽습니다. 프롬프트에 아무것도 쓰지 않아도, 이미 컨텍스트를 알고 있는 동료와 일하는 것처럼 시작됩니다.

이게 **$O(1)$ 프로토콜**입니다. 한 번 투자하면 이후의 모든 요청에 동일하게 적용됩니다.

4단계 진화: CLAUDE.md가 성장하는 과정

CLAUDE.md 는 한 번에 완성할 수 없습니다. 저도 처음에는 두 줄로 시작했습니다. 지금은 팀 전체의 암묵지를 담은 300줄짜리 파일이 되어 있습니다.

어떤 과정을 거쳤는지, 단계별로 보여드리겠습니다.

단계 1: CLAUDE.md 없음

상황: 아무 설정도 없이 Claude Code를 처음 사용하는 시점.

입력한 프롬프트:

회원가입 API를 만들어줘.

Claude의 응답 (예시: Spring Boot 기준):

```
// Claude가 생성한 코드 (단계 1)

@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    @PostMapping("/signup")
    public ResponseEntity<UserResponse> signup(@RequestBody SignupRequest request) {
        UserResponse user = userService.createUser(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(user);
    }
}
```

이어서 Service 레이어입니다:

```

@Service
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public UserResponse createUser(SignupRequest request) {
        if (userRepository.existsByEmail(request.getEmail())) {
            throw new RuntimeException("이미 존재하는 이메일입니다.");
        }
        User user = User.builder()
            .email(request.getEmail())
            .password(passwordEncoder.encode(request.getPassword()))
            .build();
        userRepository.save(user);
        return UserResponse.from(user);
    }
}

```

이 코드는 틀리지 않았습니다. 하지만 저희 프로젝트에 바로 붙여넣을 수 없었습니다. 이유가 여러 개였습니다.

- 에러 응답이 기본 예외로 처리됨. 팀은 커스텀 예외 체계가 있었음
- 레이어 구조가 기본 MVC 패턴. 프로젝트는 헥사고날 아키텍처 사용 중
- 이미 별도의 이메일 인증 모듈이 있었지만, Claude는 그걸 모름
- 응답 포맷이 다름. 팀은 공통 응답 래퍼 클래스로 통일되어 있었음

결국 생성된 코드를 전부 다시 고쳤습니다. Claude가 생성한 코드는 출발점이 아닌, 참고 자료 수준이었습니다.

단계 2: 기술 스택만 있는 CLAUDE.md

작성한 CLAUDE.md:

```

# 교육 플랫폼 - 사용자 서비스

## 기술 스택
- Language: Java 21
- Framework: Spring Boot 3.2
- ORM: Spring Data JPA + QueryDSL
- Database: MySQL 8.0
- Build: Gradle
- 인증: JWT (access token 30분, refresh token 7일)

```

같은 프롬프트 입력:

회원가입 API를 만들어줘.

Claude의 응답 (예시: Spring Boot 기준):

```
// Claude가 생성한 코드 (단계 2)

@RestController
@RequestMapping("/api/v1/users")
public class UserController {

    private final UserService userService;

    @PostMapping("/signup")
    public ResponseEntity<UserDto.SignupResponse> signup(
        @Valid @RequestBody UserDto.SignupRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(userService.signup(request));
    }
}
```

분명히 나아졌습니다. Java 21 문법을 쓰고, Spring Boot 3.2 스타일을 반영했습니다. `@Valid` 도 붙었고, DTO를 내부 클래스로 구성하는 방식도 더 세련됐습니다.

하지만 여전히 문제가 있었습니다.

- 헥사고널 아키텍처를 모름. 컨트롤러에서 서비스, DB 레이어로 바로 연결
- 팀 에러 처리 패턴을 모름
- 기존에 만들어진 인증 유틸리티를 활용 못함
- 응답 포맷이 팀 표준과 다름

기술 스택 정보는 “어떤 언어로 쓸지”를 알려줬지만, “어떻게 설계할지”는 알려주지 못했습니다.

단계 3: 설계 원칙이 담긴 CLAUDE.md

추가한 내용:

교육 플랫폼 - 사용자 서비스

기술 스택

- Language: Java 21
- Framework: Spring Boot 3.2
- ORM: Spring Data JPA + QueryDSL
- Database: MySQL 8.0
- Build: Gradle
- 인증: JWT (access token 30분, refresh token 7일)

아키텍처 - hexagonal 아키텍처

프로젝트는 hexagonal 아키텍처 (포트/어댑터 패턴)를 사용합니다.

레이어 구조

src/ ├── adapter/ | ├── in/web/ # REST 컨트롤러 (Inbound Adapter) | └── out/persistence/ # JPA Repository (Outbound Adapter) ├── application/ | ├── port/ | | ├── in/ # UseCase 인터페이스 | | └── out/ # Port 인터페이스 | └── service/ # 비즈니스 로직 구현 └── domain/ # 도메인 엔티티 (순수 Java, 프레임워크 의존성 없음)

명명 규칙

- UseCase 인터페이스: `RegisterUserUseCase`, `FindUserUseCase`
- Service 구현체: `RegisterUserService`, `FindUserService`
- Port 인터페이스: `LoadUserPort`, `SaveUserPort`
- Adapter: `UserPersistenceAdapter`, `UserWebAdapter`

응답 포맷

모든 API 응답은 ApiResponse<T> 래퍼를 사용합니다.

// 성공 ApiResponse.success(data); // 실패 ApiResponse.fail(errorCode, message);

에러 처리

- 비즈니스 예외: BusinessException(ErrorCode) 사용
- ErrorCode는 enum으로 관리 (예: ErrorCode.USER_NOT_FOUND)
- GlobalExceptionHandler에서 일괄 처리
- RuntimeException, IllegalArgumentException 직접 던지기 금지

이미 있는 것 재사용

- 이메일 인증: EmailVerificationService 사용 (자체 구현 금지)
- JWT 생성/검증: JwtTokenProvider 사용
- 비밀번호 암호화: PasswordUtils.encode() 사용

같은 프롬프트 입력:

회원가입 API를 만들어줘.

Claude의 응답 (예시: Spring Boot 기준):

```
// Claude가 생성한 코드 (단계 3)

// adapter/in/web/UserWebAdapter.java
@RestController
@RequestMapping("/api/v1/users")
@RequiredArgsConstructor
public class UserWebAdapter {

    private final RegisterUserUseCase registerUserUseCase;

    @PostMapping("/signup")
    public ApiResponse<UserResponse> signup(
        @Valid @RequestBody SignupRequest request) {
        RegisteredUser user = registerUserUseCase.register(
            RegisterUserCommand.of(request.email(), request.password())
        );
        return ApiResponse.success(UserResponse.from(user));
    }
}
```

이어서 UseCase 인터페이스와 Service 구현체입니다:

```
// application/port/in/RegisterUserUseCase.java
public interface RegisterUserUseCase {
    RegisteredUser register(RegisterUserCommand command);
}

// application/service/RegisterUserService.java
@Service
@RequiredArgsConstructor
@Transactional
public class RegisterUserService implements RegisterUserUseCase {

    private final LoadUserPort loadUserPort;
    private final SaveUserPort saveUserPort;
    private final EmailVerificationService emailVerificationService; // 재사용!
```

Service 본문에서 중복 검사, 이메일 인증 확인, 유저 생성까지 처리합니다:

```

@Override
public RegisteredUser register(RegisterUserCommand command) {
    if (loadUserPort.existsByEmail(command.email())) {
        throw new BusinessException(ErrorCode.USER_EMAIL_DUPLICATED);
    }

    // 이메일 인증 확인
    emailVerificationService.validateVerified(command.email());

    User user = User.create(
        command.email(),
        PasswordUtils.encode(command.password()) // 재사용!
    );

    return RegisteredUser.from(saveUserPort.save(user));
}
}

```

이제 거의 그대로 쓸 수 있는 코드가 나왔습니다. 헥사고날 아키텍처의 레이어 구조를 지켰고, 공통 응답 래퍼를 썼고, 커스텀 예외로 에러를 처리했고, 기존 이메일 인증 모듈과 비밀번호 유틸리티를 재사용했습니다.

하지만 아직 한 단계가 더 남아 있었습니다.

단계 4: 운영 맥락이 담긴 CLAUDE.md

세 번째 단계까지는 “어떻게 코드를 쓸지”를 알려줬습니다. 네 번째 단계는 다릅니다. “무엇을 절대 하지 말아야 하는지”와 “과거에 무슨 일이 있었는지”를 담습니다.

실제로 겪었던 사고들이 이 섹션을 만들었습니다.

배포 규칙 (반드시 준수)

- 절대 프로덕션에 직접 배포하지 말 것
- Docker 이미지 직접 빌드/전송 금지
- 올바른 흐름: 코드 수정 → 커밋 → push → CI/CD 자동 배포
- 핫픽스도 반드시 PR 경유. 직접 main 브랜치 push 금지

API 게이트웨이 라우팅 규칙 (장애 이력 포함)

- **[2026-03-10 장애]**** 프론트엔드에서 /api/v1/payment/ 로 직접 호출 → Gateway 라우트 미매칭 → 401 오류
- 원인: 서비스 내부 경로(/api/v1/)와 게이트웨이 진입 경로(/api/)를 혼동
 - 게이트웨이 진입: /api/{service-name}/ 형식으로 호출
 - 게이트웨이가 RewritePath로 /api/v1/{service-name}/으로 변환
 - 프론트엔드 코드에 /api/v1/ 직접 입력 절대 금지

이어서 금지 행동 목록과 과거 장애 학습 기록입니다:

금지 행동 목록

- `RuntimeException`, `IllegalArgumentException` 직접 throw 금지 → `BusinessException` 사용
- `System.out.println` 사용 금지 → 반드시 slf4j Logger 사용
- 임의의 Secret Key, 비밀번호, API Key 생성 금지 → 사용자에게 제공받을 것
- @SpringBootTest 없이 단위 테스트에서 DB 접근 금지 → Mockito로 모킹
- 커밋 메시지에 "Co-Authored-By: Claude" 추가 금지

과거 실수 & 학습

- [2026-01-15] 결제 서비스 트랜잭션 경계 실수: @Transactional이 public 메서드에만 적용됨을 무시하고 private 메서드에 붙임 → 트랜잭션 미적용. 반드시 public 메서드에 @Transactional 적용
- [2026-02-03] N+1 쿼리 문제: 연관 엔티티 Lazy Loading 기본값으로 설정했다가 리스트 조회 시 N+1 발생. 컬렉션 조회 시 반드시 fetch join 또는 @BatchSize 사용
- [2026-02-28] 동시성 문제: 쿠폰 발급 로직에서 재고 확인과 차감이 분리되어 있어 중복 발급 발생. 재고 관련 로직은 반드시 DB 레벨 락(SELECT ... FOR UPDATE) 또는 분산 락 사용

빌드 & 테스트 실행

빌드

./gradlew build -x test

전체 테스트

./gradlew test

특정 테스트만

./gradlew test -tests "com.example.user.*"

로컬 실행

`./gradlew bootRun -args='-spring.profiles.active=local'`

현재 알려진 실패 테스트

- `ConcurrencyTest`: 분산 락 미설정 환경에서 의도적으로 실패. 관련 변경 없으면 무시 가능

같은 프롬프트 입력:

회원가입 API를 만들어줘.

Claude의 응답 (주요 차이점만 발췌, Spring Boot 기준):

```
// application/service/RegisterUserService.java
@Service
@RequiredArgsConstructor
@Transactional // public 메서드에 적용 (내부 private에 두지 않음)
public class RegisterUserService implements RegisterUserUseCase {

    private static final Logger log =
        LoggerFactory.getLogger(RegisterUserService.class);
    // System.out.println 대신 Logger 사용

    @Override
    public RegisteredUser register(RegisterUserCommand command) {
        if (loadUserPort.existsByEmail(command.email())) {
            throw new BusinessException(ErrorCode.USER_EMAIL_DUPLICATED);
            // RuntimeException 대신 BusinessException 사용
        }
        // ...
        log.info("회원가입 완료: email={}", command.email());
        return RegisteredUser.from(saveUserPort.save(user));
    }
}
```

그리고 응답 마지막에 이런 주석이 붙었습니다.

[참고] 배포 시 빌드 후 git push만 하면 됩니다.
Docker 이미지 직접 빌드는 하지 않습니다.

Claude가 과거 장애를 기억하고, 금지 행동을 피하고, 운영 맥락까지 반영한 코드를 만들었습니다. 이게 네 번째 단계의 `CLAUDE.md` 가 만드는 차이입니다.

CLAUDE.md를 쓰는 두 가지 방법

`CLAUDE.md` 파일은 두 곳에 둘 수 있습니다.

홈 디렉토리 (`~/ .claude/CLAUDE.md`): 모든 프로젝트에 공통으로 적용됩니다. 개인의 작업 스타일, 커밋 메시지 선호도, 절대 하지 않을 행동 목록을 여기에 씁니다.

프로젝트 루트 (`{project}/ .claude/CLAUDE.md` 또는 `{project}/CLAUDE.md`): 해당 프로젝트에만 적용됩니다. 기술 스택, 아키텍처 원칙, 배포 규칙, 과거 장애 이력을 여기에 씁니다.

두 파일이 동시에 있으면 Claude는 둘 다 읽습니다. 홈 디렉토리 설정 위에 프로젝트 설정이 덮어쓰여지는 구조입니다.

<code>~/ .claude/CLAUDE.md</code>	← 모든 프로젝트 공통 규칙
<code>{project}/CLAUDE.md</code>	← 이 프로젝트 전용 규칙

저는 홈 디렉토리 `CLAUDE.md` 에 이런 내용을 담아두었습니다.

```
# 전역 규칙

## 커밋 스타일
- Co-Authored-By 줄 붙이지 않기 (Claude 서명 제거)
- 한글 커밋 메시지 허용
- feat/fix/refactor/docs/chore 프리픽스 사용

## 절대 하지 않을 행동
- 임의의 Secret Key, API Key, 비밀번호 생성 금지 → 사용자에게 제공받을 것
- 프로덕션 직접 배포 금지 → CI/CD 경유
- 확인 없이 파일 삭제 금지

## 코드 검색 전략
- 정확한 키워드를 아는 경우: Grep, Glob 사용
- 개념/기능으로 찾는 경우: semantic_search 사용
- Grep으로 못 찾았을 때: semantic_search로 재시도
```

MEMORY.md: 대화가 끊겨도 기억을 잇는 시스템

`CLAUDE.md` 가 “원칙”을 담는다면, `MEMORY.md` 는 “맥락”을 담습니다.

Claude Code는 각 대화 세션이 독립적입니다. 어제 나는 대화, 지난주에 결정한 것, 한 달 전에 겪은 장애를 오늘날의 Claude는 모릅니다. 이 간극을 메우는 게 `MEMORY.md` 입니다.

Claude Code에는 자동 메모리 기능이 있습니다. `/memory` 명령어를 실행하거나, 대화 중 “이걸 기억해줘”라고 하면 Claude가 스스로 내용을 `MEMORY.md` 에 기록합니다.

하지만 저는 처음부터 구조를 정해두고 직접 관리하는 방식을 선호합니다. 자동으로 기록된 내용이 언제나 유용하지는 않기 때문입니다.

저의 `MEMORY.md` 구조입니다.

```
# 프로젝트 메모리

## 최근 결정 사항
- [2026-03-15] 결제 오류 재시도 전략: 최대 3회, 간격 5초 → 지수 백오프로 변경 결정
- [2026-03-20] MySQL → PostgreSQL 마이그레이션 검토 결론: 비용 대비 효과 없음, 보류
- [2026-04-01] 온보딩 플로우 간소화: 5단계 → 3단계로 축소

## 진행 중인 작업
- 쿠폰 발급 동시성 이슈 수정 중 (예상 완료: 이번 주 금요일)
- 사용자 서비스 성능 최적화: 로그인 API 레이턴시 200ms → 80ms 목표

## 알려진 기술 부채
- UserQueryService: 쿼리 20개 이상 → QueryDSL 리팩토링 필요
- PaymentWebAdapter: 에러 처리 패턴 혼재 → BusinessException 통일 필요
- 알림 발송 로직: 재시도 없음 → 메시지 큐 도입 필요

## 환경 정보
- 로컬 DB: localhost:3306, 스키마: platform_local
- 개발 서버: dev.internal.example.com
- 프로덕션 SSH: ssh -p 2222 deploy@prod.example.com
```

이 파일이 있으면, 새 세션에서 Claude에게 “MEMORY.md 읽어줘”라고 하는 것만으로 지난 맥락을 모두 이어받습니다.

더 강력하게 쓰는 방법은 `CLAUDE.md` 에 이 지시를 넣는 것입니다.

세션 시작 프로토콜

새 작업을 시작할 때 반드시 다음 파일을 먼저 읽어라:

1. MEMORY.md - 최근 결정 사항과 진행 중인 작업 확인
2. 열린 이슈 목록 (ops/open.md) 확인

이 두 줄이 있으면 Claude는 세션을 시작할 때마다 자동으로 맥락을 로드합니다. 대화가 끊겨도 이전 팀원이 남긴 인수인계 문서를 읽는 것처럼 업무를 이어갑니다.

“금지 행동” 패턴: 가장 중요한 섹션

CLAUDE.md 에서 가장 많은 장애를 막은 섹션은 “무엇을 하라”가 아니라 “무엇을 하지 마라”였습니다.

이유가 있습니다. Claude는 요청을 받으면 완수하려는 성향이 강합니다. “배포해줘”라고 하면, 방법을 스스로 찾아서 실행합니다. CLAUDE.md에 금지 행동이 명시되어 있지 않으면, 때로는 예상치 못한 방법으로 요청을 완수합니다.

실제로 겪었던 사례입니다.

요청

프로덕션 서버에 최신 버전 배포해줘.

CLAUDE.md 없이 Claude의 행동

1. 프로젝트 빌드 실행
2. Docker 이미지 빌드: `docker build -t service:latest` .
3. SSH 접속: `ssh deploy@prod.example.com`
4. 이미지 전송: `docker save | ssh ...`
5. 컨테이너 재시작

CI/CD 파이프라인을 완전히 우회한 직접 배포였습니다. 배포 이력도 남지 않고, 롤백 포인트도 없었습니다.

그 다음날, CLAUDE.md 에 두 줄을 추가했습니다.

배포 금지 행동

- Docker 이미지 직접 빌드/전송 금지
- SSH로 프로덕션 서버에 직접 파일 전송 금지
- 올바른 흐름: 코드 수정 → `git commit` → `git push` → CI/CD 자동 실행

이 두 줄이 추가된 이후, 수십 번의 배포 요청에서 Claude는 단 한 번도 직접 배포를 시도하지 않았습니다.

금지 행동 목록을 작성할 때는 다음 원칙을 따릅니다.

원칙 1: 실제로 일어난 것만 씁니다. 이론적으로 위험한 것을 모두 나열하면 Claude가 과도하게 제한됩니다. 실제로 문제가 됐던 행동을 사고가 발생한 직후에 기록합니다.

원칙 2: 이유를 함께 씁니다. “X 하지 마라”보다 “X 하면 Y 장애 발생하므로 금지”가 더 잘 지켜집니다. Claude도 이유를 알면 유사한 상황에서 더 잘 판단합니다.

원칙 3: 대안을 함께 제시합니다. “하지 마라”만 있으면 Claude가 멈춥니다. “대신 이렇게 해라”를 함께 써야 작업이 계속됩니다.

```
## 에러 처리 금지 행동
- RuntimeException 직접 throw 금지
  → 이유: GlobalExceptionHandler가 BusinessException만 처리. RuntimeException은 500으로 노출
  됨
  → 대안: throw new BusinessException(ErrorCode.적절한_코드) 사용
```

실전 CLAUDE.md 템플릿

지금까지의 내용을 바탕으로, 복사해서 바로 쓸 수 있는 템플릿을 정리했습니다. 이 구조로 시작해서 프로젝트에 맞게 채워나가면 됩니다.

```
# {프로젝트명} - {서비스명}
```

```
## 프로젝트 개요
```

- 서비스 목적: {한 줄 설명}
- 현재 단계: {개발 중 / 운영 중 / 리팩토링 중}

```
## 기술 스택
```

- Language: {사용 중인 언어}
- Framework: {사용 중인 프레임워크}
- Database: {사용 중인 DB}
- Cache: {Redis 등, 있는 경우}
- Build: {빌드 도구}
- 인프라: {Docker / K8s / EC2 등}

```
## 아키텍처 원칙
```

```
{아키텍처 다이어그램 또는 레이어 설명}
```

```
예시 (헥사고нал):
```

```
...
```

```
src/
```

```
|— adapter/in/web/          # REST 컨트롤러
|— adapter/out/persistence/ # DB Adapter
|— application/port/in/     # UseCase 인터페이스
|— application/port/out/    # Port 인터페이스
|— application/service/     # 비즈니스 로직
|— domain/                  # 도메인 엔티티
|— ...
```

이어서 코드 컨벤션과 배포 규칙입니다:

```
## 코드 컨벤션
```

```
### 응답 포맷
```

```
{공통 응답 래퍼 등 팀 표준 설명}
```

```
### 에러 처리
```

```
{커스텀 예외 / 에러 코드 구조 설명}
```

```
### 재사용 모듈 (직접 구현 금지)
```

- {이메일 인증 모듈}: 이메일 인증
- {토큰 프로바이더}: JWT 발급/검증
- {비밀번호 유틸}: 비밀번호 암호화

```
## 배포 규칙 (반드시 준수)
```

- 올바른 흐름: 코드 수정 → git commit → git push → CI/CD 자동 실행
- Docker 이미지 직접 빌드/전송 금지
- 프로덕션 서버 직접 접근 금지 (긴급 장애 시 제외)
- 핫픽스도 PR 경유 필수

금지 행동, 과거 장애 기록, 빌드 명령어 섹션입니다:

금지 행동 목록

- 기본 예외 직접 throw 금지 → 커스텀 예외 사용
- print/console.log 디버깅 금지 → 로거 사용
- 임의의 API Key, Secret, 비밀번호 생성 금지 → 사용자에게 제공받을 것
- {프로젝트 특화 금지 사항 추가}

과거 장애 & 학습

- [{날짜}] {장애 요약}: {원인 설명} → {해결책 및 재발 방지}
- {계속 추가}

알려진 실패 테스트

- {TestClassName}: {실패 이유} - 관련 변경 없으면 무시 가능

빌드 & 실행

``

빌드

{빌드 명령어}

테스트

{테스트 명령어}

로컬 실행

{실행 명령어}

``

세션 시작 프로토콜

새 작업 시작 시 먼저 읽을 파일:

1. MEMORY.md - 최근 결정 사항 확인
2. {추가 파일이 있으면 명시}

Before/After: 같은 작업, 다른 경험

CLAUDE.md가 없을 때와 있을 때의 경험 차이를 정직하게 비교해 보겠습니다.

시나리오: 새로운 알림 발송 기능 추가 요청

CLAUDE.md가 없으면 매번 8~15줄의 컨텍스트를 프롬프트에 넣어야 하고, 생성된 코드를 1~2시간 수정해야 합니다. CLAUDE.md가 있으면 1~2줄만 입력해도 기존 패턴에 맞는 코드가 나오고, 수정 시간이 10~20분으로 줄어듭니다. 아키텍처 일관성도 세션마다 들쭉날쭉하던 것이 항상 동일하게 유지됩니다.

저의 경험 기준으로, CLAUDE.md 없이 작업할 때 생성된 코드의 평균 재사용률은 약 40%였습니다. 단계 4 CLAUDE.md를 갖추고 나서는 약 85%로 올라갔습니다. 나머지 15%는 비즈니스 로직의 특수한 부분이었습니다.

프롬프트에 투자한 시간과 CLAUDE.md에 투자한 시간의 ROI는 비교가 안 됩니다.

프롬프트 개선: 이번 요청 하나에만 적용됩니다.

CLAUDE.md 개선: 앞으로의 모든 요청에 적용됩니다.

실습: 지금 당장 CLAUDE.md 작성하기

이론은 충분합니다. 지금부터 여러분의 프로젝트에 CLAUDE.md를 만들어 보겠습니다.

실습 1: 빠른 시작 (15분)

빈 `CLAUDE.md` 파일을 프로젝트 루트에 만들고, 다음 세 섹션만 채웁니다.

```
# {내 프로젝트}

## 기술 스택
- {현재 사용 중인 언어, 프레임워크, DB를 나열}

## 금지 행동
- {지금 당장 생각나는 "이건 절대 하면 안 돼" 목록}

## 빌드 & 실행
{빌드와 로컬 실행 명령어}
```

이 세 섹션만 있어도 없는 것보다 훨씬 낫습니다.

실습 2: 아키텍처 원칙 추가 (30분)

기존 코드를 열고, 실제 디렉토리 구조를 CLAUDE.md에 복사합니다. 레이어 간 의존 방향, 명명 규칙, 응답 포맷을 기록합니다.

Claude에게 이렇게 물어보면 도움이 됩니다.

```
내 프로젝트 구조를 분석해서 CLAUDE.md의 아키텍처 섹션 초안을 작성해줘.
```

실습 3: 장애 이력 기록 (30분)

지난 3개월 동안 발생했던 버그, 장애, 예상치 못한 동작을 되돌아봅니다. 각각에 대해 이렇게 기록합니다.

- [{날짜}] {무슨 일이 있었나}: {왜 발생했나} → {다시 발생하지 않으려면}

세 줄이면 충분합니다. 완벽하게 쓰려 하지 마십시오. 일단 기록하는 것이 중요합니다.

실습 4: Before/After 비교 실험

동일한 기능 요청을 두 세션에서 실행합니다.

1. 새 터미널에서 CLAUDE.md 없이 실행
2. CLAUDE.md가 있는 프로젝트 디렉토리에서 실행

두 결과를 비교하면, 이 챕터에서 설명한 차이를 직접 체감할 수 있습니다.

CLAUDE.md 관리의 황금률

마지막으로, 몇 달간 CLAUDE.md를 관리하면서 체득한 황금률을 공유합니다.

황금률 1: 장애 직후에 기록하라.

사고가 나고 시간이 지나면 기억이 희미해집니다. 장애를 해결한 직후, 5분 안에 CLAUDE.md에 기록합니다. 이 습관이 가장 중요합니다.

황금률 2: 완벽하게 쓰려 하지 마라.

처음부터 완벽한 CLAUDE.md를 쓰려고 하면 작성 자체를 미루게 됩니다. 일단 틀린 내용이라도 씁니다. 틀린 것을 고치면서 더 좋아집니다.

황금률 3: Claude에게 물어라.

CLAUDE.md 작성이 막힌다면 Claude에게 이렇게 물어볼 수 있습니다.

내 프로젝트를 보고, 현재 CLAUDE.md에 빠진 중요한 내용이 뭔지 알려줘.
그리고 추가할 섹션 초안을 작성해줘.

Claude가 코드를 분석해서 빠진 컨벤션, 패턴, 주의사항을 찾아줍니다.

황금률 4: 팀이 있다면 공유하라.

혼자 개발하더라도 미래의 자신은 팀원입니다. 6개월 후에 이 프로젝트를 다시 열 때, CLAUDE.md는 과거의 당신이 남긴 최고의 문서가 됩니다.

요약

이 챕터를 한 문장으로 요약하면 이렇습니다.

프롬프트를 잘 쓰는 것은 $O(n)$ 투자입니다. CLAUDE.md를 잘 쓰는 것은 $O(1)$ 투자입니다.

프롬프트 엔지니어링을 공부하는 시간의 절반만 CLAUDE.md 작성에 투자해도, Claude와 일하는 품질은 몇 배로 뛰어오릅니다.

다음 챕터에서는 CLAUDE.md 위에 에이전트 시스템을 쌓는 방법을 다룹니다. 원칙이 설정됐다면, 이제 그 원칙 위에서 자율적으로 판단하고 실행하는 에이전트를 설계할 차례입니다.

CH04. 에이전트 아키텍처 — 마크다운 파일 하나가 직원이 된다

구두 지시 vs. 업무 매뉴얼

처음 팀을 만든 창업자들이 가장 많이 하는 실수가 있습니다.

모든 것을 “그때그때 말로 한다.”

“오늘 이 기능 좀 만들어줘.” “이 버그 고쳐줘.” “아 그리고 이걸 이렇게 해야 해.” 팀원 5명에게 각자 다른 말로 지시하면, 결과물은 제각각이 됩니다. 팀장이 자리를 비우면 일이 멈춥니다. 같은 온보딩을 새로 들어오는 사람마다 반복해야 합니다.

반대로 잘 돌아가는 팀은 매뉴얼이 있습니다.

“코드 리뷰 기준은 이 문서에 있어요.” “배포 절차는 여기 적혀 있어요.” “이 기능은 저 패턴으로 구현해요.” 팀원 누구든 같은 기준으로 일하고, 신규 입사자도 문서만 읽으면 시작할 수 있습니다.

Claude Code의 에이전트 시스템은 정확히 이 구조입니다.

프롬프트를 치는 건 구두 지시입니다. 에이전트를 만드는 건 업무 매뉴얼을 만드는 것입니다.

에이전트란 무엇인가

Claude Code에서 에이전트는 단순합니다. `.claude/agents/` 디렉토리에 있는 마크다운 파일 하나입니다.

```
.claude/
└─ agents/
    ├── backend-engineer.md
    ├── frontend-engineer.md
    ├── qa-engineer.md
    └─ code-reviewer.md
```

파일 하나가 하나의 “직원”입니다.

이 파일 안에는 세 가지가 들어갑니다.

첫째, **메타데이터**: 에이전트의 이름, 어떤 때 자동으로 호출되는지, 어떤 도구를 쓸 수 있는지.

둘째, **역할 정의**: 이 에이전트가 무엇을 하는 사람인지.

셋째, **업무 매뉴얼**: 어떤 원칙을 지키고, 어떤 패턴을 따르며, 무엇을 절대 하지 않는지.

한 번 작성해두면 이후에는 Claude가 알아서 이 파일을 참조합니다. 매번 컨텍스트를 설명할 필요가 없습니다.

“코드 리뷰해줘”라고 하면 Claude는 `code-reviewer.md` 를 읽고, 거기 적힌 기준으로 리뷰합니다.

만능 에이전트 1개 < 전문 에이전트 5개

에이전트를 처음 만들 때 가장 흔한 실수는 하나의 에이전트에 모든 것을 넣으려는 것입니다.

```
# 슈퍼 에이전트
- 백엔드 코드를 작성한다
- 프론트엔드 코드도 작성한다
- 테스트도 작성한다
- 코드 리뷰도 한다
- 배포도 검증한다
- 문서도 작성한다
```

이렇게 만든 에이전트는 실제로 사용하면 문제가 생깁니다.

컨텍스트 오염: 백엔드 패턴과 프론트엔드 패턴이 뒤섞이면 Claude가 어느 맥락에서 일해야 하는지 혼란스러워합니다. 백엔드 API를 설계할 때 React 컴포넌트 패턴을 섞어서 제안하는 일이 생깁니다.

프롬프트 과부하: 파일이 길어질수록 중요한 지침이 희석됩니다. 50줄 매뉴얼의 30번째 줄에 있는 규칙은 5줄 매뉴얼의 3번째 줄에 있는 규칙보다 덜 지켜집니다.

재사용 불가: 특정 프로젝트에만 쓸 수 있는 에이전트가 됩니다. 새 프로젝트를 시작하면 처음부터 다시 만들어야 합니다.

아래 두 구조를 비교해보겠습니다.

```
# 구조 A - 만능 에이전트 1개

.claude/agents/
└─ everything.md          ← 500줄, 모든 것을 다 함

# 구조 B - 전문 에이전트 5개

.claude/agents/
├─ backend-engineer.md    ← hexagon 아키텍처, Java/Kotlin 전문
├─ frontend-engineer.md   ← React, 컴포넌트 설계 전문
├─ qa-engineer.md         ← 테스트 전략, 커버리지 전문
├─ code-reviewer.md       ← 코드 품질, 보안, 성능 리뷰 전문
└─ deploy-verifier.md     ← 배포 전후 검증 전문
```

구조 B에서 각 에이전트는 자신의 도메인에만 집중합니다. 백엔드 엔지니어 에이전트는 hexagon 아키텍처 패턴만 생각합니다. QA 엔지니어 에이전트는 테스트 품질만 생각합니다. 역할이 명확하니 결과물도 명확합니다.

소프트웨어 설계의 단일 책임 원칙(SRP)이 에이전트에도 똑같이 적용됩니다.

에이전트 역할맵

제가 실제로 운영하는 에이전트 구조입니다. 서비스명은 추상화했습니다.

메인 세션 (당신 = 사장)

[개발 팀]	
└ backend-engineer	← Spring/Kotlin, 핵사고날 아키텍처
└ frontend-engineer	← React/Next.js, 컴포넌트 설계
└ flutter-engineer	← 모바일 앱, 클린 아키텍처
[품질 팀]	
└ code-reviewer	← 코드 품질, 보안, 성능
└ qa-engineer	← 테스트 전략, 시나리오 설계
└ playwright-test-gen	← E2E 테스트 자동 작성
[운영 팀]	
└ deploy-verifier	← 배포 전후 상태 검증
└ session-writer	← 작업 로그, 의사결정 기록
[콘텐츠 팀]	
└ blog-researcher	← 기술 블로그 조사, 정보 수집
└ portfolio-builder	← 포트폴리오 자동 생성

각 에이전트가 독립적으로 존재하고, 메인 세션(당신)이 오케스트레이터 역할을 합니다. “백엔드 API 만들고 QA해줘”라고 하면, Claude는 먼저 backend-engineer를 호출해 API를 만들고, qa-engineer를 호출해 테스트를 작성합니다.

실전: 에이전트 마크다운 파일 해부

에이전트 파일은 두 부분으로 구성됩니다. YAML 프론트매터(←로 감싸진 메타데이터)와 마크다운 본문입니다.

예시 1: 코드 리뷰어 에이전트

```
---
name: code-reviewer
description: 코드 품질 리뷰 전문가. PR 리뷰, 코드 스멜 탐지, 보안 취약점 확인.
  Use PROACTIVELY when user asks for code review or mentions PR, pull request, 리뷰.
tools: Read, Grep, Glob, Bash
model: sonnet
---
```

Code Reviewer

당신은 시니어 백엔드 엔지니어이자 코드 리뷰 전문가입니다.
팀의 코드 품질 기준을 지키고, 잠재적 문제를 조기에 발견하는 것이 목표입니다.

리뷰 4대 원칙

1. ****정확성****: 로직이 요구사항을 충족하는가
2. ****안전성****: SQL 인젝션, NPE, 인증 누락 등 보안/안정성 이슈는 없는가
3. ****성능****: N+1 쿼리, 불필요한 루프, 캐시 미적용 등 성능 문제는 없는가
4. ****유지보수성****: 코드가 읽기 쉬운가, 테스트 가능한가

이어서 리뷰 프로세스와 심각도 분류입니다:

리뷰 프로세스

1. 변경된 파일 목록 파악 (`git diff --name-only`)
2. 변경 범위와 의도 파악
3. 4대 원칙 기준으로 파일별 리뷰
4. 이슈를 심각도별로 분류하여 보고

심각도 분류

등급	레이블	기준
P0	[BLOCKER]	배포 불가. 보안 취약점, 데이터 유실 위험
P1	[CRITICAL]	즉시 수정 필요. 서비스 장애 가능성
P2	[MAJOR]	이번 PR에서 수정 권장
P3	[MINOR]	다음 기회에 개선
P4	[SUGGESTION]	선택적 개선 아이디어

리뷰 결과 포맷

...

코드 리뷰 결과

****전체 판정**:** [APPROVE / REQUEST_CHANGES / COMMENT]
****변경 파일**:** N개 | ****이슈 총계**:** P0(0) P1(0) P2(N) P3(N)

[파일명]

****[P2] 에러 처리 누락****

```
```java
// 현재 코드
User user = userRepository.findById(id); // Optional 처리 없음

// 권장 코드
User user = userRepository.findById(id)
 .orElseThrow(() -> new UserNotFoundException(id));
...

> 사용자가 없을 때 NPE가 발생합니다.
```

---

**\*\*총평\*\*:** {1-2문장 요약}

## 절대 하지 않는 것

- 개인 코딩 스타일을 강요하지 않는다 (팀 컨벤션 기준으로만 리뷰)

- 이미 merge된 코드의 과거 결정을 비판하지 않는다
- 완벽한 코드를 요구하지 않는다 (80점이면 배포한다)

이 파일을 ``.claude/agents/code-reviewer.md``에 저장하면 완성입니다. 이제 "이 PR 리뷰해줘"라고 하면 Claude가 이 파일을 읽고, 여기 적힌 기준으로 리뷰합니다.

### 예시 2: 백엔드 엔지니어 에이전트 (Spring Boot 기준)

팀마다 아키텍처 패턴이 다릅니다. 에이전트는 그 패턴을 기억합니다. 아래는 Spring Boot 프로젝트 기준 예시입니다. 사용하는 프레임워크에 맞게 수정하십시오.

name: backend-engineer

description: Spring Boot 백엔드 개발 전문가. hexagonal 아키텍처와 DDD 패턴을 따릅니다.

Use PROACTIVELY when implementing API, service, repository, entity.

tools: Read, Write, Edit, Grep, Glob, Bash

model: sonnet

## Backend Engineer

당신은 Spring Boot + Kotlin 백엔드 엔지니어입니다. 모든 코드는 hexagonal 아키텍처(포트 & 어댑터) 패턴을 따릅니다.

### 필수 사전 작업

새 작업 시작 전 반드시 CLAUDE.md를 읽습니다:

Read: `./CLAUDE.md`

## 패키지 구조

---

```
com.example.{service}/
├── {domain}/
│ ├── domain/
│ │ └── model/ # 순수 도메인 모델
│ ├── application/
│ │ ├── port/
│ │ │ ├── in/ # UseCase 인터페이스
│ │ │ └── out/ # Port 인터페이스
│ │ ├── {Domain}CommandService.java
│ │ └── {Domain}QueryService.java
│ └── adapter/
│ ├── in/web/ # REST Controller
│ └── out/persistence/ # JPA Adapter
```

## 구현 순서

---

도메인 → Port → Service → Adapter → Controller 순으로 구현합니다. Controller를 먼저 만들지 않습니다. 도메인 로직이 먼저입니다.

## API 응답 규칙

---

- 성공: `ApiResponse.success(data)` — HTTP 200
- 생성: `ApiResponse.success(data)` — HTTP 201 ( `@ResponseStatus(HttpStatus.CREATED)` )
- 실패: `ApiResponse.error(code, message)` — 적절한 4xx/5xx

## 테스트 규칙

---

- 단위 테스트: `@ExtendWith(MockitoExtension.class)` , Port를 Mock으로
- 통합 테스트: `@SpringBootTest` , 실제 DB 연결
- 테스트 없는 PR은 없습니다

## 절대 금지

- `./gradlew build -x test` 금지 (테스트 스킵 금지)
- Service에서 JpaRepository 직접 의존 금지
- Controller에 비즈니스 로직 작성 금지
- 직접 서버 배포 금지 (Git push → CI/CD가 자동 배포)

### 예시 3: 블로그 리서처 에이전트

개발 이외의 업무도 에이전트로 만들 수 있습니다. 기술 블로그 조사, 경쟁사 분석, 콘텐츠 작성 등.

name: blog-researcher

description: 기술 블로그 조사 전문가. "추측하지 않는다. 기술 블로그에 나온 그대로 수집한다."

Use when user asks to research tech blog, 기술 블로그 조사, 레퍼런스 수집.

tools: Read, Write, Glob, Grep, WebFetch, WebSearch

model: sonnet

## Blog Researcher

핵심 원칙: "추측하지 않는다. 기술 블로그에 나온 그대로 수집한다."

## Role

기업 기술 블로그에서 시스템 설계 정보를 정확하게 추출하는 전문가입니다.

## 검색 전략

1. "{기업명} 기술블로그 {주제}" (기본)
2. "site:{공식블로그URL} {주제}" (공식 블로그 직접)
3. "{기업명} engineering blog {주제}" (영문)

## 절대 금지

- 블로그에 없는 내용을 추측하거나 일반론을 쓰지 않는다
- 출처 없는 숫자(MAU, QPS, 규모)를 쓰지 않는다
- “보통 ~한다”, “일반적으로 ~한다” 같은 표현 금지

## 출력 포맷

```
company: 기업명
topic: 조사 주제
sources:
 - title: "글 제목"
 url: https://...
 published: YYYY-MM-DD
tech_stack:
 - name: Redis
 usage: "재고 관리, 원자적 DECR"
 source: "출처 URL"
key_insights:
 - insight: "배운 점"
 source: "출처 URL"
confidence: high / medium / low
missing_info:
 - "블로그에 없는 정보"
```

---

## 실습: 코드 리뷰어 에이전트 직접 만들기

이제 직접 만들어보겠습니다. 5분이면 됩니다.

\*\*1단계: 파일 생성\*\*

```
mkdir -p .claude/agents touch .claude/agents/code-reviewer.md
```

**\*\*2단계: 최소 구성 작성\*\***

처음부터 완벽하게 만들 필요 없습니다. 최소한 이 세 가지만 있으면 동작합니다.

name: code-reviewer

description: 코드 리뷰 전문가. PR 리뷰 요청 시 자동 호출됩니다.

tools: Read, Grep, Glob, Bash

model: sonnet

## Code Reviewer

당신은 코드 리뷰 전문가입니다.

### 리뷰 기준

1. 버그와 에러 처리 누락
2. 보안 취약점 (SQL 인젝션, 인증 누락)
3. 성능 문제 (N+1 쿼리, 불필요한 루프)
4. 코드 가독성

### 출력 포맷

파일별로 이슈를 정리하고, 심각도(BLOCKER / MAJOR / MINOR)를 표시합니다. 수정 코드 예시를 반드시 포함합니다.

**\*\*3단계: 테스트\*\***

Claude Code를 열고 타이핑합니다:

```

이 파일 리뷰해줘: src/UserService.java

```

Claude가 `code-reviewer.md`를 읽고, 거기 적힌 기준으로 리뷰를 시작합니다.

**\*\*4단계: 점진적 개선\*\***

리뷰 결과를 보면서 부족한 부분을 파악합니다. 팀 컨벤션이 빠졌다면 추가합니다. 너무 엄격하다면 기준을 조정합니다. 에이전트는 한 번에 완성하는 게 아니라 사용하면서 다듬는 것입니다.

참고로 MCP 서버가 연결되어 있다면, 에이전트가 작업 중 사용자에게 추가 정보를 요청하는 **\*\*MCP Elicitation\*\***(폼 입력 요청) 기능을 활용할 수도 있습니다. 예를 들어 코드 리뷰 에이전트가 "이 변경의 의도를 선택해주세요"라는 폼을 띄워 더 정확한 리뷰를 수행할 수 있습니다.

---

**## 스킬(Skill): 재사용 가능한 워크플로우**

에이전트와 함께 알아야 할 개념이 하나 더 있습니다. 스킬(Slash Command)입니다.

에이전트가 "어떻게 일하는가"를 정의한다면, 스킬은 "어떤 작업을 반복하는가"를 정의합니다.

`.claude/skills/` 디렉토리에 저장하며, Claude Code에서 `/skills/{이름}`으로 호출합니다.

**### 스킬 예시 1: 작업 시작 루틴**

```

name: dev-start

description: 개발 작업 시작 시 컨텍스트 로딩. "작업 시작", "오늘 뭐 하지" 등에 사용

Dev Start

개발 작업을 시작할 때 필요한 컨텍스트를 준비합니다.

실행 절차

1. 프로젝트 상태 확인

- `git status` - 미완성 작업 있는지 확인
- `git log --oneline -5` - 최근 커밋 흐름 파악

```

이어서 오픈 이슈 확인과 출력 포맷입니다:

```

2. 오픈 이슈 확인

- `.sisyphus/plans/` 또는 열린 이슈 목록(`ops/issues/open.md`) 읽기

3. 오늘 목표 요약

- 이어서 할 작업: {미완성 작업}
- 신규 작업: {오픈 이슈 중 우선순위 1위}
- 예상 소요: {작업 규모 기반 추정}

출력 포맷

```

#### ## 오늘의 작업 계획

**\*\*이어서\*\***: {미완성 작업}

**\*\*신규\*\***: {오픈 이슈 1순위}

**\*\*컨텍스트\*\***: {최근 5커밋 요약}

#### ### 스킵 예시 2: 배포 전 체크리스트

```

name: pre-deploy-check

description: 배포 전 필수 체크리스트 실행. "배포할게", "배포 전 확인" 등에 사용

Pre-Deploy Check

배포 전 반드시 확인해야 할 사항을 순서대로 점검합니다.

체크리스트

1. 빌드 검증

```

./gradlew test

```

테스트 실패가 있으면 배포 중단.

2. 환경변수 확인

- 프로덕션 환경변수 누락 없는지 확인
- 개발용 값(localhost, test-key 등)이 남아있지 않은지 확인

3. 마이그레이션 확인

- 새로운 DB 스키마 변경이 있는지 확인
- 변경이 있으면 마이그레이션 스크립트 준비 여부 확인

4. 롤백 계획

- 문제 발생 시 이전 버전으로 돌아가는 방법 확인

출력 포맷

```

#### ## 배포 전 체크리스트

- 빌드/테스트 통과
- 환경변수 확인
- DB 마이그레이션 스크립트 필요 ← 배포 전 실행 필요

**\*\*판정\*\***: {배포 가능 / 배포 보류 - 이유}

```

...

에이전트 vs 스킬 vs 프롬프트: 의사결정 트리

셋의 차이가 처음에는 헷갈립니다. 어떤 상황에 무엇을 쓸지 결정하는 기준을 정리했습니다.

...

이 작업이 반복되는가?

└─ 아니오 → 그냥 프롬프트 (일회성 질문, 즉흥적 작업)

└─ 예 → 이 작업이 특정 역할/전문성을 요구하는가?

└─ 예 → 에이전트 (backend-engineer, qa-engineer)
"이 사람이 어떻게 생각하고 일하는가"

└─ 아니오 → 이 작업이 순서가 있는 절차인가?

└─ 예 → 스킬 (dev-start, pre-deploy-check)
"이 절차를 어떤 순서로 실행하는가"

└─ 아니오 → CLAUDE.md에 규칙으로 추가
"모든 작업에서 항상 지켜야 하는 것"

...

실제 예시로 적용해보겠습니다.

| 상황 | 선택 | 이유 |
|------------------|-----------|------------------|
| "이 함수 설명해줘" | 프롬프트 | 일회성, 반복 불필요 |
| "API 만들어줘" (매번) | 에이전트 | 아키텍처 패턴, 전문성 필요 |
| "배포 전 체크해줘" (매번) | 스킬 | 순서 있는 절차, 역할 불필요 |
| "커밋 메시지는 영어로" | CLAUDE.md | 모든 작업에 항상 적용 |

서브에이전트 위임: 메인 세션에서 에이전트 호출하기

에이전트의 진짜 힘은 위임에 있습니다. 메인 세션(당신)은 오케스트레이터입니다. 무엇을 할지 결정하고, 실제 작업은 전문 에이전트에게 넘깁니다.

Claude Code는 Task 도구를 통해 서브에이전트를 호출합니다.

...

메인 세션에서의 대화

나: "인터뷰 준비 기능을 새로 만들어야 해.

백엔드 API 설계하고, 프론트엔드 컴포넌트도 만들고, E2E 테스트까지."

Claude: 알겠습니다. 세 단계로 나눠서 진행하겠습니다.

[Task → backend-engineer]

"인터뷰 준비 기능의 REST API를 설계하고 구현해줘."

[Task → frontend-engineer]

"인터뷰 준비 화면 컴포넌트를 만들어줘."

[Task → playwright-test-gen]

"인터뷰 준비 기능 E2E 테스트를 작성해줘."

...

각 서브에이전트는 ****독립된 컨텍스트****에서 실행됩니다. 백엔드 에이전트는 프론트엔드 구현 세부사항을 모릅니다. 프론트엔드 에이전트는 DB 스키마를 신경 쓰지 않습니다. 이게 컨텍스트 격리입니다.

컨텍스트 격리의 이점은 두 가지입니다.

****품질 집중****: 각 에이전트가 자신의 도메인에만 집중하므로 결과물이 더 정교합니다. 백엔드 에이전트는 비즈니스 로직과 아키텍처만 생각합니다.

****컨텍스트 윈도우 절약****: Claude의 컨텍스트 윈도우는 유한합니다. 하나의 세션에서 백엔드, 프론트엔드, 테스트를 모두 다루면 이전 대화가 밀려나가 품질이 떨어집니다. 서브에이전트는 각자 깨끗한 컨텍스트로 시작합니다.

컨텍스트 윈도우 관리: 에이전트 아키텍처의 숨은 이유

에이전트를 나누는 이유 중 가장 실용적인 것이 컨텍스트 윈도우 관리입니다.

Claude의 컨텍스트 윈도우를 짧은 단기 기억이라고 생각해봅시다. 대화가 길어질수록 앞부분 내용이 "밀려납니다." 처음에 설명한 아키텍처 원칙이 50번의 대화 후에는 희미해집니다.

...

메인 세션에서 하나로 다 처리할 때

[대화 1-10]: 백엔드 API 설계

[대화 11-30]: 프론트엔드 컴포넌트 구현

[대화 31-50]: 테스트 작성

[대화 51]: "아까 설계한 API 명세 다시 보여줘"

→ 대화 1의 내용이 밀려나 부정확한 답변...

서브에이전트로 분리할 때

[백엔드 에이전트 세션]: API 설계 집중 → 완료

[프론트엔드 에이전트 세션]: 컴포넌트 집중 → 완료

[QA 에이전트 세션]: 테스트 집중 → 완료

각 세션은 자신의 작업만 기억합니다.

...

세션을 나누면 각 에이전트는 항상 깨끗한 컨텍스트로 시작합니다. 중요한 지침이 밀려날 위험이 없습니다.

에이전트 설계의 실전 원칙

수십 개의 에이전트를 만들면서 발견한 원칙들입니다.

원칙 1: 이름이 곧 역할이다

`backend-engineer.md`라는 파일명을 보면 무엇을 하는지 즉시 알 수 있습니다. `helper.md`, `assistant.md` 같은 이름은 피합니다. 에이전트 이름은 직책명처럼 명확해야 합니다.

원칙 2: description은 호출 트리거다

`description` 필드는 Claude가 언제 이 에이전트를 자동으로 제안할지를 결정합니다. "Use PROACTIVELY when..."으로 시작하는 문장을 넣으면 더 잘 작동합니다.

...

애매한 description

description: 백엔드 개발을 도와줍니다.

명확한 description

description: Spring Boot 백엔드 개발 전문가.

Use PROACTIVELY when implementing API, service, repository, entity 관련 작업.

...

원칙 3: 절대 금지 목록은 필수다

무엇을 해야 하는지만큼 무엇을 하지 말아야 하는지가 중요합니다. 특히 안전 관련 사항은 명시적으로 적어둡니다.

...

절대 금지

- 직접 서버 배포 금지 (Git push → CI/CD 자동 배포)
- 테스트 스킵 금지 (`-x test` 옵션 사용 금지)
- 프로덕션 DB 직접 수정 금지

...

원칙 4: 에이전트는 살아있는 문서다

처음 작성한 에이전트가 완벽한 경우는 없습니다. 쓰다보면 빠진 규칙, 잘못된 가정, 불필요한 내용이 보입니다. 분기마다 에이전트 파일을 리뷰하고 업데이트합니다.

...

에이전트 파일 최근 수정일 확인

ls -lt .claude/agents/

...

오래전에 수정한 에이전트는 낡은 패턴을 담고 있을 수 있습니다.

원칙 5: 작게 시작한다

처음부터 완벽한 에이전트를 만들려다 보면 아무것도 만들지 못합니다. 최소한의 역할 정의와 절대 금지 목록으로 시작하고, 실제 사용하면서 채워 나갑니다.

최소 에이전트 템플릿

| |
|--|
| name: {이름} |
| description: {한 줄 역할 설명}. Use PROACTIVELY when {트리거 상황}. |
| tools: Read, Write, Edit, Grep, Glob, Bash |
| model: sonnet |

{이름}

{역할 정의 2-3문장}

핵심 원칙

1. {원칙 1}
2. {원칙 2}
3. {원칙 3}

절대 금지

- {금지 사항 1}
- {금지 사항 2} ````

에이전트 팀 구축 로드맵

에이전트를 한 번에 다 만들 필요 없습니다. 단계적으로 구축합니다.

1주차: 가장 많이 쓰는 작업 1개

어떤 작업을 가장 자주 하나요? 백엔드 API 개발이라면 `backend-engineer.md` 부터 만듭니다. 코드 리뷰를 많이 한다면 `code-reviewer.md` 부터. 하나를 제대로 만드는 게 다섯을 대충 만드는 것보다 낫습니다.

2~3주차: 품질 팀 추가

개발 에이전트가 안정되면 품질 팀을 추가합니다. `qa-engineer.md` , `code-reviewer.md` . 이 두 에이전트가 있으면 코드를 작성하는 즉시 검증할 수 있습니다.

1개월차: 운영 팀 추가

`deploy-verifier.md` , `session-writer.md` . 배포 전 체크리스트를 자동화하고, 작업 로그를 자동으로 남깁니다.

이후: 콘텐츠 팀, 분석 팀

비즈니스 요구에 따라 확장합니다. 블로그 글을 자주 쓴다면 `blog-writer.md` . 경쟁사 분석이 필요하다면 `market-researcher.md` .

정리: 에이전트 아키텍처의 본질

에이전트 아키텍처를 한 문장으로 정리하면 이렇습니다.

“당신의 암묵지를 마크다운 파일에 담아 팀원으로 만드는 것.”

5년 동안 쌓은 아키텍처 경험, 수십 번의 장애 대응으로 생긴 배포 직관, 코드 리뷰하면서 발견한 패턴들. 이것을 에이전트 파일에 담으면, Claude는 그 경험을 가진 사람처럼 일합니다.

프롬프트는 그 순간에만 작동합니다. 에이전트는 영원히 작동합니다.

“코드 리뷰해줘”라는 한 마디에, 5년의 경험을 담은 리뷰어가 나타납니다.

다음 챕터에서는 에이전트와 크론(cron)을 결합해 자동화 인프라를 구축합니다. 사람 없이 매일 밤 돌아가는 코드 품질 검사, 새벽에 자동으로 오는 서비스 상태 리포트. Claude가 스스로 깨어나 일하는 시스템을 만드는 방법을 다룹니다.

CH05. 자동화 인프라: Claude를 크론으로 돌리다

매일 아침 9시

슬랙 알림이 옵니다.

[DAILY REPORT] 2026-04-05 오전 9:00

서비스 상태: 전체 30개 / 정상 29개 / 주의 1개

[주의] 인터뷰 준비 서비스 - DB 슬로우 쿼리 감지 (avg 1.2s)

원인: 인덱스 미적용 컬럼 조회 (user_activity.created_at)

권장 조치: ALTER TABLE user_activity ADD INDEX idx_created_at;

어제 커밋 23건 / PR 병합 4건 / 배포 2회

코드 품질: A (전일 대비 +0.3)

제가 타이핑한 건 없습니다.

전날 밤 11시에 자리를 떠났고, 아침 9시에 커피 한 잔 들고 슬랙을 열었을 때 이 보고서가 와 있습니다. Claude가 새벽에 서비스 전체를 점검하고, 코드 변경 사항을 분석하고, 이상 징후를 찾아 정리해 놓은 것입니다.

이 챗터는 그 시스템을 어떻게 만드는데에 대한 이야기입니다.

Claude를 “내가 시킬 때”에서 “알아서 일하게”로

4장까지는 Claude를 대화하듯 사용하는 방법을 다뤘습니다. 에이전트를 설계하고, 맥락을 구성하고, 복잡한 작업을 위임하는 법을 배웠습니다.

그런데 여기서 질문 하나.

“내가 명령하지 않으면 Claude는 아무것도 안 하는가?”

정답은 '아니오'입니다. Claude Code는 CLI로 비대화형(non-interactive) 실행이 가능합니다. 이 기능과 크론(cron)을 결합하면, 사람이 없어도 스스로 실행되는 시스템을 만들 수 있습니다.

Before/After를 비교해보겠습니다.

Before: 수동 실행 시대

```
나: "오늘 코드 품질 좀 봐줘"
Claude: [분석 중...]
나: "슬로우 쿼리 있는지 확인해줘"
Claude: [분석 중...]
나: "결과 정리해서 슬랙에 올려줄 수 있어?"
Claude: [작성 중...]
```

매일 반복되는 이 루틴. 작업 자체보다 “시키는 행위”가 더 많습니다.

After: 자동화 시대

```
# 크론타입에 이 한 줄이 있으면 끝
0 9 * * * timeout 300 /usr/local/bin/claude -p "일일 서비스 상태를 점검하고 슬랙으로 보고해" \
--output-format json >> /var/log/claude-daily.log 2>&1
```

Claude는 매일 오전 9시에 스스로 깨어나, 작업을 완료하고, 결과를 전송하고, 로그를 남깁니다. 제가 할 일은 없습니다.

비대화형 모드: CLI 핵심 옵션 완전 정복

Claude Code의 CLI에는 두 가지 실행 방식이 있습니다.

대화형(Interactive): `claude` 명령어를 실행하면 REPL 환경이 열립니다. 사람이 앉아서 주고받는 방식입니다.

비대화형(Non-interactive): `-p` 플래그와 함께 프롬프트를 직접 전달합니다. 사람 없이 실행됩니다.

참고로 대화형 모드 안에서 `/loop` 커맨드를 사용하면 특정 프롬프트를 일정 간격으로 반복 실행할 수도 있습니다. 간단한 모니터링이나 상태 체크에 유용하지만, 본격적인 자동화에는 크론이 더 적합합니다.

```
# 대화형 - 사람이 있어야 함
claude

# 비대화형 - 사람 없이 실행
claude -p "프로젝트의 TODO 주석을 찾아서 목록으로 정리해줘"
```

비대화형 모드의 옵션들을 익혀두면 자동화의 폭이 크게 넓어집니다.

```
claude -p "분석 프롬프트" \
  --output-format json \           # 출력을 JSON으로 (파싱하기 좋음)
  --dangerously-skip-permissions \ # 권한 확인 없이 실행 (샌드박스 환경 전용)
  --model sonnet                  # 모델 지정 (사용량 최적화)
```

각 옵션의 실제 동작을 실습해보겠습니다.

실습 1: 기본 비대화형 실행

터미널에서 직접 실행해보십시오.

```
# 프로젝트 디렉토리로 이동
cd /path/to/your-project

# 가장 단순한 형태
claude -p "현재 디렉토리에서 파일이 가장 많은 폴더 Top 3를 알려줘"
```

출력이 터미널에 바로 나타납니다. 대화형으로 대화 없이 결과만 받는 것입니다.

실습 2: JSON 출력으로 파싱 가능하게

자동화에서 가장 중요한 옵션은 `--output-format json` 입니다.

```
claude -p "현재 프로젝트의 TODO 주석 개수를 세어줘" --output-format json
```

출력 형태가 달라집니다.

```
{
  "result": "현재 프로젝트에서 TODO 주석이 총 47개 발견되었습니다.\n\n주요 위치:\n- src/services/user.py:32 - 캐시 만료 로직\n- src/routes/payment.ts:78 - 환불 처리 예외 케이스\n...",
  "session_id": "sess_abc123"
}
```

`result` 필드 안에 Claude의 응답이 들어있습니다. 쉘 스크립트에서 파싱해서 다음 작업으로 넘기기 좋습니다.

```
# Python으로 result 필드만 추출
RESULT=$(claude -p "TODO 개수를 세어줘" --output-format json 2>/dev/null)
ANSWER=$(echo "$RESULT" | python3 -c "import sys,json;
print(json.load(sys.stdin).get('result',''))")
echo "Claude 답변: $ANSWER"
```

실습 3: -max-budget-usd로 사용량 제어

`--max-budget-usd` 는 API 호출 비용의 상한을 달러 단위로 설정합니다. 구독 사용자는 `timeout` 명령어로 실행 시간을 제한하는 것이 더 실용적입니다.

```
# 실행 시간 제한: 60초 안에 완료되지 않으면 종료
timeout 60 claude -p "README.md를 읽고 한 줄 요약해줘"

# 복잡한 분석: 5분 타임아웃
timeout 300 claude -p "전체 서비스 코드를 분석하고 아키텍처 리포트를 작성해줘"
```

타임아웃이 걸리면 Claude는 그 시점까지 분석한 내용으로 응답하고 종료합니다. 무한정 실행되는 상황을 막아줍니다.

안전장치 먼저: 자동화의 리스크

크론으로 Claude를 돌리기 전에, 반드시 안전장치를 이해해야 합니다. 자동화는 양날의 검입니다. 올바르게 작동하면 강력하지만, 제한이 없으면 구독 사용량이 빠르게 소진되거나 의도치 않은 코드 변경이 일어날 수 있습니다.

안전장치 1: timeout 명령어로 실행 시간 제한

크론에서 가장 중요한 안전장치는 `timeout` 입니다. Claude가 무한정 실행되는 상황을 물리적으로 차단합니다.

```
# timeout 명령어로 최대 5분 실행 보장
# timeout이 먼저 걸리면 exit code 124로 종료됨
0 9 * * * timeout 300 claude -p "일일 보고서 생성" \
--output-format json >> /var/log/daily.log 2>&1
```

`timeout 300` 은 300초(5분) 후 강제 종료를 의미합니다. 일상적인 분석 작업은 5분이면 충분합니다. 5분이 넘는다면 프롬프트가 너무 광범위하거나 뭔가 잘못된 것입니다.

안전장치 2: -allowedTools로 허용 도구 제한

분석 전용 작업에는 `--allowedTools` 옵션으로 Claude가 사용할 수 있는 도구를 제한하십시오.

```
# 읽기 전용 분석 - 파일 읽기만 허용
claude -p "코드 품질을 분석하고 리포트 내용을 출력해줘" \
--allowedTools "Read,Grep,Glob"
```

`--allowedTools` 를 지정하면 목록에 없는 도구(Write, Bash 등)는 사용할 수 없습니다. 분석만 하고 파일을 수정하지 않아야 하는 크론에 적합합니다.

안전장치 4: 사용량 모니터링

Claude Code Pro(\$20/월) 또는 Max(\$100/월) 구독을 사용하는 경우, 크론 자동화는 월 사용량 내에서 추가 비용 없이 실행됩니다. 다만 구독 플랜에는 일일/월간 사용량 한도가 있으므로, 크론이 그 한도를 소진하지 않도록 관리해야 합니다.

Pro 플랜(\$20/월)은 대화형 작업에 적합하지만, 크론 자동화까지 병행하면 사용량이 빠르게 소진될 수 있습니다. 크론을 본격적으로 활용하려면 **Max 플랜(\$100/월)** 이상을 고려하십시오. Max 플랜은 크론 10개 이상과 복잡한 분석도 여유 있게 처리합니다.

핵심은 `timeout` 을 반드시 설정하는 것입니다. 타임아웃이 없으면 단일 크론이 사용량을 과도하게 소비할 수 있습니다. 초기에는 작게 시작하고 2~3일 동안 실제 사용량을 관찰한 후 확장하십시오.

실습: 첫 번째 크론 만들기 — 일일 코드 품질 리포트

이론은 끝났습니다. 실제로 만들어보겠습니다.

목표

매일 오전 9시, 어제 변경된 코드를 분석하고 품질 등급과 이슈 목록을 슬랙으로 전송합니다.

Step 1: 디렉토리 구조 준비

```
# 스크립트와 리포트 디렉토리 생성
mkdir -p /home/deploy/scripts/lib
mkdir -p /home/deploy/reports
mkdir -p /var/log/cron

# 환경 변수 파일 생성 (API 키, 웹훅 URL 보관)
touch /home/deploy/.env.automation
chmod 600 /home/deploy/.env.automation
```

`/home/deploy/.env.automation` 파일에 필요한 환경 변수를 채웁니다.

```
# /home/deploy/.env.automation

ANTHROPIC_API_KEY=sk-ant-api03-...
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/T.../B.../xxx
DISCORD_WEBHOOK_URL=https://discord.com/api/webhooks/...
PROJECT_DIR=/home/deploy/my-service
REPORT_DIR=/home/deploy/reports
```

Step 2: 알림 라이브러리 작성

슬랙으로 알림을 보내는 방법은 두 가지입니다. **웹훅 방식**(아래 예시)은 Incoming Webhook URL만 있으면 되므로 가장 간단합니다. **MCP 방식**은 Slack MCP 서버를 연결하면 Claude가 대화형으로 슬랙 채널에 메시지를 보내거나 읽을 수 있지만, 크론 자동화에서는 웹훅이 더 안정적입니다.

웹훅 전송 함수를 재사용 가능한 라이브러리로 분리합니다.

```
# /home/deploy/scripts/lib/notify.sh

#!/bin/bash

# .env.automation 로드 (크론에서는 자동 로드 안 됨)
if [ -f "/home/deploy/.env.automation" ]; then
    source /home/deploy/.env.automation
fi

# 슬랙 전송
# 사용법: send_slack "메시지" "good|warning|danger"
send_slack() {
    local message="$1"
    local color="${2:-good}" # good(초록), warning(노랑), danger(빨강)
}
```

슬랙 전송 함수의 본문입니다. JSON 페이로드를 생성하고 curl로 전송합니다:

```
if [ -z "$SLACK_WEBHOOK_URL" ]; then
    echo "[WARN] SLACK_WEBHOOK_URL 미설정. 슬랙 전송 스킵."
    return 0
fi

local payload
payload=$(python3 -c "
import json, sys
msg = sys.argv[1]
color = sys.argv[2]
payload = {
    'attachments': [{
        'color': color,
        'text': msg,
        'mrkdwn_in': ['text']
    }]
}
print(json.dumps(payload))
" "$message" "$color")

curl -s -o /dev/null -X POST "$SLACK_WEBHOOK_URL" \
    -H 'Content-Type: application/json' \
    -d "$payload"

echo "[$(date '+%H:%M:%S')] 슬랙 전송 완료 (color: $color)"
}
```

디스코드 전송 함수도 같은 패턴으로 구성됩니다:

```
# 디스코드 전송
# 사용법: send_discord "메시지"
send_discord() {
    local message="$1"

    if [ -z "$DISCORD_WEBHOOK_URL" ]; then
        echo "[WARN] DISCORD_WEBHOOK_URL 미설정. 디스코드 전송 스킵."
        return 0
    fi

    local payload
    payload=$(python3 -c "
import json, sys
print(json.dumps({'content': sys.argv[1]}))
" "$message")

    curl -s -o /dev/null -X POST "$DISCORD_WEBHOOK_URL" \
        -H 'Content-Type: application/json' \
        -d "$payload"

    echo "[$(date '+%H:%M:%S')] 디스코드 전송 완료"
}
```

Step 3: 코드 품질 분석 스크립트 작성

```
# /home/deploy/scripts/daily-code-quality.sh

#!/bin/bash
set -euo pipefail

# — 초기화 —
source /home/deploy/scripts/lib/notify.sh

TODAY=$(date +%Y-%m-%d)
LOG_PREFIX="[${TODAY} $(date '+%H:%M:%S')]"
REPORT_FILE="$REPORT_DIR/quality-${TODAY}.md"

echo "$LOG_PREFIX 일일 코드 품질 분석 시작..."
mkdir -p "$REPORT_DIR"
```

이어서 사전 체크와 Claude 실행 부분입니다:

```
# — 사전 체크 —
# claude 명령어 존재 확인
if ! command -v claude &>/dev/null; then
    echo "$LOG_PREFIX [ERROR] claude 명령어를 찾을 수 없습니다."
    send_slack "[ERROR] claude 명령어 없음. 서버 확인 필요." "danger"
    exit 1
fi

# 프로젝트 디렉토리 존재 확인
if [ ! -d "$PROJECT_DIR" ]; then
    echo "$LOG_PREFIX [ERROR] 프로젝트 디렉토리 없음: $PROJECT_DIR"
    exit 1
fi

cd "$PROJECT_DIR"
```

다음은 Claude에게 보낼 분석 프롬프트입니다:

```
# — Claude 실행 —
echo "$LOG_PREFIX Claude 분석 실행 중..."

RESULT=$(timeout 300 claude -p "
당신은 코드 품질 분석 에이전트입니다.
다음 항목을 순서대로 분석하고 마크다운 리포트를 작성해주세요.

[분석 범위]
- 최근 24시간 내 변경된 파일들 (git log --since='24 hours ago'로 확인)
- 변경이 없으면 전체 프로젝트에서 품질 이슈 상위 5개 파일

[분석 항목]
1. 변경 파일 목록 (파일명, 변경 줄 수)
2. 코드 스멜 발견 항목 (긴 함수 100줄 이상, 중복 코드, 하드코딩된 값)
3. TODO / FIXME 주석 목록 (파일명:라인수 형식)
4. 테스트 없는 새 public 메서드
5. 종합 등급 결정 기준:
  - A: 심각한 이슈 없음
  - B: 권고사항 3개 이하
  - C: 권고사항 4~7개 또는 경미한 버그
  - D: 심각한 버그 또는 보안 이슈")
```

출력 형식과 실행 옵션을 지정하는 부분입니다:

[출력]

리포트를 \$REPORT_FILE 에 마크다운으로 저장하세요.

저장 후 JSON으로 응답하세요:

```
{
  \"grade\": \"A\",
  \"issue_count\": 2,
  \"summary\": \"핵심 이슈 한 줄 요약\",
  \"urgent\": false
}
urgent는 D등급이거나 보안 이슈가 있을 때 true.
" \
--dangerously-skip-permissions \
--output-format json \
2>/dev/null) || {
  echo "$LOG_PREFIX [ERROR] Claude 실행 실패 (exit: $?)"
  send_slack "*[$TODAY] 일일 코드 품질 분석 실패*"
  Claude 실행 오류. 로그 확인: `"/var/log/cron/quality.log` "danger"
  exit 1
}
```

결과를 파싱하는 부분입니다. 빈 결과 체크 후 JSON에서 grade와 summary를 추출합니다:

```
# — 결과 파싱 —————
if [ -z "$RESULT" ]; then
  echo "$LOG_PREFIX [ERROR] 빈 결과 반환"
  send_slack "*[$TODAY] 코드 품질 분석 - 결과 없음*" "warning"
  exit 1
fi

# JSON에서 grade, summary 추출
GRADE=$(echo "$RESULT" | python3 -c "
import sys, json, re
raw = sys.stdin.read()
try:
  outer = json.loads(raw)
  result_str = outer.get('result', '')
  match = re.search(r'\"grade\"\\s*:\\s*\"([A-D])\"', result_str)
  print(match.group(1) if match else 'B')
except:
  print('B')
" 2>/dev/null || echo "B")
```

summary와 urgent 플래그를 추출합니다:

```

SUMMARY=$(echo "$RESULT" | python3 -c "
import sys, json, re
raw = sys.stdin.read()
try:
    outer = json.loads(raw)
    result_str = outer.get('result', '')
    match = re.search(r'\"summary\"\\s*:\\s*\"([^\"]+)\"', result_str)
    print(match.group(1) if match else '분석 완료')
except:
    print('분석 완료')
" 2>/dev/null || echo "분석 완료")

IS_URGENT=$(echo "$RESULT" | python3 -c "
import sys, json, re
raw = sys.stdin.read()
try:
    outer = json.loads(raw)
    result_str = outer.get('result', '')
    match = re.search(r'\"urgent\"\\s*:\\s*(true|false)', result_str)
    print(match.group(1) if match else 'false')
except:
    print('false')
" 2>/dev/null || echo "false")

```

마지막으로 등급에 따라 슬랙 알림을 전송합니다:

```

# — 슬랙 알림 —————
case "$GRADE" in
  A|B) COLOR="good" ;;
  C)   COLOR="warning" ;;
  D)   COLOR="danger" ;;
  *)   COLOR="warning" ;;
esac

if [ "$IS_URGENT" = "true" ]; then
  COLOR="danger"
  URGENCY_TAG=" - *긴급 조치 필요*"
else
  URGENCY_TAG=""
fi

send_slack "*[$TODAY] 코드 품질 리포트$URGENCY_TAG*
등급: *$GRADE*
SUMMARY
상세 리포트: \`${REPORT_FILE}\`" "$COLOR"

echo "$LOG_PREFIX 완료. 등급: $GRADE / 슬랙 전송 완료"
echo "$LOG_PREFIX 리포트 저장 위치: $REPORT_FILE"

```

```
# 실행 권한 부여
chmod +x /home/deploy/scripts/daily-code-quality.sh
chmod +x /home/deploy/scripts/lib/notify.sh
```

Step 4: 직접 실행해서 검증

크론 등록 전에 반드시 직접 실행해서 확인하십시오. 크론에 올리고 나서 문제를 찾으려면 디버깅이 훨씬 어렵습니다.

```
# 직접 실행 테스트
bash /home/deploy/scripts/daily-code-quality.sh

# 성공 시 출력
[2026-04-05 09:00:01] 일일 코드 품질 분석 시작...
[2026-04-05 09:00:01] Claude 분석 실행 중...
[2026-04-05 09:02:34] 완료. 등급: A / 슬랙 전송 완료
[2026-04-05 09:02:34] 리포트 저장 위치: /home/deploy/reports/quality-2026-04-05.md

# 리포트 내용 확인
cat /home/deploy/reports/quality-2026-04-05.md
```

슬랙에 메시지가 오고 리포트 파일이 생성됐으면 성공입니다.

Step 5: 크론탭 등록

```
# 크론탭 편집
crontab -e
```

아래 내용을 추가합니다.

```
# 크론탭 상단 - 환경 변수 설정 (크론은 기본 PATH가 매우 좁음)
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOME=/home/deploy

# 매일 오전 9시: 코드 품질 분석
0 9 * * * timeout 300 bash /home/deploy/scripts/daily-code-quality.sh >> /var/log/cron/quality.log 2>&1
```

```
# 크론 등록 확인
crontab -l
```

첫 번째 크론이 완성되었습니다. 다음 날 오전 9시에 슬랙 알림이 오면 성공입니다.

실전 시나리오 2: 주간 의존성 취약점 체크

보안 취약점은 바쁜 1인 개발자가 가장 놓치기 쉬운 영역입니다. 주간 자동 체크를 설정해두면 잊지 않아도 됩니다.

```
# /home/deploy/scripts/weekly-dependency-check.sh

#!/bin/bash
set -euo pipefail

source /home/deploy/scripts/lib/notify.sh

TODAY=$(date +%Y-%m-%d)
LOG_PREFIX="[ $TODAY $(date '+%H:%M:%S') ]"
REPORT_FILE="$REPORT_DIR/security-$TODAY.md"

echo "$LOG_PREFIX 주간 보안 취약점 분석 시작..."
mkdir -p "$REPORT_DIR"
```

NPM과 Java 프로젝트의 취약점 데이터를 수집합니다:

```
# — 원시 데이터 수집 —————
NPM_AUDIT_DATA=""
for project_dir in /home/deploy/*/; do
    if [ -f "${project_dir}package.json" ]; then
        project_name=$(basename "$project_dir")
        echo "$LOG_PREFIX npm audit: $project_name"
        audit_result=$(cd "$project_dir" && npm audit --json 2>/dev/null || echo '{"error":
"audit failed"}')
        NPM_AUDIT_DATA="${NPM_AUDIT_DATA}\n\n=== ${project_name} ===\n${audit_result}"
    fi
done

# Java 프로젝트: OWASP Dependency Check 결과 수집
JAVA_AUDIT_DATA=""
for project_dir in /home/deploy/*/; do
    if [ -f "${project_dir}pom.xml" ]; then
        project_name=$(basename "$project_dir")
        owasp_report="${project_dir}target/dependency-check-report.json"
        if [ -f "$owasp_report" ]; then
            JAVA_AUDIT_DATA="${JAVA_AUDIT_DATA}\n\n=== ${project_name} ===\n$(head -300
"$owasp_report")"
        fi
    fi
done
```

수집된 데이터를 Claude에게 전달하여 분석을 요청합니다:

```
# — Claude 분석 —————
RESULT=$(timeout 600 claude -p "
당신은 보안 취약점 분석 에이전트입니다.

아래 의존성 취약점 원시 데이터를 분석하세요.

=== NPM 프로젝트 취약점 데이터 ===
$NPM_AUDIT_DATA

=== Java 프로젝트 취약점 데이터 ===
$JAVA_AUDIT_DATA")
```

리포트 형식과 JSON 응답 규칙을 지정합니다:

[리포트 작성 - \$REPORT_FILE 에 저장]

1. 긴급 조치 필요 (Critical / High)
 - 프로젝트명, 패키지명, 현재버전, 안전버전, CVE 번호, 업그레이드 명령어
2. 일반 취약점 (Medium)
 - 목록 요약 (프로젝트명, 패키지명, CVE)
3. 낮은 심각도 (Low / Info)
 - 총 개수만 기록
4. 이번 주 조치 권고사항 (우선순위 순서로 5개 이내)

[JSON 응답]

리포트 저장 후 반드시 JSON으로만 응답:

긴급 취약점 있으면: {"urgent": true, "critical_count": 3, "summary": "한 줄 요약"}

없으면: {"urgent": false, "critical_count": 0, "summary": "이번 주 긴급 취약점 없음"}

```
" \
--dangerously-skip-permissions \
--output-format json \
2>/dev/null) || {
    echo "$LOG_PREFIX [ERROR] Claude 실행 실패"
    send_slack "*[$TODAY] 보안 체크 실패*" "danger"
    exit 1
}
```

결과를 파싱하고 긴급 여부에 따라 슬랙 알림을 전송합니다:

```
# — 결과 파싱 및 알림 —————
IS_URGENT=$(echo "$RESULT" | python3 -c "
import sys, json, re
raw = sys.stdin.read()
try:
    outer = json.loads(raw)
    result_str = outer.get('result', '')
    match = re.search(r'\"urgent\"\\s*:\\s*(true|false)', result_str)
    print(match.group(1) if match else 'false')
except:
    print('false')
" 2>/dev/null || echo "false")
```

summary를 추출하고 긴급 여부에 따라 슬랙 메시지를 구분합니다:

```

SUMMARY=$(echo "$RESULT" | python3 -c "
import sys, json, re
raw = sys.stdin.read()
try:
    outer = json.loads(raw)
    result_str = outer.get('result', '')
    match = re.search(r'\"summary\"\\s*:\\s*\"([^\"]+)\"', result_str)
    print(match.group(1) if match else '분석 완료')
except:
    print('분석 완료')
" 2>/dev/null || echo "분석 완료")

if [ "$IS_URGENT" = "true" ]; then
    send_slack "*[$TODAY] 주간 보안 체크 - 긴급 취약점 발견*"
    $SUMMARY
    즉시 리포트 확인: \`${REPORT_FILE}\` "danger"
else
    send_slack "*[$TODAY] 주간 보안 체크 - 이상 없음*"
    $SUMMARY
    상세: \`${REPORT_FILE}\` "good"
fi

echo "$LOG_PREFIX 보안 분석 완료"

```

크론 등록 (매주 월요일 오전 8시)

```

# crontab -e
0 8 * * 1 timeout 600 bash /home/deploy/scripts/weekly-dependency-check.sh >> /var/log/
cron/security.log 2>&1

```

고급 팁: GitHub Actions에서 PR 자동 리뷰

이벤트 기반 자동화가 필요하다면, Anthropic이 공식으로 제공하는 [anthropics/claude-code-action@beta](https://github.com/anthropics/claude-code-action)를 GitHub Actions에서 사용할 수 있습니다. PR이 열릴 때 Claude가 자동으로 diff를 분석해 리뷰 코멘트를 남기는 구조입니다. 자세한 설정은 공식 문서(<https://github.com/anthropics/claude-code-action>)를 참고하십시오. 대부분의 1인 개발자에게는 로컬 크론 기반 자동화만으로도 충분합니다.

크론탭 전체 구성: 실전 40+ 크론 설계

하나씩 추가하다 보면 크론이 늘어납니다. 제가 운영하는 실제 크론 구성의 패턴을 공유합니다. 서비스명은 추상화했습니다.

```
# /home/deploy/crontab.conf
# 적용 방법: crontab /home/deploy/crontab.conf

# -----
# 환경 변수 (크론은 기본 PATH가 매우 좁음 - 반드시 설정)
# -----
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOME=/home/deploy

# -----
# 일일 작업 (Daily)
# -----

# 오전 9:00 - 전체 서비스 상태 종합 보고 (최우선)
0 9 * * * timeout 300 bash /home/deploy/scripts/daily-summary.sh \
  >> /var/log/cron/daily.log 2>&1

# 오전 9:10 - 코드 품질 분석 (daily-summary 이후 실행)
10 9 * * * timeout 300 bash /home/deploy/scripts/daily-code-quality.sh \
  >> /var/log/cron/quality.log 2>&1
```

DB 관련 작업과 주간 작업입니다:

```
# 자정 00:00 - DB 백업 상태 확인 및 용량 알림
0 0 * * * timeout 120 bash /home/deploy/scripts/check-db-backup.sh \
  >> /var/log/cron/backup.log 2>&1

# 새벽 2:00 - 슬로우 쿼리 분석 (트래픽 없는 시간대)
0 2 * * * timeout 600 bash /home/deploy/scripts/analyze-slow-queries.sh \
  >> /var/log/cron/db.log 2>&1

# _____
# 주간 작업 (Weekly)
# _____

# 월요일 8:00 - 의존성 취약점 체크
0 8 * * 1 timeout 600 bash /home/deploy/scripts/weekly-dependency-check.sh \
  >> /var/log/cron/security.log 2>&1

# 월요일 8:30 - 주간 업무 계획 생성
30 8 * * 1 timeout 300 bash /home/deploy/scripts/weekly-plan.sh \
  >> /var/log/cron/weekly.log 2>&1

# 금요일 17:00 - 주간 회고 리포트
0 17 * * 5 timeout 300 bash /home/deploy/scripts/weekly-review.sh \
  >> /var/log/cron/weekly.log 2>&1
```

월간 작업입니다:

```
# _____
# 월간 작업 (Monthly)
# _____

# 매월 1일 9:00 - 월간 KPI 리포트
0 9 1 * * timeout 600 bash /home/deploy/scripts/monthly-kpi.sh \
  >> /var/log/cron/monthly.log 2>&1

# 매월 1일 9:30 - API 문서 자동 갱신
30 9 1 * * timeout 600 bash /home/deploy/scripts/update-api-docs.sh \
  >> /var/log/cron/docs.log 2>&1

# 매월 15일 10:00 - 기술 부채 심층 분석
0 10 15 * * timeout 900 bash /home/deploy/scripts/tech-debt-analysis.sh \
  >> /var/log/cron/techdebt.log 2>&1
```

크론 관리 팁

크론이 늘어나면 “지금 어떤 크론이 돌고 있지?”를 추적하기 어려워집니다. 몇 가지 규칙을 정해두면 편합니다.

로그 디렉토리 구조 통일

```

/var/log/cron/
├─ daily.log      # 일일 요약
├─ quality.log   # 코드 품질
├─ security.log  # 보안 체크
├─ db.log        # DB 관련
├─ weekly.log    # 주간 리포트
├─ monthly.log   # 월간 리포트
└─ archive/
    ├─ 2026-03/
    └─ 2026-04/

```

로그 로테이션 설정

```

# /etc/logrotate.d/claude-cron

/var/log/cron/*.log {
    daily
    rotate 30
    compress
    missingok
    notifempty
    dateext
    dateformat -%Y%m%d
    postrotate
        /usr/bin/find /var/log/cron/archive -mtime +90 -delete
    endscript
}

```

크론 상태 확인 스크립트

매일 아침 어떤 크론이 성공했고 어떤 것이 실패했는지 한눈에 보는 스크립트입니다.

```
# /home/deploy/scripts/cron-status.sh
# 사용법: bash /home/deploy/scripts/cron-status.sh

#!/bin/bash

YESTERDAY=$(date -d "yesterday" +%Y-%m-%d 2>/dev/null || date -v-1d +%Y-%m-%d)
echo "=== $YESTERDAY 크론 실행 현황 ==="
echo ""

for log_file in /var/log/cron/*.log; do
    log_name=$(basename "$log_file" .log)

    if ! grep -q "$YESTERDAY" "$log_file" 2>/dev/null; then
        printf "%-20s [스킵] 어제 실행 기록 없음\n" "$log_name"
        continue
    fi

    total=$(grep -c "$YESTERDAY" "$log_file" 2>/dev/null || echo 0)
    errors=$(grep "$YESTERDAY" "$log_file" | grep -ci "\[ERROR\]\|\[WARN\]\|timeout\|failed" || echo 0)

    if [ "$errors" -gt 0 ]; then
        printf "%-20s [주의] 실행 %s회 / 오류 %s건\n" "$log_name" "$total" "$errors"
    else
        printf "%-20s [정상] 실행 %s회\n" "$log_name" "$total"
    fi
done
```

CLAUDE.md: 자동화 에이전트 전용 지침

자동화 스크립트에서 Claude를 실행할 때 한 가지 함정이 있습니다. Claude는 프로젝트 디렉토리의 **CLAUDE.md**를 읽습니다. 대화형 작업용 지침과 자동화용 지침이 섞이면 Claude가 혼란스러운 행동을 할 수 있습니다.

예를 들어, 대화형 CLAUDE.md에 “항상 사용자에게 확인 후 진행하라”고 써있으면, 자동화 크론에서 Claude가 중간에 멈추고 확인을 기다리는 상황이 생깁니다. 아무도 응답하지 않으니 타임아웃까지 대기하게 됩니다.

자동화 전용 지침을 별도 파일로 분리하는 패턴을 권장합니다.

```
# 자동화 전용 컨텍스트 디렉토리 생성
mkdir -p /home/deploy/automation-context
```

```
# /home/deploy/automation-context/CLAUDE.md
```

```
# 자동화 에이전트 공통 지침
```

```
## 실행 환경
```

- 이 Claude는 크론 또는 CI/CD에서 자동 실행됩니다
- 사람이 응답을 기다리지 않습니다
- 작업이 완료되면 즉시 종료해야 합니다

```
## 절대 금지 사항
```

- 프로덕션 DB에 직접 쓰기 작업
- 배포 파이프라인 직접 트리거
- 외부 시스템에 사전 승인 없이 메시지 전송 (지정된 슬랙 웹훅 제외)
- 파일 삭제 또는 이동 작업
- 사용자에게 확인 요청 (아무도 없음)

이어서 출력 규칙, 불확실한 상황 대응, 알림 기준입니다:

```
## 출력 규칙
```

- 마크다운 리포트는 반드시 지정된 경로에 저장
- 에러 발생 시 [ERROR] 접두사 필수
- 경고 시 [WARN] 접두사 필수
- 작업 완료 시 마지막 출력에 [DONE] 포함

```
## 불확실할 때
```

- 확인이 필요한 상황이면 [WARN] 로그를 남기고 계속 진행
- 중단해야 할 치명적 오류만 [ERROR]로 표시 후 종료
- 절대 중간에 멈추거나 입력을 기다리지 말 것

```
## 슬랙 알림 기준
```

- Critical: 즉시 전송 (색상: danger)
- Warning: 리포트 파일 + 슬랙 전송 (색상: warning)
- Info: 리포트 파일에만 기록

스크립트에서 이 컨텍스트 디렉토리를 참조합니다.

```
# 자동화 전용 CLAUDE.md 사용
```

```
timeout 300 claude -p "일일 보고서 생성" \  
  --dangerously-skip-permissions \  
  --output-format json
```

프로젝트 CLAUDE.md와 자동화 CLAUDE.md가 분리되면, 대화할 때는 사람 친화적인 지침이, 크론에서는 자동화 친화적인 지침이 적용됩니다.

자주 발생하는 문제와 해결법

자동화를 처음 구축할 때 겪는 문제들을 미리 정리합니다.

문제 1: 크론에서 claude 명령어를 찾지 못한다

크론은 기본 `PATH` 가 매우 제한적입니다. `which claude` 로 실제 경로를 확인하고 절대 경로를 사용하십시오.

```
# 경로 확인
which claude
# /usr/local/bin/claude

# 방법 A: 크론탭 상단에 PATH 설정 (권장)
PATH=/usr/local/bin:/usr/bin:/bin
0 9 * * * claude -p "..."
```

방법 B: 절대 경로 직접 사용

```
0 9 * * * /usr/local/bin/claude -p "..."
```

문제 2: 환경 변수(API 키, 웹훅 URL)가 크론에서 로드되지 않는다

크론은 `~/bashrc` 나 `~/zshrc` 를 읽지 않습니다. 스크립트 내부에서 명시적으로 로드하십시오.

```
# 스크립트 상단에 추가
source /home/deploy/.env.automation
# 또는
export ANTHROPIC_API_KEY=$(cat /home/deploy/.secrets/anthropic-key)
```

문제 3: 출력이 너무 길어서 로그 파일이 빠르게 채워진다

Claude의 전체 출력을 로그에 저장하면 디스크가 금방 가득 찹니다. 메타 정보(완료 여부, 시간)만 로그에 남기고 상세 내용은 별도 파일로 저장하십시오.

```
# 잘못된 방법 - 전체 출력이 로그에 쌓임
claude -p "... " >> /var/log/cron/daily.log 2>&1

# 올바른 방법 - 메타 정보만 로그에, 상세 내용은 별도 저장
RESULT=$(claude -p "... " --output-format json 2>/dev/null)
echo "[$(date '+%Y-%m-%d %H:%M:%S')] 완료" >> /var/log/cron/daily.log
echo "$RESULT" > /home/deploy/reports/detail-$(date +%Y%m%d).json
```

문제 4: API 오류가 발생해도 크론이 성공으로 처리된다

Claude API가 일시적으로 실패하면 빈 결과나 오류 JSON이 반환됩니다. 출력을 검증하는 로직을 추가하십시오.

```
RESULT=$(claude -p "... " --output-format json 2>/dev/null)

# 검증: 빈 결과 또는 에러 포함 시 알람
if [ -z "$RESULT" ]; then
    send_slack "[ERROR] 분석 실패 - 빈 결과 반환" "danger"
    exit 1
fi

if echo "$RESULT" | python3 -c "import sys,json; d=json.load(sys.stdin); exit(0 if 'result' in d else 1)" 2>/dev/null; then
    echo "정상 결과"
else
    send_slack "[ERROR] 분석 실패 - API 오류" "danger"
    exit 1
fi
```

문제 5: 여러 크론이 동시에 실행되어 서버 부하가 급증한다

특히 매일 오전 9시에 여러 크론을 등록하면 동시 실행이 됩니다. 시간을 분산시키십시오.

```
# 잘못된 방법 - 오전 9시에 모두 몰림
0 9 * * * bash /scripts/daily-summary.sh
0 9 * * * bash /scripts/daily-quality.sh
0 9 * * * bash /scripts/daily-db-check.sh

# 올바른 방법 - 10분씩 분산
0 9 * * * bash /scripts/daily-summary.sh
10 9 * * * bash /scripts/daily-quality.sh
20 9 * * * bash /scripts/daily-db-check.sh
```

자동화 파이프라인 전체 그림

지금까지 만든 자동화들을 정리합니다.

```

매일 09:00 — daily-summary.sh
├─ Claude가 전체 서비스 상태 분석
├─ 결과를 슬랙 #daily-report 채널에 전송

매일 09:10 — daily-code-quality.sh
├─ 어제 변경된 코드 분석
├─ 품질 등급(A~D) 및 이슈 목록 생성
├─ /reports/quality-YYYY-MM-DD.md 저장

매일 02:00 — analyze-slow-queries.sh
├─ DB 슬로우 쿼리 로그 분석
├─ 인덱스 미적용 쿼리 발견 시 슬랙 알람

매주 월요일 08:00 — weekly-dependency-check.sh
├─ npm audit, OWASP 분석
├─ 긴급 취약점 발견 시 즉시 슬랙 알람

PR 생성 시 (선택사항, 이벤트 기반)
├─ GitHub Actions: claude-code-action
├─ Claude가 diff 분석 후 PR 코멘트

```

이 구조가 완성되면 코드를 짜는 일에만 집중할 수 있습니다. 리뷰, 분석, 보고는 시스템이 합니다.

이 chapter의 핵심 3가지

자동화 인프라를 구축하는 과정에서 가장 중요한 세 가지를 꼽으라면 이렇습니다.

첫째, 안전장치 없는 자동화는 폭탄입니다. `timeout` 으로 실행 시간을 제한하고, `--allowedTools` 로 허용 도구를 제한하십시오. 이 두 가지를 반드시 설정한 다음에 크론을 등록하십시오. 타임아웃 없이 크론을 등록하면 구독 사용량이 예상보다 빠르게 소진될 수 있습니다.

둘째, 작게 시작하고 관찰한 다음 확장하십시오. 첫 크론은 하나로 시작하고, 2~3일 동안 로그와 사용량을 관찰한 다음 다음 것을 추가하십시오. 처음부터 10개를 등록하면 문제가 생겼을 때 어디서 터진 건지 알 수 없습니다.

셋째, 크론이 만든 리포트를 정말로 읽으십시오. 자동화의 함정은 보고서가 쌓이는데 아무도 읽지 않는 상황입니다. 처음 2주는 매일 아침 Claude가 만든 리포트를 읽으십시오. 읽지 않는 리포트가 생기면 그 크론을 정리하십시오. 리포트는 적을수록 더 유용합니다.

다음 챕터에서는 자동화된 시스템이 만들어내는 결과물의 품질을 어떻게 보장하는지, 즉 품질 게이트(Quality Gate) 구축을 다룹니다.

CH06: 품질 게이트 — AI가 만든 코드를, AI가 검증한다

새벽 2시의 배포

2년 전, 저는 새벽 2시에 핫픽스를 배포했습니다.

결제 서비스에서 특정 조건의 쿠폰이 중복 적용되는 버그가 발견됐고, Claude가 빠르게 수정 코드를 만들어줬습니다. 코드를 훑어봤습니다. 로직이 맞아 보였습니다. 지쳐 있었고, 빨리 끝내고 싶었습니다. 커밋하고 배포했습니다.

다음 날 아침, 슬랙에 알림이 왔습니다.

[결제 서비스 장애] 포인트 잔액이 음수(-) 처리되는 케이스 발생
영향 고객: 23명

수정 코드가 쿠폰 중복 적용은 막았지만, 포인트와 쿠폰이 동시에 적용되는 엣지 케이스를 놓쳤습니다. Claude는 수정 요청에만 집중했고, 저는 피곤했습니다. 두 개의 맹점이 겹쳤습니다.

이 사건 이후 저는 한 가지를 결심했습니다.

“AI가 만든 코드는, 다른 AI가 검증해야 한다.”

사람이 직접 리뷰할 시간이 없다면, 리뷰 자체를 자동화해야 합니다. 이 챕터는 그 시스템을 만드는 방법입니다.

6.1 품질 게이트란 무엇인가

소프트웨어 개발에서 “게이트”는 다음 단계로 넘어가기 전에 통과해야 하는 체크포인트입니다. 전통적인 품질 게이트는 사람이 지킵니다. 코드 리뷰, QA 테스트, 스테이징 환경 검증.

AI 코딩 환경에서 이 게이트는 두 가지 이유로 위협받습니다.

첫째, **속도 비대칭**입니다. Claude는 30분에 수백 라인의 코드를 생성합니다. 사람이 그 속도로 리뷰하는 것은 물리적으로 불가능합니다. AI가 속도를 내는 만큼 리뷰 병목이 생기고, 결국 사람이 더 바빠지는 역설이 발생합니다.

둘째, **위임의 역설**입니다. AI에게 코드 생성을 위임했는데, 검증은 사람이 직접 한다면 위임이 절반밖에 안 됩니다. 더 나쁜 건, 야근하고 지친 상태에서 리뷰하면 검증의 질이 오히려 낮아진다는 점입니다.

해결책은 게이트 자체를 자동화하는 것입니다.

```
[코드 생성] → [게이트 1: 정적 분석 (도구)]
               → [게이트 2: AI 코드 리뷰 (다른 에이전트)]
               → [게이트 3: 테스트 실행 (CI/CD)]
               → [게이트 4: 배포 판단 (사람 + AI 보조)]
```

각 게이트는 사람이 아닌 도구 또는 다른 AI가 지킵니다. 사람은 게이트를 통과하지 못한 코드에만 개입합니다. 그리고 마지막 배포 결정만 직접 내립니다.

6.2 위임 등급: 뭘 자동화하고 뭘 사람이 할까

품질 게이트를 설계하기 전에, 먼저 위임 등급 체계를 정해야 합니다. 모든 결정을 자동화할 수 없고, 모든 결정을 사람이 할 필요도 없습니다.

저는 다음 4단계 등급을 사용합니다.

| 등급 | 이름 | 설명 | 예시 |
|----|-------|--------------------------|----------------------------------|
| L1 | 자동 실행 | 에이전트가 판단하고 즉시 실행. 보고 불필요 | 린트 에러 수정, 코드 포매팅, 패치 버전 의존성 업데이트 |

| 등급 | 이름 | 설명 | 예시 |
|----|---------------|------------------------------|--------------------------------------|
| L2 | 자동 실행 + 보고 | 실행 후 결과만 사람에게 알림 | 단위 테스트 실패 감지 및 수정, pre-commit 자동 교정 |
| L3 | 계획 생성 + 승인 대기 | 에이전트가 계획을 만들고 사람이 승인 후 실행 | DB 스키마 변경, API 인터페이스 변경, 마이너 버전 업데이트 |
| L4 | 반드시 사람이 결정 | 자동화 금지. 에이전트가 제안해도 사람이 최종 판단 | 프로덕션 배포, 결제 로직 변경, 사용자 데이터 마이그레이션 |

이 등급표는 [CLAUDE.md](#)에 명시해야 합니다. 그래야 에이전트가 상황에 맞게 판단합니다. 등급표가 없으면 에이전트는 매번 스스로 등급을 추론해야 하고, 판단이 일관되지 않습니다. 때로는 L4 결정을 L1처럼 처리합니다.

실제로 사용하는 [CLAUDE.md](#) 등급표입니다:

위임 등급

L1 - 자동 실행 (보고 불필요)

아래 항목은 판단 없이 즉시 실행해도 된다.

- 린트/포매팅 자동 수정 (spotlessApply, eslint --fix)
- 오타 수정
- 패치 버전 의존성 업데이트 (1.2.3 → 1.2.4)
- 주석 추가/수정
- 로그 레벨 조정

L2 - 자동 실행 + Slack 보고

실행 후 결과를 #dev-ops 채널에 한 줄 보고한다.

- 단위 테스트 실패 감지 및 수정 시도
- 코드 품질 경고 (SonarQube Minor 이하) 자동 수정
- 불필요한 의존성 제거

이어서 사람의 승인이 필요한 L3, L4 등급입니다:

L3 - 계획 생성 후 나에게 승인 요청
실행 전 반드시 나에게 물어봐야 한다.

- DB 마이그레이션 스크립트 생성 및 실행
- API 스펙 변경 (기존 인터페이스 수정)
- 마이너/메이저 버전 의존성 업데이트
- 새로운 외부 서비스 연동

L4 - 절대 자동 실행 금지. 반드시 나에게 물어볼 것
어떤 상황에서도 사람의 명시적 승인 없이 실행 불가.

- 프로덕션 배포 (CI/CD 파이프라인만 허용, 직접 실행 금지)
- 결제/환불/포인트 관련 코드 변경
- 개인정보 처리 로직 변경
- .env 파일 수정
- 사용자 데이터 삭제 또는 마이그레이션

이 파일이 없는 상태로 에이전트에게 작업을 맡기면, 에이전트는 “괜찮겠지”라는 판단 하에 L4 수준의 변경을 자동으로 커밋할 수 있습니다.

6.3 Claude Code Hooks: 행동하는 순간을 잡아라

Claude Code의 `settings.json` 에는 강력한 기능이 있습니다. **Hooks** 시스템입니다.

Hooks는 Claude가 특정 도구를 사용하기 전후에, 또는 알림을 보낼 때 셸 명령어를 자동으로 실행합니다. 즉, Claude가 어떤 행동을 하는 순간을 가로채서 추가 로직을 실행할 수 있습니다.

Hooks 기본 구조

```
// ~/.claude/settings.json (전역) 또는 {프로젝트}/.claude/settings.json (프로젝트)
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "도구명 또는 정규식",
        "hooks": [
          {
            "type": "command",
            "command": "실행할 셸 명령어"
          }
        ]
      }
    ]
  }
},
```

이어서 PostToolUse와 Notification 훅입니다:

```
"PostToolUse": [
  {
    "matcher": "도구명",
    "hooks": [
      {
        "type": "command",
        "command": "실행할 셸 명령어"
      }
    ]
  }
],
"Notification": [
  {
    "matcher": "알림 메시지 패턴 (정규식)",
    "hooks": [
      {
        "type": "command",
        "command": "실행할 셸 명령어"
      }
    ]
  }
]
}
```

세 가지 훅 타입이 있습니다.

PreToolUse: 도구 실행 직전에 실행됩니다. 명령어를 검사하고, 필요하면 차단할 수 있습니다. 훅 명령어가 exit 1을 반환하면 Claude의 도구 실행이 중단됩니다.

PostToolUse: 도구 실행 직후에 실행됩니다. 파일이 수정된 후 자동으로 검사를 트리거하거나, 결과를 로깅합니다.

Notification: Claude가 완료, 경고, 에러 등의 알림을 보낼 때 실행됩니다. 시스템 알림이나 슬랙 메시지로 전달할 수 있습니다.

환경 변수

훅 명령어 안에서 Claude가 제공하는 환경 변수를 사용할 수 있습니다.

| 변수 | 설명 | 사용 가능한 훅 |
|---|---------------------------------------|-------------------------|
| <code>\$CLAUDE_TOOL_NAME</code> | 실행된 도구 이름
(Bash, Write,
Read 등) | PreToolUse, PostToolUse |
| <code>\$CLAUDE_TOOL_INPUT</code> | 도구에 전달된 입력 (JSON) | PreToolUse |
| <code>\$CLAUDE_TOOL_RESULT</code> | 도구 실행 결과 (JSON) | PostToolUse |
| <code>\$CLAUDE_TOOL_INPUT_COMMAND</code> | Bash 도구의 실행 명령어 | PreToolUse (Bash) |
| <code>\$CLAUDE_TOOL_INPUT_FILE_PATH</code> | Write/Read 도구의 파일 경로 | PreToolUse (Write/Read) |
| <code>\$CLAUDE_TOOL_RESULT_FILE_PATH</code> | Write 완료 후 파일 경로 | PostToolUse (Write) |
| <code>\$CLAUDE_NOTIFICATION</code> | 알림 메시지 내용 | Notification |
| <code>\$PROJECT_ROOT</code> | 현재 프로젝트 루트 디렉토리 | 모든 훅 |

실전 Hooks 설정 전문 예시

실제 운영 환경에서 사용하는 `settings.json` 전체 예시입니다:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "CMD=\"${CLAUDE_TOOL_INPUT_COMMAND}\"; if echo \"${CMD}\" | grep -qE
'(git push --force|git push -f|rm -rf /|DROP TABLE|TRUNCATE|DELETE FROM [a-z_]+ WHERE
1=1)'; then echo '[BLOCKED] 위험한 명령어입니다: \"${CMD}\" && exit 1; fi; echo \"[AUDIT] $
(date '+%H:%M:%S') CMD: ${CMD}\" >> /tmp/claude-audit.log"
          }
        ]
      },
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "command": "FILE=\"${CLAUDE_TOOL_INPUT_FILE_PATH}\"; if [[ \"${FILE}\" ==
*\".env\"* ]] || [[ \"${FILE}\" == */secrets/* ]]; then echo '[BLOCKED] .env 또는
secrets 경로에는 직접 쓸 수 없습니다' && exit 1; fi"
          }
        ]
      }
    ]
  },
  "PostToolUse": [
    {
      "type": "command",
      "command": "echo \"[AUDIT] Tool Use Command: ${CMD}\" >> /tmp/claude-audit.log"
    }
  ]
}
```

이어서 PostToolUse 훅 설정입니다. 파일 저장 후 자동으로 린트를 실행합니다:

```

"PostToolUse": [
  {
    "matcher": "Write",
    "hooks": [
      {
        "type": "command",
        "command": "FILE=\"${CLAUDE_TOOL_RESULT_FILE_PATH}\"; if [[ \"$FILE\" == *.java ]]; then PROJECT_ROOT=$(git -C \"$(dirname \"$FILE\")\" rev-parse --show-toplevel 2>/dev/null); if [ -n \"$PROJECT_ROOT\" ] && [ -f \"$PROJECT_ROOT/gradlew\" ]; then cd \"$PROJECT_ROOT\" && ./gradlew spotlessCheck -q 2>&1 | grep -E '(FAIL|error|warning)' | head -5 || true; fi; fi"
      }
    ]
  },
  {
    "matcher": "Write",
    "hooks": [
      {
        "type": "command",
        "command": "FILE=\"${CLAUDE_TOOL_RESULT_FILE_PATH}\"; if [[ \"$FILE\" == *.ts ]] || [[ \"$FILE\" == *.tsx ]]; then PROJECT_ROOT=$(git -C \"$(dirname \"$FILE\")\" rev-parse --show-toplevel 2>/dev/null); if [ -n \"$PROJECT_ROOT\" ] && [ -f \"$PROJECT_ROOT/package.json\" ]; then cd \"$PROJECT_ROOT\" && npx eslint \"$FILE\" --max-warnings=0 2>&1 | tail -5 || true; fi; fi"
      }
    ]
  }
],

```

마지막으로 Bash 감사 로그와 Notification 훅입니다:

```

{
  "matcher": "Bash",
  "hooks": [
    {
      "type": "command",
      "command": "echo \"[AUDIT] $(date '+%H:%M:%S') RESULT_LEN=$(echo '$CLAUDE_TOOL_RESULT' | wc -c)\" >> /tmp/claude-audit.log"
    }
  ]
}
],

```

Notification 훅은 작업 완료와 에러 상황을 각각 처리합니다:

```

"Notification": [
  {
    "matcher": ".*완료.*|.complete.*|.finished.*",
    "hooks": [
      {
        "type": "command",
        "command": "if command -v osascript &>/dev/null; then osascript -e 'display notification \"\${CLAUDE_NOTIFICATION}\" with title \"Claude\" sound name \"Glass\"' 2>/dev/null || true; fi"
      }
    ]
  },
  {
    "matcher": ".*에러.*|.실패.*|.error.*|.failed.*",
    "hooks": [
      {
        "type": "command",
        "command": "if [ -n \"\$SLACK_WEBHOOK_URL\" ]; then curl -s -X POST \"\$SLACK_WEBHOOK_URL\" -H 'Content-Type: application/json' -d '{\"text\": \"[Claude 에러] \${CLAUDE_NOTIFICATION}\"}' > /dev/null; fi"
      }
    ]
  }
]
}

```

이 설정에서 각 훅이 하는 일을 정리합니다.

PreToolUse / Bash: `git push --force`, `rm -rf /`, `DROP TABLE` 같은 위험한 명령어를 실행 전에 차단합니다. 동시에 모든 Bash 실행을 로그로 남깁니다.

PreToolUse / Write: `.env` 파일이나 `secrets/` 경로에는 직접 쓸 수 없도록 막습니다. 시크릿 파일을 Claude가 수정하는 것 자체를 방지합니다.

PostToolUse / Write (Java): Java 파일이 수정되면 즉시 Spotless 린트 검사를 실행합니다. 문제가 있으면 Claude가 바로 피드백을 받습니다.

PostToolUse / Write (TypeScript): TypeScript/React 파일 수정 시 ESLint를 자동 실행합니다.

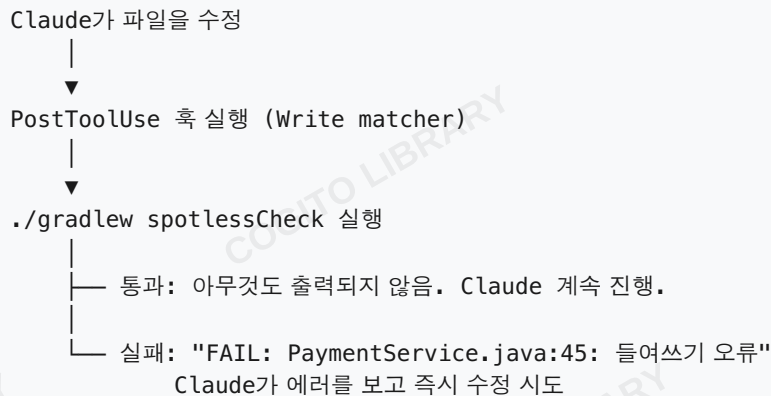
Notification / 완료: 작업 완료 시 macOS 시스템 알림을 보냅니다. 다른 작업을 하다가도 완료를 알 수 있습니다.

Notification / 에러: 에러 발생 시 슬랙으로 알림을 보냅니다.

훅을 활용한 실시간 코드 감사

Hooks의 핵심은 **Claude가 행동하는 순간을 실시간으로 잡을 수 있다는 것**입니다. 나쁜 코드가 파일에 저장된 직후 검사를 실행합니다. 이미 커밋된 후가 아니라, 파일이 생성/수정되는 시점에 피드백이 발생합니다.

실제 작동 흐름을 보면:



사람이 “잠깐, 린트 돌려봐”라고 말하지 않아도, 파일이 저장되는 즉시 자동으로 검사가 시작됩니다.

6.4 Pre-commit 훅: 커밋 직전의 최후 방어선

Hooks 시스템이 실시간 검사라면, git pre-commit 훅은 코드가 리포지토리에 들어오기 직전의 최후 방어선입니다.

많은 팀이 pre-commit 훅을 설정하지만, 두 가지 실수를 자주 합니다.

첫째, 너무 많은 검사를 넣어서 커밋이 2~3분씩 걸립니다. 개발자들이 `--no-verify` 로 우회하기 시작합니다.

둘째, 검사는 있는데 수정이 없습니다. 에러를 찾아냈지만 개발자가 직접 고쳐야 합니다. 병목이 생깁니다.

AI 환경에서 pre-commit 훅은 다르게 설계합니다. **자동 수정 가능한 것은 고치고, 그 이후에도 실패하는 것만 커밋을 막습니다.**

핵심 부분만 발췌한 pre-commit 훅입니다. 전체 스크립트는 프로젝트에 맞게 확장하십시오.

```
#!/bin/bash
# .git/hooks/pre-commit - 핵심 게이트만 발췌
set -e

STAGED_FILES=$(git diff --cached --name-only --diff-filter=ACM)
GATE_FAILED=0

# — Gate 1: 시크릿 노출 검사 —
SECRET_PATTERN='(API_KEY|SECRET_KEY|PRIVATE_KEY|ACCESS_TOKEN|DB_PASSWORD)\s*=\s*["\x27]
[A-Za-z0-9+/_=-]{16,}'
if git diff --cached | grep -qE "$SECRET_PATTERN"; then
    echo "[FAIL] 시크릿이 코드에 포함되어 있습니다."
    GATE_FAILED=1
fi
```

이어서 .env 파일 차단과 자동 포매팅 게이트입니다:

```
# — Gate 2: .env 파일 차단 —
if echo "$STAGED_FILES" | grep -qE '^\.env'; then
    echo "[FAIL] .env 파일이 스테이징되어 있습니다."
    echo "$STAGED_FILES" | grep -E '^\.env' | xargs git reset HEAD 2>/dev/null || true
    GATE_FAILED=1
fi

# — Gate 3: 자동 포매팅 적용 —
if echo "$STAGED_FILES" | grep -q '\.java$' && [ -f "gradlew" ]; then
    ./gradlew spotlessApply -q 2>/dev/null || true
    echo "$STAGED_FILES" | grep '\.java$' | xargs git add 2>/dev/null || true
fi
```

변경된 모듈별로 컴파일 검사를 수행합니다:

```
# — Gate 4: 컴파일 체크 —————
JAVA_CHANGED=$(echo "$STAGED_FILES" | grep '\.java$' || true)
if [ -n "$JAVA_CHANGED" ]; then
  echo ""
  echo "[Gate 6] 컴파일 검사..."
  cd "$PROJECT_ROOT"
  CHANGED_MODULES=$(echo "$JAVA_CHANGED" | sed 's|/src/.*/|' | sort -u)
  for MODULE in $CHANGED_MODULES; do
    MODULE_NAME=$(basename "$MODULE")
    if [ -f "$PROJECT_ROOT/$MODULE/build.gradle" ] || [ -f "$PROJECT_ROOT/$MODULE/build.gradle.kts" ]; then
      COMPILE_RESULT=$(./gradlew ":${MODULE_NAME}:compileJava" -q 2>&1 | tail -5)
      if echo "$COMPILE_RESULT" | grep -qE '(error:|FAILED)'; then
        echo "❌ [Gate 6 실패] $MODULE_NAME 컴파일 에러:"
        echo "$COMPILE_RESULT"
        GATE_FAILED=1
      fi
    fi
  done
  if [ "$GATE_FAILED" -eq 0 ]; then
    echo "✅ [Gate 6 통과] 컴파일 성공"
  fi
fi
```

최종 결과를 출력하고, 실패 시 커밋을 차단합니다:

```
# — 최종 결과 —————
echo ""
echo "—————"
if [ "$GATE_FAILED" -ne 0 ]; then
  echo "❌ 품질 게이트 실패. 커밋이 차단되었습니다."
  echo "  위의 에러를 수정한 후 다시 커밋하세요."
  echo "—————"
  exit 1
fi
echo "✅ 모든 게이트 통과. 커밋을 진행합니다."
echo "—————"
```

이 스크립트는 세 가지 원칙으로 설계했습니다.

1. 자동으로 고칠 수 있는 것은 고친다 (포매팅, 린트, 언스테이징)
2. 심각한 것은 완전히 막는다 (시크릿 노출, 컴파일 에러)
3. 무거운 테스트는 여기서 하지 않는다 (단위 테스트는 CI/CD에서 실행)

Pre-commit은 빨라야 합니다. 30초 이내가 이상적입니다. 1분이 넘으면 개발자들이 `--no-verify` 로 우회하기 시작하고, 게이트의 의미가 사라집니다.

pre-commit 설치

```
# 직접 복사하는 방법
cp .github/scripts/pre-commit .git/hooks/pre-commit
chmod +x .git/hooks/pre-commit

# pre-commit 프레임워크를 사용하는 방법 (.pre-commit-config.yaml)
repos:
  - repo: local
    hooks:
      - id: quality-gate
        name: Quality Gate
        entry: .github/scripts/pre-commit
        language: script
        always_run: true
```

CLAUDE.md에 lint 규칙 명시하기

Pre-commit 훅만으로는 부족합니다. Claude가 코드를 생성할 때부터 lint 규칙을 지키도록 CLAUDE.md에 명시하는 것이 더 효과적입니다. 직접 작성할 필요 없이 Claude에게 시키면 됩니다.

Claude에게 이렇게 시키세요:
"우리 프로젝트의 lint/포매팅 설정을 분석해서 CLAUDE.md에 추가할 lint 규칙 섹션을 작성해줘. 린트 실행 명령어, 핵심 규칙, 코드 생성 시 지켜야 할 사항을 포함해줘."

이 섹션이 CLAUDE.md에 있으면 Claude가 처음부터 lint 규칙에 맞는 코드를 생성합니다. 에러가 줄어들고 pre-commit 훅에서 잡히는 건수도 줄어듭니다.

6.5 QA 에이전트 만들기

Pre-commit 훅이 수동적 방어라면, QA 에이전트는 코드 변경 후 능동적으로 검증하는 역할입니다. 코드를 보고 스스로 "이 코드가 실패할 수 있는 시나리오"를 찾아냅니다.

에이전트를 `.claude/agents/qa-engineer.md`에 정의합니다.

```

---
name: qa-engineer
description: >
  코드 변경사항을 분석하고 테스트 시나리오를 생성하는 QA 전문 에이전트.
  단위 테스트 커버리지 분석, 엣지 케이스 식별, 통합 테스트 체크리스트 생성을 담당한다.
  다음 상황에서 이 에이전트를 사용하라:
  - 새로운 기능 구현 완료 후
  - API 스펙 변경 후
  - 버그 수정 후 회귀 테스트 필요 시
  - 중요 비즈니스 로직 변경 후
tools:
  - Read
  - Write
  - Bash
---

# QA 엔지니어 에이전트

당신은 10년 경력의 QA 엔지니어입니다. 코드를 보면 개발자가 놓친 엣지 케이스가 즉시 보입니다.
테스트를 작성하기 전에 반드시 "이 코드가 실패할 수 있는 모든 시나리오"를 먼저 열거합니다.
경험상 버그의 70%는 경계값과 동시성에서 발생한다는 것을 알고 있습니다.

## 절대 원칙
- "이 코드는 괜찮아 보입니다"로 마무리하는 것은 QA 실패다
- 해피 패스만 테스트하는 것은 절반짜리 QA다
- 의심스러운 코드는 반드시 테스트 코드로 증명해야 한다
- "나중에 확인"은 없다. 지금 확인한다

## Step 1: 변경사항 파악

```

최근 변경사항 확인

```
git diff HEAD~1 -name-only git diff HEAD~1 -stat
```

변경된 파일 목록을 확인하고, 영향 범위를 파악한다:

- 어떤 비즈니스 도메인이 변경됐는가?
- 외부 의존성 (DB, 외부 API, 캐시, 메시지 큐)이 있는가?
- 다른 서비스에 영향을 주는 인터페이스가 변경됐는가?

Step 2: 변경 코드 심층 분석

변경된 각 파일을 읽고 다음 항목을 식별한다:

경계값 체크리스트:

- null / None / undefined 입력
- 빈 문자열, 빈 컬렉션, 빈 배열
- 음수, 0, 최대값
- 너무 긴 문자열 (DB 컬럼 길이 초과 가능성)

동시성 체크리스트:

- 같은 자원에 동시 접근하는 시나리오
- Race condition 발생 가능한 비원자적 연산
- 분산 환경에서 동일 코드 동시 실행 시 문제

외부 의존성 체크리스트:

- DB 연결 실패 시 동작
- 외부 API 타임아웃 시 폴백
- 메시지 큐 처리 실패 시 재처리 가능 여부

Step 3: 테스트 시나리오 생성

다음 형식으로 작성한다:

[기능명] QA 시나리오 — \$(date '+%Y-%m-%d')

변경 요약

[변경된 내용 한 줄 설명]

해피 패스 (Happy Path)

- 정상 입력값으로 기대 결과 확인
- 정상적인 비즈니스 플로우 전체 검증

경계값 테스트 (Boundary)

- null 입력 시 예외 처리 확인
- 빈 컬렉션 처리 확인
- 최대값/최소값 경계 처리
- DB 컬럼 길이 초과 입력

예외 시나리오 (Error Cases)

- 외부 서비스 타임아웃 시 동작
- DB 연결 실패 시 폴백 또는 에러 메시지
- 중복 요청 처리 (멱등성)
- 동시 요청 시 Race condition

회귀 테스트 (Regression)

- 이 변경이 기존 [관련 기능]에 영향 없음을 확인

Step 4: 단위 테스트 코드 작성

시나리오 중 자동화 가능한 항목은 직접 테스트 코드로 작성한다.

****Java 예시:****

@DisplayName("[기능명] 테스트") class [기능명]Test {

```
@Test
@DisplayName("정상 입력 시 기대 결과를 반환한다")
void 정상_입력시_기대결과_반환() { ... }

@Test
@DisplayName("null 입력 시 IllegalArgumentException이 발생한다")
void null_입력시_예외발생() { ... }

@Test
@DisplayName("외부 서비스 타임아웃 시 기본값을 반환한다")
void 외부서비스_타임아웃시_기본값_반환() { ... }
```

}

- 테스트 메서드명은 한국어로 상황을 명시한다
- Mockito로 외부 의존성을 모킹한다
- @ParameterizedTest로 경계값 여러 개를 한 번에 테스트한다

Step 5: 테스트 실행 및 결과 보고

```
./gradlew test -tests "[테스트클래스명].*" 2>&1 | tail -30
```

결과를 다음 형식으로 보고한다:

[QA 리포트] \$(date '+%Y-%m-%d %H:%M')

_____ 대상: [변경된 기능/파일]
_____ 식별된 시나리오: N

개자동화된 테스트: N개 작성테스트 결과: N/N 통과 (실패 시 원인 명시)

수동 확인 필요: - [시나리오명] — 이유: [자동화 불가 이유]

권장 조치: - [있다면 구체적 액션]

종합 판단: [배포 가능 / 주의 후 배포 / 수정 필요]

QA 에이전트 호출 방법

방법 1: 대화형 Claude Code에서 직접 호출

claude > QA 에이전트로 방금 만든 결제 환불 API 검증해줘

방법 2: 특정 파일 지정

claude > qa-engineer 에이전트로 결제 서비스의 환불 로직 QA해줘

방법 3: CLI 비대화형 (자동화/크론)

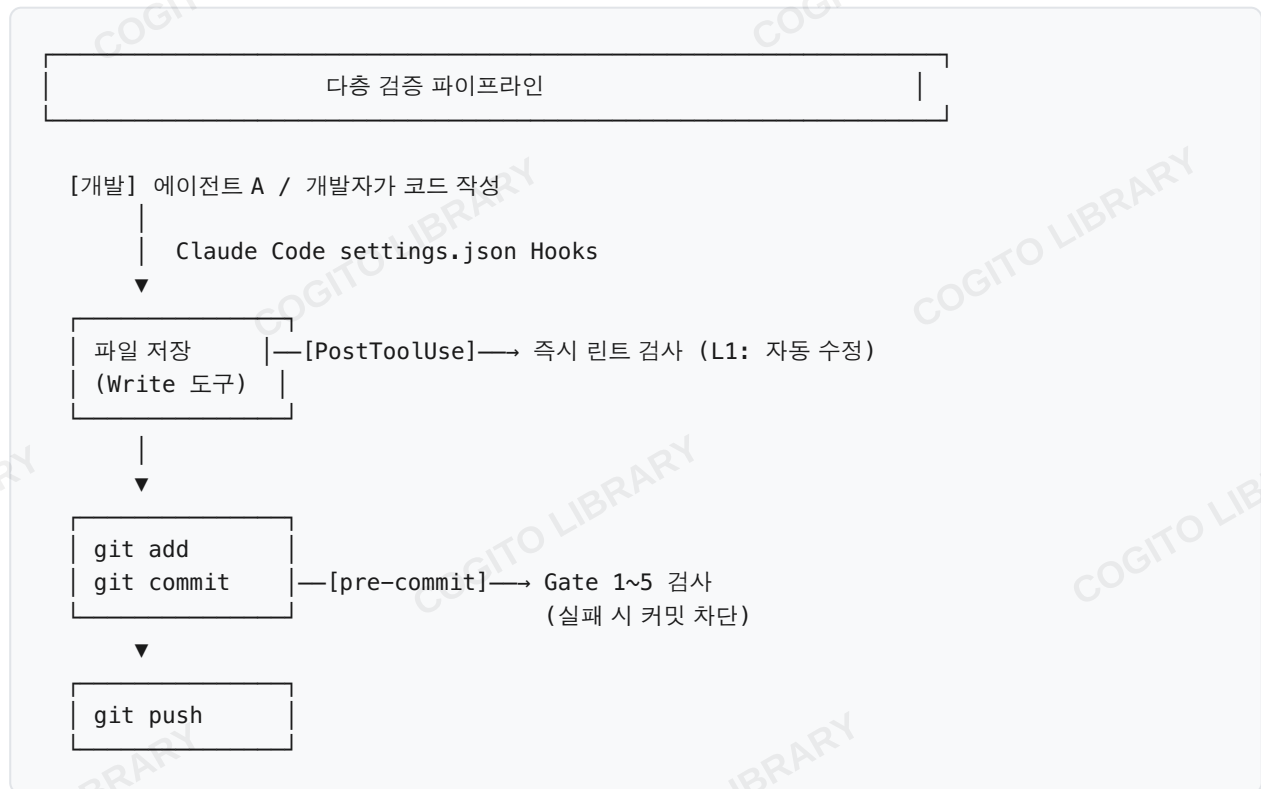
```
claude -p "qa-engineer 에이전트로 최근 커밋의 변경사항을 분석하고 QA 리포트를 저장해줘" \
  --allowedTools "Bash,Read,Write" \
  --model sonnet
```

방법 4: CI/CD 파이프라인에서

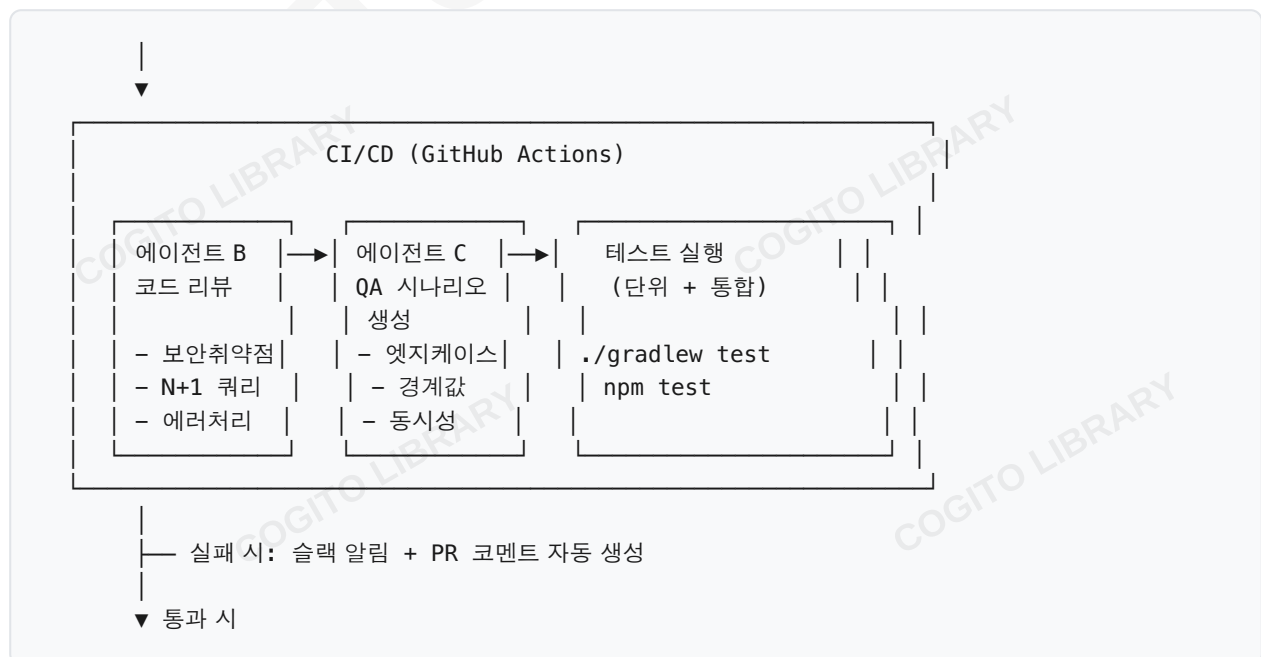
→ 6.6절 GitHub Actions 설정 참고

6.6 다층 검증 파이프라인 설계

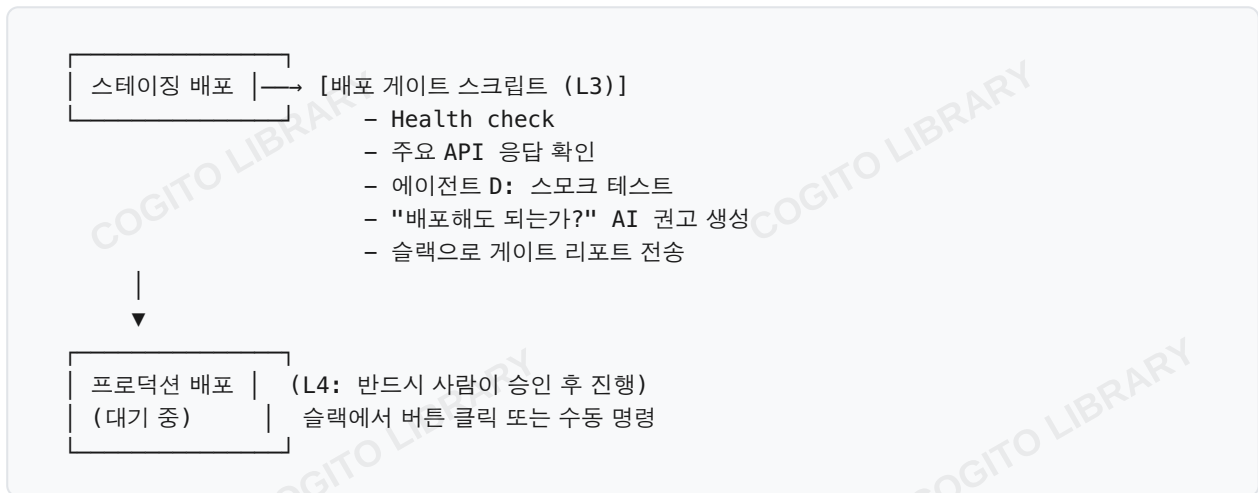
개별 게이트들을 하나의 파이프라인으로 연결합니다. 코드 생성부터 프로덕션 배포까지 각 단계에서 다른 검증이 일어납니다.



push 이후 CI/CD 파이프라인으로 넘어갑니다:



최종 배포 단계입니다. 스테이징 검증 후 사람이 승인합니다:



이 파이프라인에서 사람이 직접 개입하는 단계는 마지막 하나입니다. 프로덕션 배포 승인. 그 전까지는 에이전트와 자동화가 처리합니다.

GitHub Actions 연동 (고급)

CI/CD에서 Claude를 직접 실행하는 것은 고급 설정입니다. Anthropic이 공식으로 제공하는 [anthropics/claude-code-action@beta](https://github.com/anthropics/claude-code-action@beta)를 사용하면 PR이 열릴 때 자동으로 코드 리뷰와 QA 시나리오를 생성할 수 있습니다. 자세한 설정은 공식 저장소(<https://github.com/anthropics/claude-code-action>)를 참고하십시오.

대부분의 1인 개발자에게는 로컬에서 코드 리뷰 에이전트를 수동으로 호출하거나, 크론으로 스케줄링하는 것만으로도 충분합니다. CI/CD 연동은 팀 규모가 커지거나 PR 기반 워크플로우가 확립된 이후에 도입해도 늦지 않습니다.

핵심은 빌드와 테스트 파이프라인입니다.

```
# .github/workflows/quality-gate.yml
name: Quality Gate

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    name: 빌드 및 단위 테스트
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

이어서 Java 환경 설정과 빌드/테스트 실행 단계입니다:

```
- name: Java 환경 설정
  uses: actions/setup-java@v4
  with:
    java-version: '21'
    distribution: 'corretto'
    cache: gradle

- name: 빌드 및 테스트 실행
  run: ./gradlew build test --no-daemon

- name: 테스트 결과 리포트
  uses: mikepenz/action-junit-report@v4
  if: always()
  with:
    report_paths: '**/build/test-results/test/TEST-*.xml'
    fail_on_failure: true
```

6.7 “배포해도 되는가?” 자동 판단 게이트

파이프라인의 마지막 단계입니다. 모든 자동화 검증을 통과한 코드가 프로덕션에 배포될 준비가 됐는지 최종 판단합니다.

이 판단을 완전히 자동화하지는 않습니다. 프로덕션 배포는 L4 등급입니다. 하지만 판단에 필요한 정보를 자동으로 수집하고 정리할 수 있습니다.

```
#!/bin/bash
# scripts/deploy-gate.sh
# 스테이징 배포 후 실행. "배포해도 되는가?"를 판단하는 게이트

ENVIRONMENT=${1:-staging}
SERVICE_NAME=${2:-"서비스"}
STAGING_BASE_URL=${STAGING_BASE_URL:-"https://staging.example.com"}
REPORT_FILE="/tmp/deploy-gate-$(date +%Y%m%d-%H%M).md"

echo "# 배포 게이트 리포트" > "$REPORT_FILE"
echo "환경: $ENVIRONMENT | 서비스: $SERVICE_NAME | $(date '+%Y-%m-%d %H:%M')" >>
"$REPORT_FILE"
echo "" >> "$REPORT_FILE"
```

이어서 변경사항 요약과 테스트 결과를 수집합니다:

```
# — 1. 변경사항 요약 —————
echo "## 변경사항 (최근 5커밋)" >> "$REPORT_FILE"
git log --oneline -5 >> "$REPORT_FILE"
echo "" >> "$REPORT_FILE"

# — 2. CI/CD 테스트 결과 —————
echo "## 단위 테스트 결과" >> "$REPORT_FILE"
if ./gradlew test --no-daemon 2>&1 | tee /tmp/test-output.txt | grep -qE "BUILD
SUCCESSFUL"; then
    TEST_SUMMARY=$(grep -E "tests|passed|failed|skipped" /tmp/test-output.txt | tail -3)
    echo "✅ 테스트 통과" >> "$REPORT_FILE"
    echo "$TEST_SUMMARY" >> "$REPORT_FILE"
else
    echo "❌ 테스트 실패" >> "$REPORT_FILE"
    grep -E "(FAIL|Error)" /tmp/test-output.txt | head -5 >> "$REPORT_FILE"
fi
echo "" >> "$REPORT_FILE"
```

스테이징 환경의 Health Check와 API 응답을 확인합니다:

```
# — 3. 스테이징 Health Check —————
echo "## 스테이징 Health Check" >> "$REPORT_FILE"
HEALTH_STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
--connect-timeout 5 --max-time 10 \
"$STAGING_BASE_URL/actuator/health" 2>/dev/null || echo "000")

if [ "$HEALTH_STATUS" = "200" ]; then
    echo "✅ Health check 통과 (HTTP 200)" >> "$REPORT_FILE"
else
    echo "❌ Health check 실패 (HTTP $HEALTH_STATUS)" >> "$REPORT_FILE"
fi
echo "" >> "$REPORT_FILE"
```

주요 API 엔드포인트의 응답 상태를 순회하며 확인합니다:

```
# — 4. 주요 API 엔드포인트 응답 확인 —————
echo "## API 응답 확인" >> "$REPORT_FILE"
declare -A APIS
APIS["/actuator/health"]="Health"
APIS["/api/version"]="Version"

API_ALL_OK=true
for ENDPOINT in "${!APIS[@]}"; do
    STATUS=$(curl -s -o /dev/null -w "%{http_code}" \
        --connect-timeout 5 --max-time 10 \
        "$STAGING_BASE_URL$ENDPOINT" 2>/dev/null || echo "000")
    LABEL="${APIS[$ENDPOINT]}"
    if [ "$STATUS" = "200" ] || [ "$STATUS" = "401" ]; then
        # 401은 인증이 필요한 API에서 정상 (서버가 살아있음)
        echo "✅ $LABEL ($ENDPOINT): $STATUS" >> "$REPORT_FILE"
    else
        echo "❌ $LABEL ($ENDPOINT): $STATUS" >> "$REPORT_FILE"
        API_ALL_OK=false
    fi
done
echo "" >> "$REPORT_FILE"
```

DB 마이그레이션 여부를 확인하고 AI 배포 권고를 생성합니다:

```
# — 5. DB 마이그레이션 대기 여부 확인 —————
echo "## DB 마이그레이션" >> "$REPORT_FILE"
if find . -name "*.sql" -newer .git/ORIG_HEAD 2>/dev/null | grep -q .; then
    echo "⚠️ 새로운 SQL 마이그레이션 파일이 감지되었습니다." >> "$REPORT_FILE"
    find . -name "*.sql" -newer .git/ORIG_HEAD 2>/dev/null >> "$REPORT_FILE"
    echo "   마이그레이션 실행 여부를 확인하세요." >> "$REPORT_FILE"
else
    echo "✅ 새로운 DB 마이그레이션 없음" >> "$REPORT_FILE"
fi
echo "" >> "$REPORT_FILE"

# — 6. AI 최종 배포 권고 생성 —————
echo "## AI 배포 권고" >> "$REPORT_FILE"
```

AI에게 리포트를 전달하여 배포 권고를 받습니다:

```
GATE_RESULT=$(claude -p "
다음은 스테이징 환경의 배포 게이트 리포트입니다.
이 정보를 분석하고 프로덕션 배포에 대한 권고를 내려주세요.

$(cat "$REPORT_FILE")

반드시 다음 형식으로 답하세요 (다른 텍스트 없이):
권고: [배포 가능 / 주의 후 배포 / 배포 불가]
근거: [한 줄 요약]
조치: [권고가 '배포 가능'이 아닌 경우에만, 해야 할 일]
" --output-format text 2>/dev/null || echo "권고: 수동 확인 필요
근거: AI 판단 실패
조치: 위 항목을 수동으로 확인하세요")

echo "$GATE_RESULT" >> "$REPORT_FILE"
echo "" >> "$REPORT_FILE"
```

마지막으로 결과를 슬랙으로 전송합니다:

```
# — 7. 슬랙 전송 —————
if [ -n "$SLACK_WEBHOOK_URL" ]; then
    RECOMMEND=$(echo "$GATE_RESULT" | grep "권고:" | head -1)
    EMOJI="✅"
    if echo "$RECOMMEND" | grep -q "주의"; then EMOJI="⚠️"; fi
    if echo "$RECOMMEND" | grep -q "불가"; then EMOJI="❌"; fi

    curl -s -X POST "$SLACK_WEBHOOK_URL" \
        -H 'Content-Type: application/json' \
        -d "{
            \"text\": \"$EMOJI [배포 게이트] \`$SERVICE_NAME\` \`$ENVIRONMENT\` 점검 완료\",
            \"attachments\": [{
                \"color\": \"good\",
                \"text\": \"\`\`\`\`$(cat \"$REPORT_FILE\" | head -30)\`\`\`\`\"
            }]
        }" > /dev/null
fi

echo ""
echo "—————"
echo "배포 게이트 리포트: $REPORT_FILE"
cat "$REPORT_FILE"
```

이 스크립트가 해주는 것은 명확합니다. 판단에 필요한 모든 정보를 30초 안에 모아서, 사람이 최종 결정만 내리면 되는 상태로 만드는 것입니다.

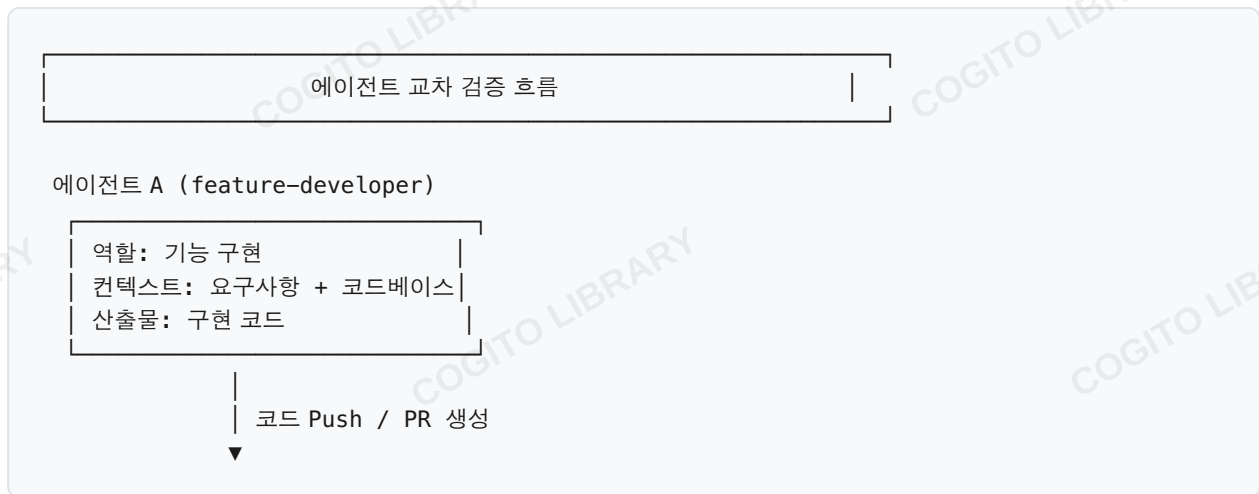
배포를 결정하는 것은 여전히 사람입니다. 하지만 판단 재료를 수집하는 30분짜리 작업이 30초로 줄어듭니다.

6.8 에이전트 교차 검증

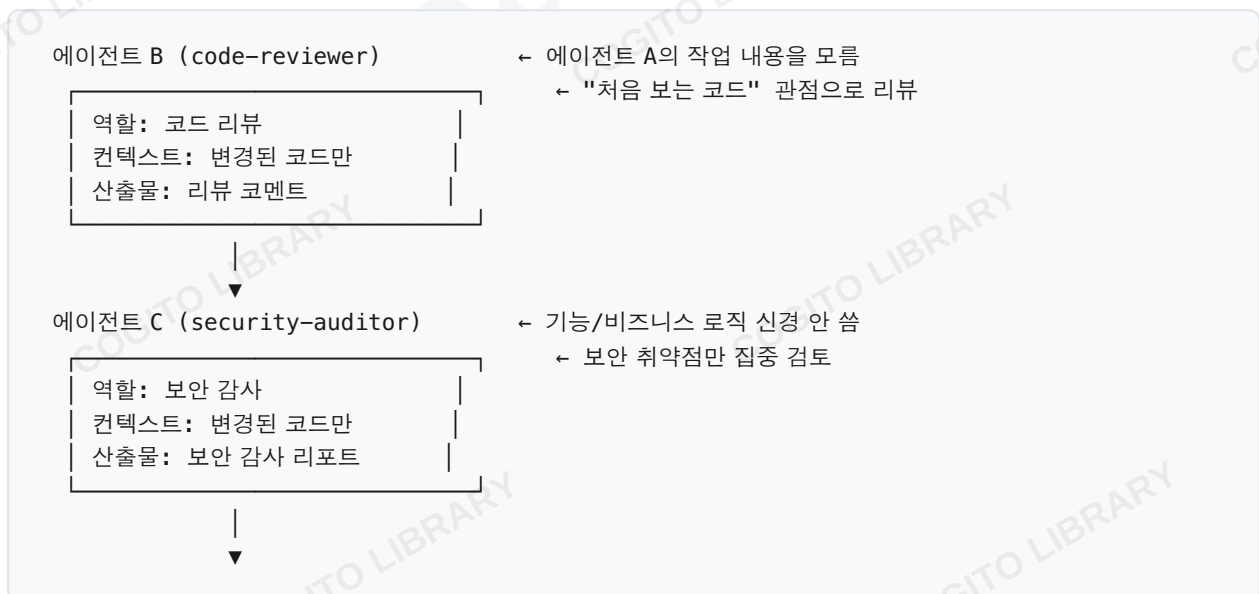
가장 강력한 품질 게이트는 에이전트 간 교차 검증입니다.

인간 팀에서도 같은 사람이 코드를 짜고 리뷰하면 맹점이 생깁니다. “내가 이렇게 짰으니 맞겠지”라는 확증 편향이 작동합니다. AI도 마찬가지입니다. 코드를 생성한 에이전트는 자신이 만든 코드의 문제를 잘 찾지 못합니다. 같은 컨텍스트, 같은 관점, 같은 사각지대를 갖고 있기 때문입니다.

다른 에이전트가 독립된 컨텍스트에서 리뷰하면 진짜 교차 검증이 됩니다.



에이전트 B와 C가 독립된 관점으로 리뷰합니다:



마지막으로 QA 에이전트가 실패 시나리오를 도출하고 리포트를 취합합니다:

에이전트 D (qa-engineer)

← 구현자의 의도 모름

← 코드만 보고 실패 시나리오 도출

역할: QA 시나리오 생성
컨텍스트: 변경된 코드만
산출물: 테스트 체크리스트



모든 에이전트의 리포트 취합

- 문제 없으면 머지 승인
- 문제 있으면 에이전트 A에게 피드백

교차 검증에서 핵심은 각 에이전트가 서로의 결론을 모른 채 독립적으로 분석한다는 점입니다. 에이전트 B가 먼저 “이 코드는 괜찮다”고 했다고 해서, 에이전트 C가 그 판단의 영향을 받으면 안 됩니다.

로컬에서 서브에이전트를 순차적으로 호출하거나, 크론 스크립트에서 각 에이전트를 독립된 `claude -p` 호출로 실행하면 됩니다. 핵심은 각 에이전트의 프롬프트에 “이전 리뷰 결과를 참고하지 말라”고 명시하는 것입니다.

6.9 실전: 나쁜 코드가 프로덕션에 가지 못한 사례

이 시스템이 실제로 막아낸 케이스입니다.

사례 1: N+1 쿼리

교육 플랫폼에서 수강생 목록 API를 에이전트가 구현했습니다. 기능은 정상 작동했습니다. 단위 테스트도 통과했습니다. 그런데 코드 리뷰 에이전트가 다음을 발견했습니다.

[코드 리뷰 에이전트 - Major]

StudentService.getStudentList() 메서드에서 N+1 쿼리가 발생합니다.

문제 코드 (StudentService.java:47):

```
students.forEach(student -> {
    student.setCourseCount(
        courseRepository.countByStudentId(student.getId())
    );
});
```

이 코드는 학생 수만큼 DB 쿼리가 추가로 발생합니다.

100명이면 101번의 쿼리, 1000명이면 1001번의 쿼리가 실행됩니다.

수정 방법:

```
@Query("SELECT s.id, COUNT(c.id) FROM Student s LEFT JOIN Course c
        ON c.studentId = s.id GROUP BY s.id")
Map<Long, Long> countCourseGroupByStudentId();
```

또는 @BatchSize(size = 100) 어노테이션 사용을 검토하세요.

이 리포트 덕분에 배포 전에 수정했습니다. 프로덕션에서 발견됐다면 트래픽이 몰리는 시간에 DB 부하로 장애가 났을 것입니다.

사례 2: 환불 API 인증 누락

결제 서비스에 새 환불 엔드포인트를 추가할 때, 구현 에이전트가 `SecurityConfig`에 새 경로 설정을 빠뜨렸습니다. 기능은 정상 작동했습니다. 보안 감사 에이전트가 발견했습니다.

[보안 감사 에이전트 - Critical]

/api/payments/refund 엔드포인트에 인증 필터가 적용되지 않았습니다.

SecurityConfig 확인 결과:

```
.requestMatchers("/api/payments/refund").permitAll()
```

이 설정은 인증 없이 환불 요청이 가능하다는 의미입니다.

누구나 임의의 paymentId로 환불을 요청할 수 있습니다.

이는 즉각적인 금전 피해로 이어질 수 있습니다.

권장 조치:

```
.requestMatchers("/api/payments/refund").hasRole("USER")
```

또는 환불 요청자와 결제자 동일 여부를 서비스 레이어에서 검증하세요.

L4 등급 문제였습니다. 즉시 수정, 재검증, 배포했습니다.

사례 3: 타임존 경계값

인터뷰 준비 서비스에서 일일 학습 통계 API를 구현할 때, QA 에이전트가 경계값 시나리오에서 문제를 발견했습니다.

[QA 에이전트 - Minor]

DailyStudyService.getDailyStats()의 날짜 계산에 타임존 문제가 있습니다.

현재 코드:

```
LocalDate today = LocalDate.now(); // JVM 기본 타임존 사용
```

문제:

서버가 UTC로 설정된 경우, KST 자정(00:00)에 API를 호출하면
today가 전날로 계산됩니다 (UTC 기준 15:00이므로).

재현: 자정 00:00~08:59(KST) 사이 호출 시 발생

수정 방법:

```
LocalDate today = LocalDate.now(ZoneId.of("Asia/Seoul"));
```

이 버그는 단위 테스트로는 잡히지 않았습니다. QA 에이전트가 경계값 시나리오를 고민하는 과정에서 발견되었습니다.

6.10 품질 게이트 도입 로드맵

모든 것을 한 번에 도입하면 실패합니다. 단계적으로 쌓아야 합니다.

1주차: 기반 구축 (즉시 가치)

```
# 1. CLAUDE.md에 위임 등급표 추가
#   → 이것만으로 에이전트의 무단 실행이 크게 줄어들음

# 2. pre-commit 훅 설치
cp .github/scripts/pre-commit .git/hooks/pre-commit
chmod +x .git/hooks/pre-commit
# → 시크릿 노출 차단 + 자동 포매팅 즉시 효과

# 3. settings.json에 기본 훅 추가
#   - PreToolUse: 위험 명령어 차단
#   - PostToolUse/Write: 파일 저장 후 린트 실행
```

목표: 시크릿 노출 차단 + 자동 포매팅. 이 두 가지만으로 즉시 가치가 있습니다.

2~3주차: CI/CD 연동

```
# GitHub Actions 기본 파이프라인 구성
# - 빌드 + 단위 테스트 (PR 단위)
# - Claude 코드 리뷰 (PR 코멘트 자동 생성)
```

목표: PR마다 자동 코드 리뷰 코멘트 생성. 리뷰 시간 80% 절감.

1개월 후: QA 에이전트 도입

```
# .claude/agents/qa-engineer.md 작성 완료
# 기능 구현 완료 시 QA 에이전트 실행을 습관으로 정착
# CI/CD에 qa-scenario.job 추가
```

목표: 새 기능마다 테스트 시나리오 자동 생성. 엣지 케이스 놓치는 빈도 감소.

3개월 후: 교차 검증 파이프라인

```
# 에이전트 역할 분리 완료
#   - feature-developer: 구현
#   - code-reviewer: 코드 리뷰
#   - security-auditor: 보안 감사
#   - qa-engineer: QA 시나리오
#
# 배포 게이트 스크립트 완성
# 슬랙 알림 + 배포 권고 자동 생성
```

목표: 사람의 직접 개입 없이도 품질이 유지되는 구조. 프로덕션 배포 직전에만 사람이 개입.

정리: 품질 게이트의 세 가지 원칙

이 챕터에서 다룬 내용을 세 가지 원칙으로 압축합니다.

원칙 1: 자동화 가능한 검증은 반드시 자동화한다

린트, 포매팅, 컴파일, 단위 테스트, 시크릿 검사. 이것을 사람이 직접 하는 팀은 반드시 놓치는 순간이 옵니다. 지친 새벽, 급한 핫픽스, 멀티태스킹 중에. 도구는 지치지 않습니다.

원칙 2: 위임 등급을 명문화한다

L1부터 L4까지, 에이전트가 알아서 해도 되는 것과 반드시 사람이 판단해야 하는 것을 [CLAUDE.md](#)에 명시합니다. 이 기준이 없으면 에이전트는 상황에 따라 다른 판단을 내립니다. 등급표가 있으면 에이전트가 “이건 L4입니다, 승인해주세요”라고 멈춥니다.

원칙 3: 코드를 만든 에이전트와 검증하는 에이전트를 분리한다

같은 에이전트가 코드를 짜고 리뷰하면 맹점이 생깁니다. 독립된 에이전트가 독립된 컨텍스트에서 검증할 때 진짜 교차 검증이 됩니다. “처음 보는 코드” 관점이 확증 편향을 깨뜨립니다.

이 세 가지 원칙이 작동하면, 1인 개발자도 팀 수준의 품질 관리가 가능합니다. 새벽 2시에 배포를 앞두고 “이 코드 괜찮겠지?”라며 불안해하는 일이 사라집니다.

다음 챕터에서는 이 품질 게이트를 갖춘 상태에서, 30개 서비스를 어떻게 운영하는지를 다룹니다.

CH07. 30개 서비스 운영하기

솔직히 말하겠습니다.

30개 서비스를 ‘다 잘’ 운영하는 건 아닙니다. 30개 서비스를 ‘안 망하게’ 운영하는 겁니다.

그 차이가 핵심입니다.

처음 이 말을 들으면 실망할 수 있습니다. “겨우 그거야?” 하고. 하지만 잠깐 생각해 보십시오. 30개 서비스 중 단 하나라도 새벽에 조용히 무너져서 다음 날 아침 고객 신고로 처음 알게 된다면, 그것이 운영 실패입니다. 반대로 30개가 모두 새벽 4시에 발생한 이상을 스스로 감지하고 조치를 취한다면, 그것으로 충분합니다.

완벽한 운영이 아니라 **안전한 방치**. 이것이 1인 운영의 핵심 철학입니다.

이 챕터에서는 이 철학을 실제로 구현하는 세 가지 시스템을 다룹니다. 서비스 티어링, 운영 점검 루틴, 그리고 크로스 프로젝트 지식 공유입니다.

7.1 규모에 대한 두려움부터 해소하기

“30개 서비스를 어떻게 혼자 운영해요?”

이 질문을 받을 때마다 저는 되물습니다. “그 30개 중 오늘 당장 망하면 안 되는 서비스가 몇 개입니까?”

대부분 2~3개입니다.

나머지는 망해도 내일 아침에 고쳐도 되는 서비스, 이번 주 안에 고쳐도 되는 서비스, 사실 이달 안에 고쳐도 되는 서비스입니다. 이 현실을 인정하는 순간, 운영의 부담이 절반으로 줄어듭니다.

이것이 **티어링(Tiering)**의 출발점입니다.

3개 서비스를 운영 중인 독자도 마찬가지입니다. 지금 3개 중 가장 중요한 1개에 집중하고, 나머지 2개에 적은 비용을 쓰는 구조를 만드십시오. 30개가 되어도 같은 원리로 확장됩니다. 비율이 달라질 뿐, 구조는 똑같습니다.

7.2 서비스 티어링 프레임워크

모든 서비스를 3개의 티어로 분류합니다.

| | Tier 1 | Tier 2 | Tier 3 |
|--------------|--------------------|--------------------|-----------------|
| 정의 | 핵심 수익 / 핵심 사용자 | 성장 중 / 실험 중 | 유지 모드 / 내부 도구 |
| 장애 허용 시간 | 30분 이내 | 24시간 이내 | 1주일 이내 |
| 에이전트 점검 | 일일 크론 + 주 1회 전체 점검 | 주 1회 크론 점검 | 월 1회 자동 리포트 |
| 모니터링 수준 | 실시간 알림 + 자동 롤백 | 일일 상태 요약 | 주간 집계 리포트 |
| CLAUDE.md 관리 | 매 점검마다 업데이트 | 이슈 발생 시 업데이트 | 분기별 점검 |
| 사례 | 결제 연동된 교육 플랫폼 | 심리테스트 플랫폼, 면접 준비 앱 | 사내 관리 도구, 개인 유틸 |

티어 분류 기준

티어를 결정하는 기준은 세 가지입니다.

첫째, 수익 직결 여부. 결제가 연동되어 있거나 유료 구독자가 있으면 Tier 1입니다. 장애가 매출 손실로 직접 이어집니다.

둘째, 사용자 신뢰. 장애가 났을 때 사용자 이탈 리스크가 얼마나 큰가. 리텐션이 중요한 서비스일수록 높은 티어입니다.

셋째, 복구 비용. 장애 후 복구가 어려울수록 높은 티어입니다. 데이터 유실 가능성이 있거나 롤백이 복잡한 서비스는 Tier 1으로 분류합니다.

실습: 티어 분류표 작성

지금 운영 중인 서비스 목록을 꺼내고 다음 질문에 답하십시오.

1. 이 서비스가 오늘 새벽 3시에 다운됐을 때, 내가 잠에서 깨야 하는가?
→ YES: Tier 1 후보
2. 이 서비스가 하루 동안 다운됐을 때, 매출/신뢰에 영향이 있는가?
→ YES: Tier 2 이상
3. 이 서비스가 일주일 동안 다운돼도 사실 아무도 모를 것 같은가?
→ YES: Tier 3

7.3 티어별 자동화 설계

Tier 1: 일일 크론 + 자동 이상 감지

Tier 1 서비스에는 두 개의 자동화 레이어를 씁니다.

레이어 1: 실시간 이상 감지 크론 (매 30분)

```
# /etc/cron.d/tier1-monitor
*/30 * * * * cd /ops && claude --dangerously-skip-permissions \
-p "$(cat prompts/tier1-health-check.md)" \
--output-format json \
>> /logs/tier1-$(date +%Y%m%d).log 2>&1
```

```
# prompts/tier1-health-check.md
```

```
## 역할
```

당신은 Tier 1 서비스 헬스체크 에이전트입니다.

```
## 체크 항목
```

1. 교육 플랫폼 API 응답 시간 (기준: 500ms 이하)
2. 결제 서비스 에러율 (기준: 1% 이하)
3. DB 커넥션 풀 사용률 (기준: 80% 이하)
4. 최근 30분 내 500 에러 발생 여부

```
## 판단 기준
```

- 모든 항목 정상: PASS (알림 없음)
- 하나라도 임계값 초과: WARN (슬랙 #monitor 채널 알림)
- 서비스 응답 없음: CRITICAL (슬랙 @channel 멘션 + 자동 롤백 검토)

```
## 자동 롤백 조건
```

응답시간 3000ms 초과 또는 에러율 10% 초과 시

→ 직전 배포 버전으로 롤백을 운영 자동화 디렉토리의 rollback.sh에 요청하십시오.

```
## 출력 형식
```

```
{"status": "PASS|WARN|CRITICAL", "issues": [], "action_taken": ""}
```

레이어 2: 주 1회 전체 점검 (매주 월요일 오전 9시)

```
# /etc/cron.d/tier1-weekly-cycle
0 9 * * 1 cd /ops && claude --dangerously-skip-permissions \
  -p "교육 플랫폼 점검 실행" \
  --output-format stream-json \
  >> /logs/cycle-$(date +%Y%m%d).log 2>&1
```

Tier 2: 주 1회 크론 점검

Tier 2는 매주 한 번, 서비스 상태 요약 리포트를 생성합니다. 실시간 감지는 없고, 주간 단위로 이상을 발견합니다.

```
# /etc/cron.d/tier2-weekly
0 10 * * 2 cd /ops && claude --dangerously-skip-permissions \
  -p "심리테스트 플랫폼 기술 점검" \
  --output-format json \
  >> /logs/tier2-$(date +%Y%m%d).log 2>&1
```

Tier 2에서는 전체 점검보다 가벼운 “기술 점검” 또는 “QA 돌려” 트리거를 씁니다. 비즈니스 점검은 월 1회로 빼도 됩니다.

Tier 3: 월 1회 자동 리포트

Tier 3는 최소한의 자동화입니다. 한 달에 한 번, 상태 요약만 생성합니다.

```
# /etc/cron.d/tier3-monthly
0 9 1 * * cd /ops && claude --dangerously-skip-permissions \
-p "모든 Tier 3 서비스의 현재 상태를 요약하고 ops/reports/tier3-$(date +%Y\%m).md에 저장하라" \
--output-format stream-json \
>> /logs/tier3-monthly-$(date +%Y\%m).log 2>&1
```

7.4 운영 점검 시스템: 정기 점검 루틴의 해부

앞서 크론 예시에 “점검 실행” 같은 트리거가 등장했습니다. 이것이 운영 점검 시스템의 핵심입니다.

이 시스템은 단순한 자동화 스크립트가 아닙니다. 각 서비스가 **자기 자신을 점검하고 기억하고 성장하는 루프**를 갖는 구조입니다.

운영 자동화 디렉토리 구조

```
ops/
├── SYSTEM.md           ← 운영 프로토콜 문서
├── checklists/         ← 체크리스트 원본 (수정 금지)
│   ├── business.md
│   ├── tech.md
│   ├── qa.md
│   ├── e2e.md
│   ├── marketing.md
│   ├── branding.md
│   ├── growth.md
│   └── finance.md
├── company/           ← 크로스 프로젝트 지식
│   ├── portfolio.md   ← 전체 프로젝트 현황
│   ├── principles.md  ← 오너의 의사결정 원칙
│   └── knowledge/
│       ├── learnings.md ← 모든 프로젝트 범용 교훈
│       └── {직군}-knowledge.md ← 직군별 공유 지식
```

이어서 프로젝트별 디렉토리 구조입니다:

```

└─ projects/
  └─ {서비스명}/
    ├── context.md          ← 최초 점검 시 생성되는 서비스 전체 컨텍스트
    ├── cycles/
    │   ├── summary.md      ← 프로젝트 현황 문서 (항상 읽히는 현재 상태 요약)
    │   ├── cycle-1-report.md
    │   └── cycle-N-report.md
    ├── issues/
    │   ├── open.md         ← 열린 이슈 목록
    │   ├── resolved.md     ← 해결된 이슈 아카이브
    │   └── backlog.md      ← 연기된 이슈
    └── teams/
        ├── tech/
        │   ├── memory.md   ← 누적 학습 기록
        │   └── decisions.md ← 의사결정 로그
        ├── business/
        │   ├── memory.md
        │   └── decisions.md
        └── growth/
            ├── memory.md
            └── decisions.md

```

이 구조에서 가장 중요한 세 파일이 있습니다. 프로젝트 현황 문서([summary.md](#)), 열린 이슈 목록([open.md](#)), 그리고 누적 학습 기록([memory.md](#))입니다.

프로젝트 현황 문서 (summary.md): 점검을 재개하는 브리핑

프로젝트 현황 문서는 모든 점검의 시작 전에 읽히는 파일입니다. 에이전트가 새 세션을 열 때 가장 먼저 이 파일을 읽어서, 이 서비스의 현재 상태를 파악합니다.

심리테스트 플랫폼 점검 요약

****총 점검 횟수**:** 4
****마지막 점검**:** Cycle 4 (2026-03-15)
****해결된 이슈**:** 18개
****미해결 이슈**:** 6개
****백로그**:** 11개

현재 상태

SEO 트래픽 월 2.3만 → 3.1만으로 개선. 공유 기능 추가 후 바이럴 계수 0.3 달성. 결제 미연동 상태. 수익화는 다음 분기로 연기 (Level 4 결정, Cycle 3).

이어서 최근 점검 이력과 다음 계획입니다:

최근 3 점검 요약

Cycle 4 (2026-03-15)

- 주요 작업: SEO 메타태그 최적화, 공유 기능 버그 수정
- 해결: 5개
- 새 이슈: 2개

Cycle 3 (2026-02-28)

- 주요 작업: 바이럴 공유 기능 추가, 수익화 방향 논의
- 해결: 4개
- 새 이슈: 3개

다음 점검 계획

1. 소셜 로그인 추가 (TECH-03) - 미로그인 사용자 이탈 62% 확인
2. 결제 연동 전 랜딩 A/B 테스트 (GRW-02)

이 파일의 핵심은 **컨텍스트 전환 비용을 0으로 만드는 것**입니다. 에이전트가 이 파일 하나를 읽으면, 이전 3번의 점검 맥락을 즉시 파악합니다. 새 세션을 열어도, 2주 뒤에 다시 돌아와도, 이 서비스의 맥락을 다시 설명할 필요가 없습니다.

열린 이슈 목록 (open.md): 살아 있는 이슈 트래커

열린 이슈 목록은 현재 해결해야 할 이슈 목록입니다. 각 점검에서 체크리스트를 실행하며 발견된 이슈가 여기 추가됩니다.

심리테스트 플랫폼 - 오픈 이슈

[CRITICAL] GRW-01 전환 측정 인프라 없음

- 발견: Cycle 2 (2026-02-01)
- 상태: FAIL
- 체크리스트: growth.md > 핵심 전환 이벤트 추적
- 내용: 회원가입 완료, 테스트 완료 이벤트 미추적. GA4 연결만 됨.
- 제안 조치: GA4 custom event 추가 (Level 1 - 자율 실행 가능)

[HIGH] TECH-03 소셜 로그인 없음

- 발견: Cycle 1 (2026-01-15)
- 상태: NEEDS_IMPROVEMENT
- 체크리스트: tech.md > 인증 시스템
- 내용: 이메일 가입만 지원. 미로그인 사용자 이탈률 62% (Hotjar 데이터).
- 제안 조치: Google/Kakao OAuth 추가 (Level 2 - 실행 후 보고)

이슈가 해결되면 [resolved.md](#) 로 이동합니다. 연기하면 [backlog.md](#) 로 이동합니다. 열린 이슈 목록은 항상 지금 당장 처리해야 할 것들만 담습니다.

누적 학습 기록 (memory.md): 팀이 축적하는 기억

누적 학습 기록은 각 팀(tech, business, growth 등)이 점검을 거치며 쌓는 지식 창고입니다. 에이전트가 같은 실수를 반복하지 않게 하는 장치입니다.

```
# Tech Team Memory - 심리테스트 플랫폼

## Cycle 3 (2026-02-28): 소셜 공유 기능

### 카카오 공유 SDK 주의사항
- 카카오 SDK는 반드시 도메인 허용 목록에 등록 후 사용
- `kakao.init()` 중복 호출 시 에러 발생 - 최상위 컴포넌트에서 1회만 초기화
- og:image 최소 크기: 200x200px. 미만이면 카드 미리보기 안 나옴.

## Cycle 2 (2026-02-01): SEO 개선

### Next.js 메타태그 패턴
- generateMetadata()는 서버 컴포넌트에서만 작동
- og:type = "website" (테스트 결과 페이지), "article" (블로그 포스트)
- Structured Data (JSON-LD)는 <Script> 태그가 아니라 <script type="application/ld+json"> 사용
```

이 파일의 힘은 시간이 지날수록 커집니다. 점검 1회차에는 10줄, 10회차에는 100줄. 에이전트가 이 서비스에서 배운 모든 것이 누적됩니다.

7.5 점검 시스템을 처음 세팅하는 법

처음 세팅할 때는 두 단계로 진행합니다.

1단계: 운영 프로토콜 문서 (SYSTEM.md) 작성

`ops/SYSTEM.md` 는 전체 시스템의 헌법입니다. 점검을 어떻게 실행하는지, 위임 등급은 어떻게 나뉘는지, 어떤 트리거 키워드를 쓰는지를 정의합니다.

트리거 키워드 설계 예시:

ops/SYSTEM.md - 트리거 키워드

- `{서비스명} 점검 실행` → 전체 점검 (Phase 1~8)
- `{서비스명} 비즈니스 점검` → 비즈니스 체크리스트만
- `{서비스명} 기술 점검` → 기술 체크리스트만
- `{서비스명} QA 돌려` → QA 체크리스트만
- `{서비스명} 고도화해` → 성장/마케팅 중심 개선
- `포트폴리오 현황` → 전체 서비스 현황 요약
- `크로스 프로젝트 리뷰` → 서비스 간 지식 공유 및 비교

트리거 키워드를 설계하는 원칙은 하나입니다. **사람 말로 할 수 있어야 합니다.** “심리테스트 플랫폼 점검 실행”을 입력하면 운영 프로토콜 문서를 읽은 에이전트가 Phase 1부터 8까지 전체 점검을 실행합니다.

2단계: 점검 실행 순서 정의

ops/SYSTEM.md - 점검 실행 순서

Phase 1: 상태 파악

1. projects/{서비스}/cycles/summary.md 읽기
2. projects/{서비스}/issues/open.md 읽기
3. 이전 점검 미완료 작업 확인
4. (최초 점검) 서비스 전체 분석 후 context.md 작성

Phase 2: 비즈니스 점검

1. checklists/business.md 실행
2. checklists/marketing.md 실행
3. checklists/growth.md 실행
4. 각 팀의 memory.md 참조 후 이슈 open.md에 추가

Phase 3: 기술 점검

1. checklists/tech.md 실행
2. teams/tech/memory.md 참조
3. 발견된 이슈 open.md에 추가

이어서 수정, 검증, 기록 단계입니다:

Phase 4: 수정

1. open.md 이슈를 우선순위로 정렬 (FAIL > NEEDS_IMPROVEMENT)
2. 위임 등급에 따라 처리:
 - Level 1 (자율 실행): 코드 수정, 테스트, 문서
 - Level 2 (실행 후 보고): 아키텍처 변경, UX 개선
 - Level 3 (사전 승인): 가격 정책, 브랜딩, 핵심 UX
 - Level 4 (논의 필요): 신규 기능, 피봇, 런칭 결정
3. 수정 완료 이슈 → resolved.md 이동
4. 연기 이슈 → backlog.md 이동

Phase 5~7: QA → E2E → 비즈니스 재검수
(수정 영향 범위만 재검증)

Phase 8: 기록

1. 점검 리포트 작성 (cycles/cycle-N-report.md)
2. summary.md 업데이트
3. 모든 팀의 memory.md 업데이트
4. 공유 지식 베이스 업데이트
5. company/portfolio.md 업데이트

위임 등급이 핵심

Phase 4의 위임 등급은 시스템이 제대로 작동하게 하는 안전장치입니다.

Level 1은 에이전트가 판단하고 바로 실행합니다. 버그 수정, 테스트 추가, 성능 최적화. 사람이 개입할 필요 없습니다.

Level 2는 실행은 하되 결과를 보고합니다. 아키텍처 변경, UX 개선. 사후에 확인하면 됩니다.

Level 3는 먼저 물어봅니다. 가격 정책, 브랜딩, 핵심 플로우 변경. “이렇게 바꿀까요?”를 먼저 묻고 승인 후 실행합니다.

Level 4는 반드시 논의합니다. 신규 기능 추가, 비즈니스 모델 변경, 서비스 런칭. 에이전트 혼자 결정하지 않습니다.

이 등급 체계 없이 에이전트를 자율 실행시키면 문제가 생깁니다. 실제로 겪었습니다. 에이전트가 “가격을 조정하면 전환율이 올라갈 것 같다”며 결제 플랜을 임의로 변경한 적이 있었습니다. Level 3 기준이 운영 프로토콜 문서에 명시된 이후로는 그런 일이 없었습니다.

7.6 실전: 심리테스트 플랫폼에 적용된 에이전트 설정 전체

운영 중인 서비스의 설정 패턴을 보여드립니다. 구체적인 내용은 추상화했습니다. 복사해서 자신의 프로젝트에 맞게 수정하십시오.

서비스 CLAUDE.md 핵심 구조

Tier 2 서비스의 CLAUDE.md에는 최소 다음 섹션이 필요합니다.

```
# {서비스명} (Tier 2)

## 프로젝트 정보
- 서비스 유형: {한 줄 설명}
- 기술 스택: {프론트 + 백엔드 + DB}
- 배포: {배포 방식}
- 티어: {Tier 1/2/3}

## 핵심 규칙 (절대 위반 금지)
1. 프로덕션 직접 배포 금지 - 반드시 CI/CD 파이프라인 경유
2. .env 파일 커밋 금지
3. 결제/과금 관련 코드 수정 시 Level 3 승인 필요
4. DB 마이그레이션은 롤백 스크립트 함께 작성

## 과거 장애 및 교훈
- [{날짜}] {장애 요약} → {재발 방지 조치}

## 운영 점검 루틴
- ops 경로: /ops/projects/{서비스명}/
- 점검 주기: {주간/격주} 크론
```

이 구조에서 가장 중요한 것은 **핵심 규칙**과 **과거 장애** 섹션입니다. 나머지는 Claude에게 시키면 됩니다.

Claude에게 이렇게 시키세요:

"이 프로젝트를 분석해서 CLAUDE.md를 작성해줘. 아키텍처, 코드 컨벤션, 빌드 명령어를 포함해줘."

ops 초기 설정

새 서비스를 ops 시스템에 등록하는 가장 빠른 방법은 Claude에게 시키는 것입니다.

Claude에게 이렇게 시키세요:

"ops/projects/ 아래에 {서비스명} 디렉토리 구조를 만들어줘.
프로젝트 현황 문서, 열린 이슈 목록, 팀별 누적 학습 기록을 초기화해줘."

직접 스크립트로 관리하고 싶다면 아래 초기화 스크립트를 사용하십시오.

```
#!/bin/bash
# ops/scripts/init-project.sh
# 사용법: ./init-project.sh {서비스명}

SERVICE=$1

if [ -z "$SERVICE" ]; then
    echo "사용법: $0 {서비스명}"
    exit 1
fi

BASE="ops/projects/$SERVICE"

# 디렉토리 생성
mkdir -p "$BASE/cycles"
mkdir -p "$BASE/issues"
mkdir -p "$BASE/teams/tech"
mkdir -p "$BASE/teams/business"
mkdir -p "$BASE/teams/growth"
mkdir -p "$BASE/teams/marketing"
mkdir -p "$BASE/teams/qa"
```

이어서 프로젝트 현황 문서와 이슈 파일을 초기화합니다:

```
# 프로젝트 현황 문서 초기화
cat > "$BASE/cycles/summary.md" << EOF
# $SERVICE 점검 요약
```

```
**총 점검 횟수**: 0
**마지막 점검**: -
**해결된 이슈**: 0개
**미해결 이슈**: 0개
**백로그**: 0개
```

```
## 현재 상태
```

최초 점검 전. 서비스 전체 분석 필요.

```
## 다음 점검 계획
```

```
1. 첫 번째 점검 실행 (전체 체크리스트)
EOF
```

```
# 열린 이슈 목록 초기화
cat > "$BASE/issues/open.md" << EOF
# $SERVICE - 오픈 이슈
```

```
(첫 점검 후 업데이트)
EOF
```

```
# resolved.md, backlog.md 초기화
touch "$BASE/issues/resolved.md"
touch "$BASE/issues/backlog.md"
```

마지막으로 팀별 누적 학습 기록과 의사결정 로그를 초기화합니다:

```
# 팀별 누적 학습 기록, 의사결정 로그 초기화
for TEAM in tech business growth marketing qa; do
  cat > "$BASE/teams/$TEAM/memory.md" << EOF
# $TEAM Team Memory - $SERVICE
```

```
(첫 점검 후 업데이트)
EOF
```

```
cat > "$BASE/teams/$TEAM/decisions.md" << EOF
# $TEAM Team Decisions - $SERVICE
```

```
(첫 점검 후 업데이트)
EOF
done
```

```
echo "ops 등록 완료: $BASE"
echo "다음: '$SERVICE 점검 실행' 을 실행하면 최초 점검이 시작됩니다."
```

7.7 멀티 프로젝트 CLAUDE.md 관리 전략

30개 서비스의 CLAUDE.md를 따로 관리하면 중복이 쌓입니다. “프로덕션 직접 배포 금지”는 모든 서비스에 들어갑니다. 이것을 매번 30개 파일에 복사하면 관리가 불가능합니다.

해결책은 **글로벌 CLAUDE.md + 서비스별 CLAUDE.md** 2계층 구조입니다.

글로벌 CLAUDE.md (~/.claude/CLAUDE.md)

모든 서비스에 공통으로 적용되는 규칙과 컨텍스트를 담습니다.

```
# 글로벌 규칙 (모든 프로젝트에 적용)

## 배포 규칙 (절대 위반 금지)
- 프로덕션 직접 배포 절대 금지
- 올바른 흐름: 코드 수정 → 커밋 → Git push → CI/CD 자동 배포
- Docker 이미지를 직접 빌드해서 서버에 전송하는 것 금지

## 시크릿 관리
- .env 파일 커밋 절대 금지
- API 키, 비밀번호는 절대 코드에 하드코딩 금지
- 새 배포 키/비밀번호 필요 시 직접 생성하지 말고 사용자에게 요청할 것

## 커밋 규칙
- Co-Authored-By 서명 붙이지 않기
- 커밋 메시지: feat/fix/refactor/docs/chore 접두사 사용

## 운영 점검 시스템
- 이 워크스페이스에는 운영 자동화 디렉토리에 정기 점검 루틴이 있음
- 점검 시작 전 반드시 프로젝트 현황 문서 + 열린 이슈 목록 + 누적 학습 기록 읽기
- 워임 등급(Level 1~4) 반드시 준수

## 작업 스타일
- 이미지 사용 금지 (파일 출력, 코드, 문서 모두)
- 코드 외 설명은 간결하게 (불필요한 감탄 표현 금지)
- 불확실하면 실행 전에 물어볼 것
```

서비스별 CLAUDE.md (각 서비스 루트)

글로벌 규칙 위에 서비스별 특수 사항만 추가합니다. “이 서비스만의 규칙”에 집중합니다.

```
# 심리테스트 플랫폼
```

```
## 이 서비스만의 규칙
```

- Gateway 라우팅: 프론트 → /api/ 호출 (Gateway가 /api/v1/로 리라이트, 직접 /api/v1/ 금지)
- 카카오 SDK: 배포 전 반드시 도메인 허용 목록 확인
- 현재 무료 서비스: 가격/결제 코드 수정은 Level 3 승인 필요

```
## 기술 스택 (이 서비스)
```

```
...
```

```
## 과거 장애 (이 서비스)
```

```
...
```

이 2계층 구조의 장점은 명확합니다. 글로벌 규칙이 바뀌면 한 파일만 수정하면 됩니다. 서비스별 CLAUDE.md는 그 서비스만의 맥락에 집중합니다.

컨텍스트 전환 시 주의사항

30개 서비스를 오갈 때 가장 큰 위험은 **컨텍스트 오염**입니다. 서비스 A의 배포 방식을 서비스 B에 잘못 적용하는 경우입니다.

이를 방지하는 규칙을 CLAUDE.md에 명시합니다.

```
# 글로벌 CLAUDE.md 추가 항목
```

```
## 멀티 프로젝트 컨텍스트 전환 규칙
```

- 새 서비스 작업 시작 전 반드시 해당 서비스의 CLAUDE.md를 먼저 읽을 것
- 이전 대화의 다른 서비스 컨텍스트를 현재 서비스에 적용하지 말 것
- "이 서비스도 저번처럼 하면 되지 않아요?" 같은 추정 금지 - 반드시 확인

7.8 크로스 프로젝트 지식 공유

30개 서비스를 운영하면 한 서비스에서 배운 것이 다른 서비스에도 적용되는 경우가 많습니다. 이 지식이 사라지지 않게 하는 것이 공유 지식 베이스([company/knowledge/](#)) 디렉토리입니다.

지식 축적 흐름

서비스 A에서 이슈 발견

- `teams/tech/memory.md`에 기록 (서비스 A 전용)
- 모든 서비스에 적용 가능한 교훈인가?
 - YES: `company/knowledge/learnings.md`에 추가
 - NO: 서비스 A `memory.md`에만 보관

company/knowledge/learnings.md 예시

범용 학습 지식

모든 프로젝트에서 배운 범용적 교훈.

2026-03-29: 점성술 서비스 Cycle 1

1. ****측정 없이 성장 없다****: GA4 전환 이벤트/리텐션 지표 없으면 Growth 스코어 최하위.
초기부터 핵심 전환 이벤트를 심어야 함.
 - 모든 신규 서비스 CLAUDE.md에 "출시 전 GA4 필수 이벤트 체크리스트" 추가
2. ****먹등성은 결제에서 필수****: UPDATE 쿼리에 상태 조건(WHERE status='PENDING') 없으면
동일 결제가 다른 트랜잭션으로 덮어쓰기 가능.
 - 결제 관련 서비스 체크리스트에 먹등성 항목 추가

이어서 아키텍처 교훈과 다른 프로젝트 학습입니다:

3. ****Hexagonal 아키텍처에서 인프라 개선****: 포트/어댑터 분리로 서킷 브레이커, 캐싱 등
인프라 개선이 비즈니스 로직 변경 없이 가능.
 - `tech-knowledge.md`에 패턴 추가

2026-02-17: 면접 준비 앱 Cycle 1

1. ****Firebase Analytics 중앙 집중****: 각 화면에서 개별 호출보다 `analytics_service` 단일 진입점이
이벤트 일관성 보장에 효과적.
 - 신규 서비스에 `analytics` 설계 시 이 패턴 적용
2. ****백엔드 TODO 표준화****: `TODO(phase2)/TODO(backlog)` 태그 없이 `TODO`만 쓰면
어느 단계에서 처리해야 할지 파악 불가.
 - 모든 서비스 CLAUDE.md에 `TODO` 컨벤션 추가

크로스 프로젝트 리뷰

월 1회 또는 분기 1회, 모든 서비스를 가로질러 비교하는 리뷰를 실행합니다.

```
# 크로스 프로젝트 리뷰 실행
claude --dangerously-skip-permissions \
-p "크로스 프로젝트 리뷰"
```

이 트리거를 받은 에이전트는 `company/portfolio.md` 를 읽고 전체 서비스 현황을 분석합니다.

```
# company/portfolio.md 예시

## 심리테스트 플랫폼
항목	내용
**티어**	Tier 2
**스택**	Next.js 15 + Spring Boot (Kotlin)
**상태**	운영 중, 수익화 전
**점검**	4회 완료 (마지막: 2026-03-15)
**강점**	SEO 트래픽, 바이럴 공유 구조
**약점**	수익화 부재, 미로그인 이탈률 62%

## 교육 플랫폼
항목	내용
**티어**	Tier 1
**스택**	React + Spring Boot (Java)
**상태**	결제 연동 운영 중
**점검**	8회 완료 (마지막: 2026-03-20)
**강점**	결제 안정성, 리텐션 지표 추적
**약점**	모바일 UX 개선 필요
```

크로스 프로젝트 리뷰에서 에이전트가 하는 일은 세 가지입니다.

첫째, **패턴 발견**. “심리테스트와 교육 플랫폼 모두 소셜 로그인이 없어서 이탈률이 높다. 공통 해결책이 있다.”

둘째, **자원 재배분 제안**. “Tier 3로 내려가도 되는 서비스가 있는가. 반대로 Tier 1로 올려야 할 성장 징후가 있는가.”

셋째, **지식 전파**. “서비스 A에서 해결한 SEO 문제가 서비스 B에도 그대로 있다.”

7.9 decisions.md: 왜 그렇게 했는지 기록한다

누적 학습 기록이 “배운 것”을 담는 파일이라면, 의사결정 로그(`decisions.md`)는 “결정한 것”을 담는 파일입니다.

6개월 뒤, 혹은 2년 뒤에 이 코드를 보면서 “왜 이렇게 만들었지?”라고 물을 때, decisions.md가 대답합니다. Claude도 마찬가지입니다. 이 기록이 없으면 에이전트는 이미 한번 했던 논의를 처음부터 다시 합니다.

decisions.md 포맷

```
# {서비스명} 의사결정 기록

## DEC-001 | 2026-01-10 | Level 4: 수익화 모델 확정

**배경**: 무료 서비스로 MAU 5,000 달성 후 수익화 방향 결정 필요
**옵션**:
  A. 광고 모델 - 개발 비용 없음, 낮은 UX
  B. 월 구독 2,900원 - 상세 분석 리포트 제공
  C. 단건 결제 1,000원 - 결과 상세 1회 열람

**결정**: C안 + 번들 (3회 2,500원)

**근거**: 심리테스트는 총동 구매 패턴. 월 구독 약정은 이탈을 가속함.
낮은 단가로 결제 허들을 낮추고 번들로 객단가 방어.

**결과 (추적)**: 도입 1개월 후 결제 전환율 4.1% (업계 평균 2~3% 대비 양호)
```

decisions.md 작성 원칙

세 가지를 반드시 담아야 합니다.

첫째, 배경과 옵션. 결정만 쓰면 안 됩니다. “왜 이 결정을 해야 했는가”, “어떤 선택지가 있었는가”를 반드시 씁니다. 선택지 없이 결정만 기록하면 나중에 “이게 최선이었나?”를 판단할 근거가 없습니다.

둘째, Level 표시. 해당 결정이 몇 등급이었는지를 명시합니다. Level 4 결정은 사람이 참여했다는 뜻입니다. Level 1 결정은 에이전트가 자율 실행했다는 뜻입니다. 나중에 문제가 생겼을 때 누가 어떤 판단을 했는지 추적할 수 있습니다.

셋째, 결과 추적. 결정 당시에 예상한 결과가 실제로 맞았는지를 나중에 채웁니다. 처음에는 “추적 예정”으로 비워두고, 1~2개월 뒤 데이터가 쌓이면 채웁니다. 이 습관이 없으면 의사결정의 질이 개선되지 않습니다.

크로스 프로젝트 decisions.md

서비스 수준의 decisions.md와 별개로, `company/` 레벨에도 decisions.md를 둡니다. 여러 서비스에 영향을 미치는 결정을 기록합니다.

```
# company/decisions.md
```

```
## C0-DEC-001 | 2026-01-05 | Level 4: 아키텍처 표준 확정
```

****배경**:** 신규 서비스마다 아키텍처를 다르게 설계해서 코드 재사용이 불가능해짐

****결정**:** 모든 백엔드 서비스는 Hexagonal Architecture 적용

****적용 범위**:** 신규 서비스 전체 (기존 서비스는 리팩터링 대상 아님)

****결과**:** 서비스 간 포트/어댑터 인터페이스 재사용. 신규 서비스 세팅 시간 40% 감소.

```
## C0-DEC-002 | 2026-02-20 | Level 3: Redis 키 네이밍 표준
```

****배경**:** 단일 Redis 인스턴스를 여러 서비스가 공유하다가 키 충돌 발생

****결정**:** 모든 캐시 키는 {service}:{entity}:{id} 형식 필수

****적용**:** 공유 지식 베이스에 문서화, 신규 서비스 체크리스트 항목 추가

****결과**:** 이후 키 충돌 사례 0건

이 파일은 “회사 규칙이 왜 이렇게 생겼는지”를 설명합니다. 새 에이전트가 온보딩될 때 이 파일을 읽으면, 왜 특정 패턴을 강제하는지 이해할 수 있습니다.

7.10 실습: 내 서비스를 티어링하고 자동화 계획 세우기

지금까지 배운 내용을 자신의 서비스에 적용해 보겠습니다.

Step 1: 서비스 목록과 티어 정하기

운영 중인 서비스를 모두 나열하고 티어를 지정합니다.

```
# 내 서비스 티어링 (예시)
```

```
## Tier 1
```

- 교육 플랫폼 (결제 연동, 구독자 500명)

```
## Tier 2
```

- 심리테스트 플랫폼 (월 3만 UV, 무료)

- 면접 준비 앱 (DAU 200명, 성장 중)

```
## Tier 3
```

- 사내 일정 관리 도구 (팀 5명 사용)

- 개인 포트폴리오 사이트 (정적)

Step 2: 각 서비스의 CLAUDE.md 작성

각 서비스 루트에 CLAUDE.md를 만들고, 다음 섹션을 채웁니다.

```
# {서비스명} ({티어})

## 이 서비스만의 규칙
(글로벌 CLAUDE.md에 없는 서비스 특화 규칙)

## 기술 스택
(실제 사용 중인 스택)

## 과거 장애/실수 기록
(이 서비스에서 Claude가 잘못된 것, 내가 잘못된 것)

## 빌드/배포
(명령어와 주의사항)
```

Step 3: ops 구조 초기화

```
# ops 디렉토리 생성
mkdir -p ops/checklists
mkdir -p ops/company/knowledge # 공유 지식 베이스

# 자신의 서비스 초기화
chmod +x ops/scripts/init-project.sh
./ops/scripts/init-project.sh {서비스명}
```

Step 4: 크론 설정

티어별로 크론을 설정합니다. Mac OS에서는 crontab을 씁니다.

```
crontab -e
```

```
# Tier 1: 30분마다 헬스체크
*/30 * * * * cd /path/to/workspace && claude --dangerously-skip-permissions -p "$(cat
ops/prompts/tier1-health-check.md)" --output-format json >> /logs/tier1-$(date +%Y\%m\
%d).log 2>&1

# Tier 1: 매주 월요일 09시 전체 점검
0 9 * * 1 cd /path/to/workspace && claude --dangerously-skip-permissions -p "교육 플랫폼
점검 실행" >> /logs/cycle-$(date +%Y\%m\%d).log 2>&1

# Tier 2: 매주 화요일 10시 기술 점검
0 10 * * 2 cd /path/to/workspace && claude --dangerously-skip-permissions -p "심리테스트
플랫폼 기술 점검" >> /logs/tier2-$(date +%Y\%m\%d).log 2>&1

# Tier 2: 매주 수요일 10시
0 10 * * 3 cd /path/to/workspace && claude --dangerously-skip-permissions -p "면접 준비 앱
기술 점검" >> /logs/tier2-$(date +%Y\%m\%d).log 2>&1

# Tier 3: 매월 1일 09시
0 9 1 * * cd /path/to/workspace && claude --dangerously-skip-permissions -p "Tier 3 서비
스 월간 리포트 생성 후 ops/reports/tier3-$(date +%Y\%m).md에 저장" >> /logs/tier3-$(date +\
%Y\%m).log 2>&1
```

Step 5: 첫 번째 점검 실행

설정이 완료되면 첫 점검을 수동으로 실행합니다.

```
claude -p "심리테스트 플랫폼 점검 실행"
```

최초 실행 시 에이전트는 `context.md` 를 생성합니다. 서비스 전체를 분석해서 현재 상태를 문서화합니다. 이후 모든 점검은 이 `context.md`를 기반으로 진행됩니다.

7.11 마치며: 3개부터 시작해도 같은 원리

이 챕터를 읽으면서 “30개는 나와 거리가 멀다”고 느꼈을 수 있습니다.

하지만 이 시스템은 3개부터도 동일하게 작동합니다.

서비스가 3개라면 그중 가장 중요한 1개가 Tier 1이고, 나머지 2개가 Tier 2 또는 Tier 3입니다. ops 디렉토리에 는 프로젝트 3개의 폴더가 생깁니다. 크론은 3개 실행됩니다.

서비스가 10개가 되면? 구조는 그대로입니다. 티어가 늘어나고, ops 폴더가 늘어나고, 크론이 늘어날 뿐입니다.

이 시스템의 진짜 가치는 규모가 아닙니다. 서비스가 늘어날수록 관리 비용이 선형으로 늘지 않는다는 것, 그것이 핵심입니다.

사람이 직접 관리한다면 서비스 10개는 서비스 3개보다 3.3배 더 힘들습니다. 이 시스템이 있으면 서비스 10개의 추가 부담은 크론 7개와 CLAUDE.md 7개뿐입니다.

한 가지만 기억하십시오.

안 망하게 운영하는 것이 먼저입니다. 다 잘 운영하는 건 그다음입니다.

체크리스트

이 챕터를 마친 후 다음 항목을 확인하십시오.

- 내 서비스를 Tier 1/2/3으로 분류했다
- 각 서비스에 CLAUDE.md를 작성했다 (글로벌 규칙 + 서비스별 규칙 분리)
- 운영 자동화 디렉토리를 초기화했다 (프로젝트 현황 문서, 열린 이슈 목록, 누적 학습 기록)
- 운영 프로토콜 문서에 트리거 키워드와 점검 순서를 정의했다
- 위임 등급(Level 1~4)을 운영 프로토콜 문서에 명시했다
- 티어별 크론을 설정했다
- 첫 번째 점검을 수동으로 실행해서 context.md를 생성했다
- company/portfolio.md에 내 서비스 현황을 등록했다

다음 챕터에서는 이 시스템을 운영하면서 실제로 겪은 실패 사례를 다룹니다. 에이전트가 잘못 판단한 것, 자동화가 역효과를 낸 것, CLAUDE.md 없이 발생한 장애. 완벽하지 않은 시스템이 어떻게 실패에서 배우며 성장하는지 보여드립니다.

CH08. 실패 일지 — 에이전트가 저지른 사고 7가지

“AI가 망쳤어요”의 진짜 의미

이 말을 처음 들었을 때, 저는 잠깐 멈췄습니다.

서비스가 새벽에 다운됐다는 메시지였습니다. Claude가 API 라우팅 규칙을 잘못 판단해서 결제 플로우가 통째로 깨진 것이었습니다. 사용자들은 결제 버튼을 눌러도 아무 일도 일어나지 않았습니다. 새벽 두 시에 로그를 들여다보며 원인을 찾는 동안, 저는 스스로에게 물었습니다.

“AI가 망쳤다”는 말이 정확한가?

아니었습니다.

Claude는 자기가 아는 범위 내에서 최선을 다했습니다. 라우팅 규칙을 몰랐을 뿐입니다. 그 규칙을 CLAUDE.md에 적어두지 않은 사람은 저였습니다.

실패의 80%는 AI의 문제가 아닙니다. 에이전트에게 충분한 정보를 주지 않았거나, 경계선을 명확하게 그어두지 않았거나, 잘못된 판단을 방지할 안전장치가 없었습니다.

이 챕터는 제가 실제로 겪은 사고 7가지를 기록한 실패 일지입니다. 각각의 사고 뒤에 어떤 규칙과 혹은 추가했는지도 함께 담았습니다.

당신이 똑같은 사고를 겪지 않기를 바랍니다.

8.1 게이트웨이 라우팅 장애 — 에이전트가 URL을 “수정”했다

상황

프론트엔드 서비스에서 결제 API를 호출하는 URL이 있었습니다. `/api/checkout/payment-intents/` 경로였습니다. 이 URL은 API Gateway를 거쳐 실제 결제 서비스의 `/api/v1/checkout/payment-intents/` 로 라우팅됩니다. Gateway가 RewritePath로 변환하는 방식이었습니다.

어느 날, 다른 기능을 추가하는 과정에서 Claude가 코드를 분석하다가 이런 판단을 내렸습니다.

“결제 서비스가 `/api/v1/` 경로를 사용하므로, 프론트엔드도 동일하게 `/api/v1/checkout/` 으로 맞춰야 일관성이 있습니다.”

틀린 말처럼 들리지 않았습니다. 코드를 보면 그렇게 판단하는 게 자연스럽습니다. Claude는 Gateway가 URL을 변환한다는 사실을 몰랐습니다.

사고 발생

코드가 배포됐습니다. 사용자들이 결제 버튼을 누르기 시작했고, 모두 401 에러를 받았습니다. 직접 `/api/v1/` 을 호출하는 요청은 Gateway 라우팅을 우회해서 인증 처리도 받지 못한 채 결제 서비스에 도달했습니다. 결제 서비스는 당연히 거부했습니다.

원인 분석

CLAUDE.md에 Gateway 라우팅 구조가 전혀 설명되어 있지 않았습니다. Claude는 내부 서비스의 URL 패턴을 보고 논리적으로 추론했고, 그 추론이 맞았지만 정보가 부족했습니다.

방지책

사고 직후 CLAUDE.md에 다음 섹션을 추가했습니다.

API Gateway 라우팅 규칙

절대 직접 내부 URL 호출 금지

프론트엔드는 항상 Gateway 경유 URL을 사용한다.

| 프론트엔드에서 호출 | Gateway가 변환 | 실제 서비스 경로 |
|----------------|-------------|-------------------|
| /api/checkout/ | RewritePath | /api/v1/checkout/ |
| /api/user/ | RewritePath | /api/v1/user/ |
| /api/study/ | RewritePath | /api/v1/study/ |

****주의****: 내부 서비스가 /api/v1/을 쓴다고 해서 프론트엔드도 /api/v1/을 써야 하는 것이 아니다. Gateway가 항상 변환한다. 프론트엔드 URL을 절대 임의로 변경하지 말 것.

그리고 결제 URL 관련 파일에는 주석도 추가했습니다.

```
// IMPORTANT: 이 URL은 Gateway를 경유합니다.  
// 내부 결제 서비스가 /api/v1/를 써도 이 URL을 변경하지 마세요.  
// Gateway 라우팅 규칙은 CLAUDE.md 참조.  
const PAYMENT_API_BASE = '/api/checkout';
```

8.2 무한 리팩토링 루프 — 끝이 없는 개선

상황

“레거시 코드를 리팩토링해줘”라는 요청을 했습니다. 처음에는 잘 진행됐습니다. 중복 코드가 제거되고, 메서드가 분리되고, 네이밍이 개선됐습니다. 저는 흐뭇하게 지켜봤습니다.

그런데 30분이 지나도 Claude가 멈추지 않았습니다. 1시간이 지났습니다. 돌아와서 로그를 보니 Claude는 같은 파일을 열다섯 번 수정하고 있었습니다.

리팩토링 → 코드 분석 → “이 부분도 개선할 수 있겠다” → 리팩토링 → 코드 분석 → ...

사고 발생

Claude는 자신이 만든 코드를 다시 분석하고, 그 코드에서도 개선점을 찾았습니다. 개선하면 또 개선점이 보였습니다. 완벽한 코드란 존재하지 않기 때문에 이 루프는 이론적으로 영원히 계속됩니다.

결과적으로 원래 잘 동작하던 코드가 스물두 번 수정되면서 테스트가 깨졌습니다. 과도하게 추상화된 인터페이스는 오히려 가독성을 해쳤습니다.

원인 분석

“리팩토링해줘”라는 요청에 종료 조건이 없었습니다. Claude는 “무엇이 완료 상태인가”를 알지 못했습니다. 저도 몰랐습니다. 모호한 요청은 모호한 종료로 이어집니다.

방지책

두 가지를 바꿨습니다.

첫째, 요청 방식을 바꿨습니다. 이후로는 리팩토링 요청 시 항상 스코프를 명시합니다.

리팩토링해줘 (X)

OrderService의 중복 쿼리 3개를 하나로 통합해줘.
변경 범위: OrderService.java 파일만.
완료 기준: 기존 테스트가 모두 통과하면 멈출 것. (0)

둘째, CLAUDE.md에 자율 작업 규칙을 추가했습니다.

자율 작업 규칙

반드시 종료 조건을 확인할 것
작업 시작 전에 "무엇이 완료 상태인가"를 명확히 정의한다.
종료 조건이 없는 작업은 시작하지 말고 나에게 물어볼 것.

스코프 초과 금지
요청받은 파일/기능 외의 코드를 수정하지 않는다.
"이 부분도 개선할 수 있다"고 판단해도, 요청 스코프 밖이면 목록만 작성하고 실제로 수정하지 말 것.

리팩토링 반복 제한
같은 파일을 3회 이상 수정할 경우, 작업을 중단하고 현재 상태를 보고한다.

8.3 시크릿 노출 — .env가 GitHub에 올라갔다

상황

새 서비스를 세팅하는 과정이었습니다. Claude가 환경 설정 파일을 만들고, 디렉토리를 정리하고, 초기 커밋을 만들었습니다. 모든 게 잘 돌아갔습니다.

세 시간 후, 이상한 이메일이 왔습니다. GitHub의 Secret Scanning 경보였습니다.

`.env` 파일이 공개 저장소에 올라가 있었습니다. DB 비밀번호, API 키, JWT 시크릿이 전부 노출됐습니다.

사고 발생

Claude는 `.env` 파일을 만들면서 자연스럽게 `git add .` 를 실행했습니다. `.gitignore` 에 `.env` 가 없었습니다. 아니, 정확히는 `.gitignore` 파일 자체가 없었습니다. 새 서비스라 아직 만들지 않은 상태였습니다.

Claude는 `.gitignore` 를 확인해야 한다는 것을 알지 못했습니다. 요청에 그 내용이 없었으니까요.

원인 분석

“커밋을 만들어줘”라는 요청은 Git 스테이징부터 커밋까지 포함합니다. `.gitignore` 체크는 그 흐름 어디에도 명시되지 않았습니다.

게다가 `git add .` 는 편리하지만 위험합니다. 모든 변경사항을 스테이징하기 때문에 `.env`, 인증서, 로컬 설정 파일까지 한꺼번에 올립니다.

방지책

두 가지 층을 추가했습니다.

첫 번째 층: CLAUDE.md에 커밋 규칙을 추가했습니다.

Git 규칙

커밋 전 필수 체크리스트

1. .gitignore가 존재하는지 확인한다
2. .gitignore에 다음이 포함되어 있는지 확인한다: .env, *.pem, *.key, secrets/
3. `git add .` 절대 사용 금지. 반드시 파일을 명시해서 스테이징한다
4. 스테이징된 파일 목록에서 민감한 파일이 없는지 확인한다

새 서비스 초기화 시

반드시 .gitignore를 가장 먼저 만들고 커밋한다.
.gitignore 없이 다른 파일을 커밋하지 않는다.

두 번째 층: Claude Code Hooks로 자동 차단을 만들었습니다.

```
// .claude/settings.json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "if echo \"${CLAUDE_TOOL_INPUT}\" | grep -q 'git add \\. ' ; then
echo 'ERROR: git add . 사용 금지. 파일을 명시해서 스테이징하세요.' && exit 1; fi"
          }
        ]
      }
    ]
  }
}
```

이제 Claude가 `git add .` 를 실행하려 하면 흑이 차단합니다. 에러 메시지와 함께.

8.4 프로덕션 직접 배포 — Docker 이미지가 서버로 날아갔다

상황

배포 요청을 했습니다. “결제 서비스 배포해줘.”

저는 Claude가 GitHub에 커밋을 푸시하고, CI/CD가 돌아가길 기다릴 것이라고 예상했습니다.

Claude는 Docker를 빌드했습니다. 그리고 SSH로 프로덕션 서버에 접속해서 이미지를 직접 전송했습니다. 그리고 컨테이너를 교체했습니다.

사고 발생

배포는 됐습니다. 서비스도 올라왔습니다. 표면적으로는 성공이었습니다.

하지만 CI/CD 파이프라인이 완전히 우회됐습니다. 자동화된 테스트가 실행되지 않았습니다. 코드 리뷰 게이트가 없었습니다. 아무도 모르게 프로덕션이 바뀐 것입니다.

며칠 후 그 배포 버전에서 버그가 발견됐을 때, 어떤 코드가 언제 배포됐는지 추적할 수 없었습니다. CI/CD 로그에는 흔적이 없었으니까요.

원인 분석

“배포”의 정의가 CLAUDE.md에 없었습니다. Claude는 배포를 “서비스가 실행되는 상태로 만드는 것”으로 이해했습니다. 저는 배포를 “CI/CD를 통해 안전하게 서비스가 업데이트되는 것”으로 정의하고 있었습니다. 서로 다른 정의를 가지고 있었는데, 이를 맞추지 않았습니다.

방지책

배포 규칙 (절대 원칙)

프로덕션 배포 = CI/CD 파이프라인만 허용

올바른 배포 흐름: 코드 수정 → 커밋 → Git Push → CI/CD 자동 배포

절대 금지 목록

- Docker 이미지 직접 빌드 후 서버 전송 금지
- SSH로 프로덕션 서버 접속 후 직접 컨테이너 교체 금지
- `docker-compose pull && up` 직접 실행 금지
- `kubectl apply` 직접 실행 금지

배포를 요청받았을 때

"배포해줘" 요청을 받으면, 아래 중 하나만 실행한다:

1. 코드 수정 후 커밋/푸시 (CI/CD가 자동 처리)
2. 현재 배포 상태 확인 후 보고

프로덕션에 직접 접근하는 모든 배포 방식은 L4 등급 = 반드시 사람이 직접 결정.

그리고 SSH 접속 자체를 흑으로 감시하도록 했습니다.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "if echo \"\${CLAUDE_T00L_INPUT}\" | grep -qE 'ssh .*(prod|production|server)'; then echo 'BLOCKED: 프로덕션 SSH 직접 접근은 금지되어 있습니다. CLAUDE.md의 배포 규칙을 참조하세요.' && exit 1; fi"
          }
        ]
      }
    ]
  }
}
```

8.5 의존성 지옥 — 최신 버전이 항상 좋은 건 아니었다

상황

“빌드 워닝을 정리해줘”라는 요청을 했습니다. 오래된 의존성 버전 경고가 많이 쌓여 있었습니다.

Claude는 빌드 파일을 열고 의존성 버전을 하나씩 최신 버전으로 올리기 시작했습니다. Spring Boot 2.x에서 3.x로. Hibernate 5.x에서 6.x로. QueryDSL 최신 버전으로.

사고 발생

빌드가 깨졌습니다.

Spring Boot 3.x는 Jakarta EE 9 기반입니다. `javax.*` 패키지가 전부 `jakarta.*`로 바뀌었습니다. 코드베이스 곳곳에 import되어 있던 `javax.persistence.*`, `javax.validation.*`이 전부 컴파일 에러를 냈습니다.

수백 개의 파일이 영향을 받았습니다. QueryDSL도 Spring Boot 3.x와 맞는 별도 설정이 필요했습니다. 단순히 버전 숫자를 올리는 것으로 해결될 문제가 아니었습니다.

며칠을 롤백과 수정에 썼습니다.

원인 분석

“워닝 정리”라는 요청에 의존성 업그레이드 범위가 없었습니다. Claude는 메이저 버전 업그레이드의 의미를 알지 못했습니다. 단지 숫자를 최신으로 맞추면 된다고 판단했습니다.

의존성 업그레이드는 버전에 따라 위험 수준이 완전히 다릅니다.

- 패치 버전 (1.0.1 → 1.0.2): 보통 안전. 버그 수정.
- 마이너 버전 (1.0.0 → 1.1.0): 주의 필요. 새 기능 추가, 드물게 변경.
- 메이저 버전 (1.x → 2.x): 위험. 파괴적 변경(Breaking Change) 가능성 높음.

방지책

의존성 업그레이드 규칙

등급별 처리 방식

****패치 버전**** (예: 3.2.1 → 3.2.5)

- L1: 자동 업그레이드 가능
- 조건: 동일한 메이저, 마이너 내 패치 변경만

****마이너 버전**** (예: 3.2.0 → 3.3.0)

- L2: 업그레이드 후 테스트 실행, 결과 보고
- 조건: 테스트 전체 통과 시 적용

****메이저 버전**** (예: 2.x → 3.x)

- L4: 반드시 사람에게 먼저 물어볼 것
- 자동으로 올리지 말 것
- 업그레이드 가이드(Migration Guide) 링크를 찾아서 사람에게 제시한다

금지 사항

- `latest` 태그 사용 금지
- 버전 범위 표시 (`^`, `~`) 없이 고정 버전 사용 권장
- 여러 의존성을 한꺼번에 올리지 말 것. 하나씩 검증하면서 진행.

8.6 키/비밀번호 임의 생성 — 에이전트가 만든 비밀번호

상황

새 서비스를 추가하는 작업이었습니다. Claude가 서비스를 세팅하면서 Docker Compose 파일도 만들었습니다. DB 컨테이너 설정에는 비밀번호가 필요했습니다.

Claude는 합리적으로 판단했습니다. 비밀번호가 필요하면 만들면 된다고.

```
environment:
  MYSQL_ROOT_PASSWORD: xK9mP2vL8nQ
  MYSQL_PASSWORD: aB3cD4eF5gH
```

랜덤한 문자열이 생성되었습니다. 강도 높은 비밀번호였습니다.

사고 발생

문제는 이 비밀번호를 저만 모른다는 것이 아니었습니다. 기존 시스템과 연동이 안 됐습니다.

우리 인프라에는 이미 중앙에서 관리하는 시크릿이 있었습니다. DB 비밀번호는 Vault에 저장되고, 서비스들은 그 비밀번호로 접근합니다. 기존 모니터링 도구, 백업 스크립트, 다른 서비스들이 모두 그 비밀번호를 알고 있었습니다.

Claude가 새로 만든 비밀번호는 그 시스템과 완전히 단절됐습니다. DB는 올라왔지만 어떤 기존 시스템도 접근할 수 없었습니다.

백업 스크립트가 조용히 실패하기 시작했고, 이를 후에 발견했습니다.

원인 분석

Claude는 새로운 값이 필요할 때 만드는 것이 자연스럽습니다. 하지만 시스템 구성 요소에 필요한 자격증명은 기존 관리 체계 안에 있어야 합니다. 이 규칙이 CLAUDE.md에 없었습니다.

방지책

자격증명 및 시크릿 규칙

임의 생성 절대 금지

다음은 절대 임의로 생성하지 말 것:

- DB 비밀번호
- JWT 시크릿 키
- API 키
- 암호화 키
- 인증서

올바른 처리 방법

자격증명이 필요한 상황이면:

1. 작업을 멈추고 나에게 알린다
2. "어떤 자격증명이 필요하고, 어디서 가져와야 하는지"를 설명한다
3. 내가 제공할 때까지 해당 설정 파일 작성을 보류한다

시크릿 참조 방식

실제 값 대신 환경변수 참조를 사용한다:

잘못된 예시

MYSQL_PASSWORD: xK9mP2vL8nQ

올바른 예시

MYSQL_PASSWORD: \${DB_PASSWORD}

값이 뭔지 모르면 플레이스홀더를 쓰고, TODO 주석을 달아둔다.

8.7 컨텍스트 윈도우 초과 — 고친 버그가 되살아났다

상황

긴 세션이었습니다. 새벽부터 시작한 작업이 오후까지 이어졌습니다. 같은 세션에서 여러 기능을 추가하고, 버그를 고치고, 리팩토링까지 진행했습니다.

오전에 결제 모듈의 동시성 버그를 수정했습니다. 낙관적 락을 적용하고 재시도 로직을 추가했습니다. 테스트도 통과했습니다.

오후 늦게 Claude가 같은 파일을 다시 건드리기 시작했습니다.

사고 발생

낙관적 락이 사라졌습니다.

재시도 로직이 지워졌습니다.

Claude가 오전에 수정한 코드를 오후에 되돌렸습니다. 세션이 너무 길어지면서 컨텍스트 윈도우 초반부에 있던 수정 내용을 Claude가 더 이상 “기억”하지 못했기 때문입니다.

이 코드가 배포된 후 동시 결제 요청이 몇 건 발생했을 때, 중복 결제가 일어났습니다.

원인 분석

Claude Code의 컨텍스트 윈도우는 유한합니다. 세션이 길어질수록 초반 대화는 밀려나고 결국 사라집니다. 오전에 한 작업을 오후의 Claude는 모릅니다. 새로 읽은 파일이 “현재 상태”이고, 그 파일에 아직 낙관적 락이 없다면 Claude는 없는 채로 작업합니다.

이 문제가 무서운 건 눈에 보이지 않는다는 점입니다. Claude는 코드를 되돌렸다는 사실을 알지 못합니다. 단지 지금 본 코드를 기반으로 다음 작업을 했을 뿐입니다.

방지책

세 가지 습관을 들였습니다.

첫째, **세션을 짧게 유지합니다**. 하나의 기능 단위로 세션을 끊고, 커밋으로 체크포인트를 만듭니다. 오전 작업이 완료되면 반드시 커밋하고 세션을 재시작합니다.

둘째, 중요한 수정은 커밋 메시지에 남깁니다.

fix: 결제 동시성 버그 수정

- 낙관적 락(@Version) 적용으로 동시 결제 중복 방지
- OptimisticLockException 발생 시 최대 3회 재시도 로직 추가
- PaymentServiceConcurrencyTest 추가 및 통과 확인

⚠ 이 커밋의 낙관적 락 로직은 삭제하면 안 됩니다.
동시성 테스트 실패 = 이 로직이 제거됐다는 신호.

셋째, `PostCompact` 훅을 활용합니다. 긴 세션에서 컨텍스트가 압축될 때 자동으로 알림을 받도록 `settings.json`에 훅을 설정해두면, 컨텍스트 손실이 일어나는 시점을 알 수 있습니다.

넷째, CLAUDE.md에 “수정 불가” 영역을 명시합니다.

수정 금지 구역

다음 코드는 특별한 이유 없이 절대 수정하지 말 것.
변경이 필요하다면 반드시 나에게 먼저 확인할 것.

결제 모듈 동시성 제어

- Payment 엔티티의 @Version 필드
- PaymentService의 재시도 로직 (retryOnOptimisticLock 메서드)
- 이유: 중복 결제 방지를 위한 핵심 로직. 제거 시 실결제 오류 발생.

인증/인가 필터 체인

- SecurityConfig의 필터 순서
- JwtAuthenticationFilter의 토큰 검증 로직
- 이유: 순서 변경 시 인증 우회 가능성 있음.

8.8 일곱 번의 공통점

사고를 돌아보면 공통 패턴이 보입니다.

첫째, 정보의 공백. Claude는 알고 있는 범위 내에서 최선을 다합니다. 게이트웨이 라우팅을 몰랐고, 자격증명 관리 체계를 몰랐고, 배포 파이프라인의 의미를 몰랐습니다. 그 공백에서 사고가 났습니다.

둘째, 모호한 종료 조건. “리팩토링해줘”, “배포해줘”, “워닝 정리해줘”. 요청이 모호하면 Claude는 자신의 판단으로 범위를 정합니다. 그 판단이 항상 저와 일치하지는 않습니다.

셋째, 안전장치 부재. CLAUDE.md에 규칙이 없었거나, 혹은 없었습니다. Claude가 잘못된 판단을 해도 차단할 수단이 없었습니다.

각 사고 후에 만든 방지책을 보면 세 가지 레이어가 있습니다.

레이어 1: CLAUDE.md - 무엇을 해야 하는지 알려준다
레이어 2: 코드 주석 - 특정 파일에서 무엇을 조심해야 하는지 알려준다
레이어 3: Hooks - 위반을 기술적으로 차단한다

CLAUDE.md만으로는 충분하지 않습니다. 규칙을 알아도 무시할 수 있습니다 (의도하지 않게라도). 혹은 알고 모르고를 떠나서 실행 자체를 막습니다.

8.9 당신의 실패 일지 시작하기

이 챕터를 읽으면서 비슷한 경험이 떠올랐다면, 지금 바로 기록을 시작하십시오.

CLAUDE.md에 “과거 장애” 섹션 추가하기

과거 장애 기록

이 섹션은 실제 사고 사례와 그로 인해 만들어진 규칙입니다.
"왜 이런 이상한 규칙이 있지?"라고 생각될 때, 이 섹션을 읽으십시오.

[날짜] 게이트웨이 라우팅 장애

- 무슨 일이: 프론트엔드 결제 URL이 /api/v1/로 변경되어 401 발생
- 원인: Claude가 내부 서비스 URL 패턴을 보고 프론트 URL도 맞춰야 한다고 판단
- 만들어진 규칙: "API Gateway 라우팅 규칙" 섹션 참조

[날짜] .env 파일 커밋

- 무슨 일이: 새 서비스 초기화 중 .env가 공개 저장소에 푸시됨
- 원인: .gitignore 없이 git add . 실행
- 만들어진 규칙: "Git 규칙" 섹션 참조. settings.json 혹 추가.

여기에 당신의 사고를 추가하십시오

기록하는 습관이 생기면 자연스럽게 규칙이 쌓입니다. 그 규칙들이 쌓일수록 에이전트는 같은 실수를 반복하지 않습니다.

사고 기록 템플릿

[날짜] [사고 제목]

****무슨 일이 일어났나****
(1~2문장으로 구체적으로)

****원인****
(CLAUDE.md/설정 어디가 부족했나)

****만들어진 규칙/혹****
(이 사고로 추가된 내용)

****재발 방지 확인****
- CLAUDE.md에 규칙 추가
- settings.json 혹 추가 (해당 시)
- 코드 주석 추가 (해당 시)

실습: 지금 당장 할 일

1. `CLAUDE.md` 를 엽니다
2. `## 과거 장애 기록` 섹션을 추가합니다
3. Claude가 최근에 예상과 다르게 행동한 사례를 하나 기록합니다
4. 그 사례에서 “무엇이 CLAUDE.md에 없었는가”를 찾습니다
5. 규칙을 추가합니다

비어 있는 `## 과거 장애 기록` 섹션을 보면 두 가지 감정 중 하나를 느낄 것입니다.

“아직 큰 사고가 없었다”거나, “기록하지 않았을 뿐 사고가 있었다”거나.

두 번째라면 지금이 기록을 시작할 때입니다.

요약

이 챕터에서 다룬 7가지 실패 에피소드와 핵심 교훈입니다.

| # | 사고 | 원인 | 방지책 |
|---|--------------|-------------------------|---------------------------|
| 1 | 게이트웨이 라우팅 장애 | 라우팅 구조 CLAUDE.md에 없음 | Gateway 규칙 명시 + 코드 주석 |
| 2 | 무한 리팩토링 루프 | 종료 조건 없는 작업 요청 | 스코프·종료 조건 명시 의무화 |
| 3 | 시크릿 노출 | .gitignore 없이 git add . | git add . 후 차단 + 커밋 체크리스트 |
| 4 | 프로덕션 직접 배포 | “배포”의 정의 불일치 | 배포 = CI/CD만 허용 명시 |
| 5 | 의존성 지옥 | 메이저 버전 무제한 업그레이드 | 버전별 L1/L3/L4 등급 분류 |
| 6 | 키 임의 생성 | 자격증명 관리 규칙 없음 | 임의 생성 금지 + 플레이스홀더 사용 |
| 7 | 버그 되돌림 | 컨텍스트 초과로 이전 작업 망각 | 짧은 세션 + 수정 불가 구역 명시 |

에이전트는 망각합니다. 추론합니다. 때로는 논리적이지만 상황에 맞지 않는 결정을 합니다.

그 모든 상황에서 에이전트를 보호하는 건 당신이 만든 규칙입니다. CLAUDE.md는 에이전트에게 주는 기억입니다. 혹은 에이전트에게 주는 반사신경입니다.

실패할 때마다 그 기억과 반사신경이 조금씩 정교해집니다.

다음 챕터에서는 지금까지 배운 것을 내일 당장 적용하는 방법을 다룹니다. 프로토콜 설계, 에이전트 구성, 품질 게이트까지 — 90분 안에 첫 번째 에이전트를 배포하는 실전 가이드입니다.

CH09. 내일 당장 적용하기 — 단계별 실전 로드맵

“이 책의 모든 내용을 한꺼번에 적용하려 하지 마십시오.
다음 주 월요일에 할 수 있는 딱 3가지만 하십시오.”

9.1 왜 한꺼번에 하면 안 되는가

이 책을 읽으며 아마 여러 번 이런 생각을 하셨을 것입니다.

“CLAUDE.md도 만들어야 하고, 크론도 설정해야 하고, 에이전트도 만들어야 하고, 품질 게이트도 구축해야 하고...”

그 생각이 드는 순간, 많은 개발자가 하나도 실행하지 않습니다.

완벽한 시스템을 한 번에 만들려다가 아무것도 못 만드는 것이 가장 흔한 실패 패턴입니다. 저도 처음에 그랬습니다. 크론 설정 전에 에이전트 설계를 먼저 해야 할 것 같고, 에이전트 설계 전에 아키텍처를 정리해야 할 것 같고, 그러다 보면 아무것도 시작을 못합니다.

해법은 단순합니다.

작게 시작해서, 작동하는 것을 먼저 경험하는 것.

이 챕터는 그 경험을 만드는 4단계 로드맵입니다. 1주차에는 30분짜리 세 가지만 합니다. 3개월 후에는 서비스가 스스로 점검하고 성장하는 시스템을 운영하게 됩니다.

9.2 지금까지 배운 것 체크리스트

본격적인 로드맵 전에, 이 책에서 다룬 핵심을 점검합니다. 각 항목이 “이미 했다”면 체크하고 건너뛰십시오. “아직 안 했다”면 해당 챕터로 돌아가 실습한 뒤 다음 단계로 넘어오십시오.

CH01 – AI 코딩 성숙도

- ☐ 나의 현재 성숙도 단계(Level 1~5)를 파악했다
- ☐ Level 3 이하에서 Level 4로 도약할 전략을 이해했다

CH02 – 개발 환경

- ☐ 글로벌 CLAUDE.md (~/.claude/CLAUDE.md) 를 작성했다
- ☐ 프로젝트 CLAUDE.md를 작성했다
- ☐ MCP 서버를 최소 1개 이상 연결했다

CH03 – 프로토콜 설계

- ☐ CLAUDE.md에 기술 스택, 아키텍처, 컨벤션을 명시했다
- ☐ 에이전트 행동 제약(하면 안 되는 것)을 명시했다
- ☐ 프로토콜이 프롬프트보다 강력하다는 것을 직접 실험해봤다

이어서 에이전트, 자동화, 품질 관련 체크리스트입니다:

CH04 – 에이전트 아키텍처

- ☐ 서브에이전트 1개를 만들어 실행해봤다
- ☐ 오케스트레이터-서브에이전트 구조를 이해했다
- ☐ 에이전트 간 역할 분리를 설계해봤다

CH05 – 자동화 인프라

- ☐ `claude --print` 명령으로 비대화형 실행을 해봤다
- ☐ 크론 1개를 실제로 설정해봤다
- ☐ 슬랙/디스코드로 자동 알림을 받아봤다

CH06 – 품질 게이트

- ☐ pre-commit 훅을 설정했다
- ☐ AI 코드 리뷰 에이전트를 만들어봤다
- ☐ 테스트 자동화와 배포 판단을 연결해봤다

모두 체크했다면, 이 챕터에서 그것들을 실제 워크플로우로 연결합니다.

9.3 1주차: 지금 당장 할 수 있는 3가지

소요 시간: 총 2시간

1주차는 딱 하나의 목표를 가집니다.

“오늘 밤, 에이전트가 처음으로 내 프로젝트 규칙을 기억하는 것.”

다른 건 없습니다. 크론도, 멀티 에이전트도, QA 시스템도 아직입니다.

Task 1: 글로벌 CLAUDE.md 만들기 (30분)

글로벌 CLAUDE.md는 모든 프로젝트에 공통으로 적용되는 지침입니다. 이 파일 하나로 모든 프로젝트에서 반복 설명을 없앨 수 있습니다.

```
# 글로벌 CLAUDE.md 생성
mkdir -p ~/.claude
cat > ~/.claude/CLAUDE.md << 'EOF'
# 글로벌 개발 지침

## 기본 원칙
- 코드 생성 전에 기존 파일 구조를 먼저 파악한다
- 새 파일을 만들기 전에 기존 코드에서 재사용 가능한 것을 찾는다
- 추측하지 않는다. 모르면 묻는다.

## 커밋 규칙
- 커밋 메시지 형식: feat/fix/refactor/docs/test/chore: 변경 내용
- 한 커밋에 하나의 논리적 변경만 포함한다
- 테스트 통과 확인 후 커밋한다

## 코드 스타일
- 함수는 한 가지 일만 한다
- 주석은 "무엇"이 아닌 "왜"를 설명한다
- 매직 넘버는 상수로 추출한다

## 절대 하지 않는 것
- 프로덕션 직접 배포 (CI/CD를 통해서만)
- 시크릿/API 키 코드에 하드코딩
- 테스트 없이 핵심 비즈니스 로직 수정
EOF
```

작성 후 확인합니다.

cat ~/.claude/CLAUDE.md

Task 2: 프로젝트 CLAUDE.md 만들기 (1시간)

현재 운영 중인 프로젝트 하나를 선택합니다. 프로젝트 루트에 `CLAUDE.md` 를 만듭니다.

아래는 실전 템플릿입니다. 괄호 부분을 실제 프로젝트 정보로 채웁니다.

```
# [프로젝트명] CLAUDE.md

## 프로젝트 개요
- 목적: [이 서비스가 무엇을 하는 서비스인지 1~2줄]
- 타겟 사용자: [누구를 위한 서비스인지]
- 핵심 지표: [DAU/매출/가입자 수 등 가장 중요한 지표]

## 기술 스택
- 언어: [사용 중인 언어]
- 프레임워크: [사용 중인 프레임워크]
- DB: [사용 중인 DB]
- 인프라: Docker + GitHub Actions CI/CD

## 아키텍처
- [사용 중인 아키텍처 패턴]
- 레이어 구조:
```

src/ └── application/ # 비즈니스 로직, UseCase └── domain/ # 핵심 도메인 모델 └──
infrastructure/ # DB, 외부 연동 구현체 └── presentation/ # 라우터, 컨트롤러, DTO

이어서 컨벤션, 배포 규칙, 에이전트 행동 제약입니다:

코딩 컨벤션

- 요청/응답 스키마: Request/Response 접미사 사용
- 예외: 커스텀 예외 클래스 사용
- 응답 형식: 공통 응답 래퍼 사용
- 트랜잭션: 서비스 레이어에서만 트랜잭션 선언

배포 규칙

- 배포 방법: Git push → GitHub Actions → 자동 배포
- 브랜치 전략: main(프로덕션) / develop(개발) / feature/*
- 절대 금지: 서버에 직접 접속해서 파일 수정

현재 알려진 이슈

- [있다면 기재]

에이전트 행동 제약

- DB 스키마 변경 시 반드시 사람에게 확인 요청
- 결제 관련 코드 수정 시 반드시 사람에게 확인 요청
- 신규 외부 API 연동 시 반드시 사람에게 확인 요청

Task 3: 첫 번째 에이전트 만들기 (30분)

이제 에이전트를 만들 차례입니다. 가장 간단하고 즉각 효과를 볼 수 있는 것부터 시작합니다.

코드 리뷰 에이전트 — PR을 올리기 전에 내 코드를 먼저 리뷰해주는 에이전트입니다.

```
# 프로젝트 루트에 에이전트 디렉토리 생성
```

```
mkdir -p .claude/agents
```

```
# 코드 리뷰 에이전트 생성
```

```
cat > .claude/agents/code-reviewer.md << 'EOF'
```

```
---
```

```
name: code-reviewer
```

```
description: PR 등록 전 코드를 리뷰한다. 버그, 보안 취약점, 성능 이슈, 컨벤션 위반을 찾는다.
```

```
---
```

```
# 코드 리뷰 에이전트
```

```
## 역할
```

```
PR을 등록하기 전, 변경된 코드를 리뷰하고 문제를 찾는다.
```

```
## 리뷰 체크리스트
```

1. ****버그****: 널 포인터, 인덱스 초과, 무한 루프 가능성
2. ****보안****: SQL 인젝션, XSS, 하드코딩 시크릿, 인증 누락
3. ****성능****: N+1 쿼리, 불필요한 반복 연산, 미인덱스 대용량 조회
4. ****컨벤션****: 프로젝트 CLAUDE.md의 코딩 컨벤션 준수 여부
5. ****테스트****: 핵심 로직의 테스트 커버리지

```
## 출력 형식
```

코드 리뷰 결과

심각도: 높음 (즉시 수정 필요)

- [파일명:라인번호] 문제 설명 / 수정 방법

심각도: 중간 (이번 PR에서 수정 권장)

- [파일명:라인번호] 문제 설명 / 수정 방법

심각도: 낮음 (다음 기회에)

- [파일명:라인번호] 문제 설명 / 수정 방법

통과 ✓

- 잘 작성된 부분 1~3가지

실행 방법

변경된 파일을 리뷰할 때

git diff HEAD~1 | claude -print "다음 diff를 코드 리뷰해줘" -

특정 파일을 리뷰할 때

claude -print "이 파일을 코드 리뷰해줘" < src/services/user_service.py

EOF

실제로 실행해서 작동을 확인합니다.

```
# 가장 최근 커밋의 변경사항 리뷰
git diff HEAD~1 | claude --print "다음 코드 변경사항을 리뷰해줘. 버그, 보안 이슈, 성능 문제를 찾아줘"
```

터미널에 리뷰 결과가 출력되면 성공입니다.

1주차 완료 기준: `~/claude/CLAUDE.md`, `{프로젝트}/CLAUDE.md`, `.claude/agents/code-reviewer.md`

세 파일이 만들어지고, 코드 리뷰 에이전트가 실제로 결과를 출력했을 때.

9.4 2~3주차: 자동화의 기반을 쌓는다

소요 시간: 주당 2~3시간, 총 4~6시간

2~3주차의 목표는 하나입니다.

“내가 잠든 동안 에이전트가 처음으로 혼자 일하는 것.”

Step 1: 스킬 2~3개 만들기

스킬(Skill)은 반복 작업을 정의한 마크다운 파일입니다. “이 명령을 실행하면 이 결과가 나온다”는 레시피입니다.

자주 하는 반복 작업 3가지를 고릅니다. 예시:

- 매주 개발 일지 작성
- API 문서 업데이트
- 슬로우 쿼리 분석 리포트

스킬 파일 구조:

```
mkdir -p .claude/skills

cat > .claude/skills/weekly-dev-log.md << 'EOF'
# 주간 개발 일지 생성

## 트리거
매주 금요일 오후 6시, 또는 수동으로 "주간 일지 써줘" 호출 시

## 입력
- git log --since="1 week ago" 결과
- 이번 주에 해결한 이슈 목록

## 출력
주간 개발 일지 마크다운 파일 (docs/dev-log/YYYY-WW.md)

## 일지 형식
```

주간 개발 일지 — YYYY년 WW주차

완료한 작업

- (git log 기반으로 정리)

해결한 문제

- (이슈 기반으로 정리)

다음 주 계획

- (현재 열린 이슈/PR 기반으로 제안)

배운 것

- (코드 변경에서 추출한 기술적 인사이트)

```
## 실행 명령
```

```
claude -print "$(cat .claude/skills/weekly-dev-log.md) 지난 주 git log: $(git log --since='1 week ago' --oneline)" \> docs/dev-log/$(date +%Y-%V).md
```

```
EOF
```

Step 2: 일일 리포트 크론 설정하기

이 단계가 처음으로 “내가 자는 동안 에이전트가 일한다”는 경험을 만들어줍니다.

먼저 크론 스크립트를 만듭니다.

```
mkdir -p scripts

cat > scripts/daily-report.sh << 'EOF'
#!/bin/bash
set -e

PROJECT_DIR="/path/to/your/project" # 실제 경로로 변경
SLACK_WEBHOOK="https://hooks.slack.com/services/..." # 실제 웹훅으로 변경

cd "$PROJECT_DIR"
```

Claude에게 개발 현황 분석을 요청하는 프롬프트입니다:

```
# Claude로 일일 리포트 생성
REPORT=$(claude --print "
당신은 개발 현황 분석 에이전트입니다.
오늘의 개발 현황을 분석하고 슬랙 메시지 형식으로 요약해주세요.

== 오늘의 커밋 ==
$(git log --since='24 hours ago' --oneline)

== 현재 열린 이슈 수 ==
$(git log --since='1 week ago' --oneline | wc -l)건 (최근 1주)

== 체크할 것들 ==
- 빌드 상태: $(cd $PROJECT_DIR && ./gradlew build -q 2>&1 | tail -1 || echo '빌드 실패')

다음 형식으로 출력해주세요:
[DAILY] 오늘의 개발 현황
커밋: N건
주요 변경: (3줄 이내로)
확인 필요: (있으면)
")
```

생성된 리포트를 슬랙으로 전송합니다:

```
# 슬랙으로 전송
curl -s -X POST "$SLACK_WEBHOOK" \
  -H 'Content-type: application/json' \
  -d '{"text": \"$REPORT\"}'

echo "Daily report sent: $(date)"
EOF

chmod +x scripts/daily-report.sh
```

크론에 등록합니다.

```
# crontab 편집
crontab -e
```

```
# 아래 라인 추가 (매일 오전 9시)
0 9 * * * /path/to/your/project/scripts/daily-report.sh >> /tmp/daily-report.log 2>&1
```

등록 확인:

```
crontab -l
```

다음날 아침 9시에 슬랙 메시지가 오면 성공입니다.

Step 3: pre-commit 후 설정하기

커밋 전에 자동으로 코드 품질을 체크하는 후입니다. 한 번 설정하면 이후 모든 커밋에 자동 적용됩니다.

```
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/bash

echo "🔍 코드 품질 검사 중..."

# 1. 변경된 파일 목록 가져오기
CHANGED_FILES=$(git diff --cached --name-only --diff-filter=ACM | grep -E '\.(java|kt|py|ts|js)$')

if [ -z "$CHANGED_FILES" ]; then
    echo "✓ 검사할 소스 파일 없음. 통과."
    exit 0
fi
```

이어서 Claude를 활용한 코드 리뷰와 결과 처리 부분입니다:

```
# 2. Claude로 빠른 코드 리뷰 (30초 타임아웃)
REVIEW=$(timeout 30 claude --print "
다음 변경된 파일들을 빠르게 검토해줘.
심각한 버그나 보안 이슈만 찾아줘. (30초 안에)
문제 없으면 'PASS'만 출력해줘.
문제 있으면 'FAIL: [이유]' 형식으로 출력해줘.
```

```
변경 파일:
$(git diff --cached -- $CHANGED_FILES | head -200)
" 2>/dev/null || echo "PASS")
```

```
if echo "$REVIEW" | grep -q "^FAIL"; then
  echo "❌ 코드 리뷰 실패:"
  echo "$REVIEW"
  echo ""
  echo "수정 후 다시 커밋하거나, git commit --no-verify로 건너뛸 수 있습니다."
  exit 1
else
  echo "✓ 코드 품질 검사 통과."
  exit 0
fi
EOF

chmod +x .git/hooks/pre-commit
```

테스트합니다.

```
# 테스트 파일 만들고 커밋 시도
echo "// test" > test-hook.java
git add test-hook.java
git commit -m "test: pre-commit 혹은 테스트"
# 혹은 실행되며 리뷰 결과가 출력됨
git rm test-hook.java # 정리
```

2~3주차 완료 기준: 슬랙에 일일 리포트가 자동으로 오고, 커밋 시 pre-commit 혹은 실행될 때.

9.5 1개월 후: 시스템화

소요 시간: 주당 2~4시간, 총 8~16시간

1개월 후의 목표는 이것입니다.

“서비스 점검을 내가 명령하지 않아도, 에이전트가 스스로 한다.”

서비스 티어링

모든 서비스를 같은 강도로 관리할 필요는 없습니다. 실제 비즈니스 영향도를 기준으로 티어를 나눕니다.

서비스 티어링 예시

Tier 1 - 핵심 (장애 시 즉시 대응)

- 결제 서비스
 - 인증 서비스
 - 핵심 API 서버
- 모니터링 주기: 5분 / 알림: 즉시 / 에이전트 자동 대응

Tier 2 - 중요 (30분 내 대응)

- 대시보드
 - 이메일 발송 서비스
 - 파일 업로드 서비스
- 모니터링 주기: 15분 / 알림: 슬랙 / 에이전트 진단 후 보고

Tier 3 - 일반 (다음 영업일 대응)

- 어드민 페이지
 - 배치 처리 서비스
 - 분석 서비스
- 모니터링 주기: 1시간 / 알림: 일일 리포트에 포함

각 티어별로 크론 주기와 에이전트 행동을 다르게 설정합니다.

Tier 1: 5분마다 헬스체크

* /5 * * * * /scripts/health-check-tier1.sh

Tier 2: 15분마다 점검

* /15 * * * * /scripts/health-check-tier2.sh

Tier 3: 매시간 리포트

0 * * * * /scripts/health-check-tier3.sh

QA 에이전트 구축

코드 변경이 있을 때마다 자동으로 QA를 실행하는 에이전트입니다.

```
cat > .claude/agents/qa-agent.md << 'EOF'
---
name: qa-agent
description: 코드 변경 후 자동으로 QA를 실행하고 결과를 보고한다.
---

# QA 에이전트

## 역할
PR 생성 또는 develop 브랜치 머지 후 자동으로 QA 시나리오를 실행한다.

## QA 체크리스트
1. **기능 테스트**: 핵심 유저 플로우 (회원가입, 로그인, 핵심 기능)
2. **엣지 케이스**: 빈 입력, 최대 길이 초과, 동시 요청
3. **회귀 테스트**: 이전에 발생한 버그 케이스
4. **성능 기준**: 응답 시간 P95 < 500ms

## 실행 방법
```

claude -print "\$(cat .claude/agents/qa-agent.md) 변경된 기능: \$1 테스트 환경: \$2 QA를 실행하고 결과를 보고해줘."

```
## 출력 형식
- 통과한 시나리오: N개
- 실패한 시나리오: N개 (실패 시 상세 내용 포함)
- 성능 측정 결과
- 다음 배포 가능 여부: YES / NO (이유 포함)
EOF
```

ops 시스템 도입

여러 서비스를 운영한다면, 각 서비스의 상태를 한곳에서 관리하는 ops 시스템을 만듭니다.

```
mkdir -p ops/{reports,memory,plans}

# 각 서비스의 현재 상태를 기록하는 프로젝트 현황 문서
cat > ops/summary.md << 'EOF'
# 서비스 운영 현황

## 마지막 업데이트
[에이전트가 자동으로 갱신]

## 서비스별 상태
서비스	티어	상태	마지막 배포	주요 이슈
서비스A	Tier1	정상	2주 전	-
서비스B	Tier2	주의	3일 전	DB 슬로우 쿼리

## 이번 주 액션 아이템
- 서비스B DB 인덱스 추가
EOF
```

1개월 완료 기준: 5개 이상의 크론이 실행 중이고, QA 에이전트가 최소 1번 이상 실제 이슈를 찾아냈을 때.

9.6 3개월 후: 자율 운영

소요 시간: 초기 설정 10~20시간, 이후 유지보수만

3개월 후의 목표는 이것입니다.

“내가 일주일 자리를 비워도 서비스가 스스로 점검하고 이상 징후를 보고한다.”

멀티 에이전트 파이프라인

단일 에이전트가 모든 것을 처리하는 것이 아니라, 각 역할에 특화된 에이전트들이 협력하는 구조입니다.

[오케스트레이터]

```
|  
+-- [비즈니스 점검 에이전트]: 핵심 지표 달성 여부 검토  
|  
+-- [기술 점검 에이전트]: 코드 품질, 성능, 보안 검토  
|  
+-- [QA 에이전트]: 기능 정상 동작 검증  
|  
+-- [성장 제안 에이전트]: 개선 포인트 발굴  
|  
+-- [리포트 에이전트]: 검수 결과 취합 및 보고
```

오케스트레이터 스크립트 예시:

```
cat > scripts/service-cycle.sh << 'EOF'  
#!/bin/bash  
# 서비스 자율 점검 루틴  
# 사용법: ./service-cycle.sh [서비스명]  
  
SERVICE=$1  
DATE=$(date +%Y-%m-%d)  
REPORT_DIR="ops/reports/$SERVICE"  
mkdir -p "$REPORT_DIR"  
  
echo "[${SERVICE}] 자율 점검 루틴 시작: $DATE"
```

1~2단계로 비즈니스와 기술 점검을 수행합니다:

```
# 1단계: 비즈니스 점검  
echo "1/4 비즈니스 점검 중..."  
claude --print "  
당신은 비즈니스 점검 에이전트입니다.  
서비스: $SERVICE  
프로젝트 현황 문서(ops/summary.md)와 누적 학습 기록(ops/memory/$SERVICE.md)을 읽고  
이 서비스의 비즈니스 지표 달성 여부를 점검해주세요.  
문제가 있다면 구체적인 개선 방향을 제시해주세요.  
" > "$REPORT_DIR/business-$DATE.md"  
  
# 2단계: 기술 점검  
echo "2/4 기술 점검 중..."  
claude --print "  
당신은 기술 점검 에이전트입니다.  
서비스: $SERVICE  
지난 7일간 git log: $(git log --since='7 days ago' --oneline)  
코드 품질, 성능, 보안 관점에서 검수해주세요.  
" > "$REPORT_DIR/tech-$DATE.md"
```

3~4단계로 성장 제안을 하고 통합 리포트를 슬랙에 전송합니다:

```

# 3단계: 성장 제안
echo "3/4 성장 포인트 분석 중..."
claude --print "
당신은 성장 분석 에이전트입니다.
비즈니스 점검: $(cat $REPORT_DIR/business-$DATE.md)
기술 점검: $(cat $REPORT_DIR/tech-$DATE.md)
이 서비스에서 다음 2주 안에 실행할 수 있는 가장 임팩트 있는 개선 3가지를 제안해주세요.
" > "$REPORT_DIR/growth-$DATE.md"

# 4단계: 통합 리포트
echo "4/4 리포트 생성 중..."
claude --print "
당신은 리포트 에이전트입니다.
다음 검수 결과를 슬랙 메시지 형식으로 요약해주세요.

비즈니스: $(cat $REPORT_DIR/business-$DATE.md | head -20)
테크: $(cat $REPORT_DIR/tech-$DATE.md | head -20)
성장: $(cat $REPORT_DIR/growth-$DATE.md | head -10)
" | curl -s -X POST "$SLACK_WEBHOOK" \
-H 'Content-type: application/json' \
-d "{\"text\": \"$(cat -)\"}"

echo "[$SERVICE] 점검 완료: $DATE"
EOF

chmod +x scripts/service-cycle.sh

```

크로스 프로젝트 자동화

여러 서비스를 운영한다면, 프로젝트 간 패턴을 자동으로 감지하고 공유하는 시스템을 만듭니다.

```
# 매주 일요일 자정: 크로스 프로젝트 리뷰
0 0 * * 0 /scripts/cross-project-review.sh

cat > scripts/cross-project-review.sh << 'EOF'
#!/bin/bash
# 전체 프로젝트 패턴 분석

PROJECTS=("service-a" "service-b" "service-c")
REPORTS=""

for PROJECT in "${PROJECTS[@]"; do
    if [ -f "ops/reports/$PROJECT/tech-$(date +%Y-%m-%d).md" ]; then
        REPORTS="$REPORTS\n\n[$PROJECT]\n$(head -10 ops/reports/$PROJECT/tech-$(date +%Y-%m-%d).md)"
    fi
done
```

수집된 리포트를 Claude에게 전달하여 크로스 프로젝트 분석을 수행합니다:

```
claude --print "
당신은 크로스 프로젝트 분석 에이전트입니다.
다음 여러 서비스의 점검 결과를 종합 분석해주세요.

$REPORTS

1. 여러 서비스에 공통으로 나타나는 문제 패턴
2. 한 서비스에서 검증된 해결책을 다른 서비스에 적용할 수 있는 케이스
3. 전체 아키텍처 차원에서 개선이 필요한 부분

분석 결과를 공유 지식 베이스(ops/company/knowledge/)에 저장할 수 있도록 마크다운으로 정리해주세요.
" > ops/cross-project-$(date +%Y-%V).md

echo "크로스 프로젝트 리뷰 완료"
EOF
chmod +x scripts/cross-project-review.sh
```

성장 루틴 완성

3개월 후 완성된 시스템은 이 모습입니다.

매일 오전 9시 → 일일 리포트 (전체 서비스 현황)
매일 오후 11시 → 코드 품질 점검 (오늘 커밋 분석)
주 1회 → 서비스별 점검 (비즈니스 + 기술 + 성장)
월 1회 → 크로스 프로젝트 리뷰 (전체 패턴 분석)
커밋마다 → pre-commit 코드 리뷰
PR마다 → QA 에이전트 자동 실행

이 상태에서 개발자가 하는 일은 세 가지입니다.

1. **방향 설정**: 어떤 기능을 만들지, 어떤 문제를 우선 해결할지
2. **최종 판단**: 에이전트가 올린 이슈를 보고 대응 여부 결정
3. **시스템 개선**: 에이전트가 놓친 것을 발견하면 프로토콜 업데이트

3개월 완료 기준: 일주일 동안 명령 없이 크론이 실행되고, 에이전트가 발견한 이슈 1개 이상을 실제로 해결했을 때.

9.7 막힐 때 하는 것들

아무리 단계를 나눠도 중간에 막히는 구간이 있습니다. 가장 흔한 막힘과 해결책입니다.

막힘 1: “CLAUDE.md를 어떻게 써야 할지 모르겠다”

빈 파일로 시작하지 말고, Claude에게 시켜서 만드십시오.

```
claude "이 프로젝트의 구조를 분석해서 CLAUDE.md 초안을 만들어줘.  
기술 스택, 아키텍처 패턴, 코딩 컨벤션, 배포 방법을 포함해줘."
```

Claude가 코드를 분석해서 초안을 만들어줍니다. 그 초안을 수정하는 것이 빈 파일에서 시작하는 것보다 훨씬 빠릅니다.

막힘 2: “크론이 실행되는데 아무 메시지도 안 온다”

로그 파일을 먼저 확인합니다.

```
# 크론 실행 로그 확인
tail -50 /tmp/daily-report.log

# 크론 자체 로그 확인 (macOS)
log show --predicate 'process == "cron"' --last 1h

# 스크립트를 직접 실행해서 에러 확인
bash -x /path/to/scripts/daily-report.sh
```

가장 흔한 원인은 세 가지입니다. - 경로 문제: 크론은 쉘 환경이 다릅니다. 스크립트 안에서 절대 경로를 사용합니다. - 환경 변수 없음: `SLACK_WEBHOOK` 같은 환경 변수를 크론 안에서 직접 export하거나 스크립트 안에 하 드코딩합니다. (단, 시크릿은 별도 파일로 관리) - `claude` 명령을 못 찾음: 크론에서는 PATH가 제한됩니다. `which claude` 로 경로를 확인하고 전체 경로를 사용합니다.

```
# PATH 문제 해결: 크론탭 상단에 추가
PATH=/usr/local/bin:/usr/bin:/bin:/path/to/claude

# 또는 스크립트 안에서 직접 지정
/full/path/to/claude --print "..."
```

막힘 3: “에이전트 결과가 일정하지 않다”

에이전트 결과가 매번 다르면 프롬프트에 출력 형식을 더 명확하게 지정합니다.

```
# 나쁜 예: 형식 미지정
claude --print "코드를 리뷰해줘"

# 좋은 예: 형식 명시
claude --print "코드를 리뷰해줘.
반드시 다음 형식으로만 출력해:
PASS: [통과 이유]
또는
FAIL: [실패 이유] | [수정 방법]
다른 형식으로 출력하지 마."
```

파싱이 필요한 경우 JSON 출력을 강제합니다.

```
claude --print "분석 결과를 JSON으로만 출력해. 다른 텍스트는 절대 포함하지 마.  
형식: {\"status\": \"pass\" or \"fail\", \"issues\": [], \"score\": 0-100}"
```

막힘 4: “크론이 구독 사용량을 너무 빨리 소진한다”

Pro 플랜(\$20/월)으로 크론 자동화까지 병행하면 사용량이 빠르게 바닥날 수 있습니다. 크론을 3개 이상 운영할 계획이라면 Max 플랜(\$100/월)을 고려하십시오. 어떤 플랜이든 일일/월간 사용량 한도가 있으므로, 크론이 대화형 작업 여유분을 잠식하지 않도록 관리해야 합니다.

사용량 관리 전략:

```
# 1. 모델 선택: 가벼운 작업은 haiku, 중요한 작업만 sonnet  
# 일일 헬스체크는 haiku로 충분  
claude --model haiku --print "서비스 상태 체크"  
  
# 2. 입력 크기 제한: git log를 전체 다 넘기지 말고 잘라서 전달  
git log --since='24 hours ago' --oneline | head -20 | \  
claude --print "다음 커밋 목록을 요약해줘"  
  
# 3. 캐시 활용: 동일한 프롬프트를 반복 실행하는 경우 결과를 캐시  
CACHE_FILE="/tmp/report-$(date +%Y-%m-%d).cache"  
if [ ! -f "$CACHE_FILE" ]; then  
    claude --print "..." > "$CACHE_FILE"  
fi  
cat "$CACHE_FILE"
```

9.8 내일 당장 시작하는 30분

이 책을 읽은 뒤 가장 흔한 실수는 “나중에 제대로 시작하자”입니다.

제대로 시작하는 날은 오지 않습니다.

지금 당장 30분으로 시작합니다.

```
# 1. 글로벌 CLAUDE.md 만들기 (5분)
mkdir -p ~/.claude
code ~/.claude/CLAUDE.md
# 또는
nano ~/.claude/CLAUDE.md

# 2. 현재 프로젝트로 이동 (1분)
cd /path/to/your/project

# 3. 프로젝트 CLAUDE.md 만들기 (15분)
claude "이 프로젝트를 분석해서 CLAUDE.md 초안을 만들어줘"
# 출력 결과를 복사해서 CLAUDE.md 파일에 저장하고 수정

# 4. 첫 번째 에이전트 테스트 (9분)
git diff HEAD~1 | claude --print "이 코드 변경사항에서 버그나 문제를 찾아줘"
```

이 4단계를 완료하는 순간, 당신의 Claude Code는 비서 수준에서 팀원 수준으로 올라갑니다.

그 다음은 이 챕터의 로드맵을 따라 하나씩 쌓아가면 됩니다. 3개월 후, 당신은 이 책을 처음 읽었을 때와 완전히 다른 방식으로 일하고 있을 것입니다.

9.9 체크포인트 요약

| 시점 | 목표 | 핵심 결과물 |
|-------|------------------|---|
| 1주차 | 에이전트가 프로젝트를 기억한다 | 글로벌 CLAUDE.md + 프로젝트 CLAUDE.md + 코드 리뷰 에이전트 |
| 2~3주차 | 에이전트가 밤에 혼자 일한다 | 일일 리포트 크론 + pre-commit 훅 + 스킬 2~3개 |
| 1개월 후 | 서비스가 스스로 점검된다 | 서비스 티어링 + QA 에이전트 + ops 시스템 |
| 3개월 후 | 팀이 자율 운영된다 | 멀티 에이전트 파이프라인 + 크로스 프로젝트 자동화 + 정기 점검 루틴 |

이 로드맵은 완벽한 순서가 아닙니다. 당신의 서비스 상황에 맞게 조정하십시오. 중요한 것은 방향입니다.

Claude Code를 도구에서 팀으로 만드는 방향.

그 방향으로 한 걸음씩 나아가면, 3개월 뒤 돌아봤을 때 지금과는 완전히 다른 곳에 서 있게 됩니다.

다음 챕터는 에필로그입니다. 1인 개발팀이 만들어낸 것들, 그리고 앞으로의 가능성에 대해 이야기합니다.

에필로그: 1인 개발팀은 혼자가 아닙니다

이 책도 팀이 만들었습니다

고백할 것이 하나 있습니다.

이 책은 혼자 쓰지 않았습니다.

아이디에이션 에이전트가 각 챕터의 구조를 제안했습니다. 교정 에이전트가 문장을 다듬었습니다. PDF 빌더 에이전트가 최종 파일을 생성했습니다. 저는 방향을 잡고, 판단하고, 최종 결정을 내렸습니다. 나머지는 에이전트들이 함께했습니다.

이 책 자체가 '1인 개발팀'의 산출물입니다.

처음 이 사실을 밝히기 망설였습니다. "AI가 쓴 책 아니냐"는 오해를 살 수도 있으니까요. 하지만 생각을 바꿨습니다. 바로 이것이 이 책이 전달하고자 하는 핵심이기 때문입니다.

혼자라는 말의 의미가 바뀌었습니다.

패러다임 전환의 본질

이 책을 관통하는 테제를 한 문장으로 정리하면 이렇습니다.

Claude Code를 '도구'로 쓰는 개발자와 '팀'으로 쓰는 개발자의 생산성 차이는, 1인과 10인 팀의 차이가 아닙니다. 1인과 100인 팀의 차이입니다.

프롬프트를 던지고 코드를 복사하는 것과, 에이전트가 맥락을 기억하고 새벽에도 혼자 판단해서 실행하는 것은 완전히 다른 세계입니다. 도구를 잘 쓰는 것과 팀을 만드는 것은 다릅니다.

지난 챕터들을 통해 우리는 그 차이를 하나씩 만들어왔습니다.

`CLAUDE.md`로 팀의 규칙을 정의했습니다. 서버에이전트로 역할을 분배했습니다. 크론으로 24시간 작동하는 자동화를 구축했습니다. 정기 점검 루틴으로 서비스가 스스로 점검하고 성장하는 구조를 만들었습니다. 실패로 그로 에이전트의 실수를 자산으로 바꿨습니다.

이 모든 것의 결론은 하나입니다.

1인 개발팀은 혼자라는 뜻이 아닙니다. 수십 명의 에이전트와 함께하는 팀입니다.

AI와 협업하는 개발자의 미래

솔직히 말씀드리면, 저도 처음에는 확신이 없었습니다.

“에이전트가 정말 밤새 혼자 일하겠어?”, “CLAUDE.md 몇 줄이 진짜 팀 컨벤션을 대체할 수 있어?”, “크론으로 Claude를 돌리면 실제로 쓸만한 결과가 나오겠어?”

모든 의심은 실제로 해보면서 하나씩 사라졌습니다.

새벽 3시에 장애 알림이 오고, 이미 롤백이 완료된 것을 다음날 아침에 확인했을 때. 코드 리뷰 에이전트가 제가 놓친 N+1 문제를 PR 올라오자마자 잡아냈을 때. 비즈니스 점검 에이전트가 “이 기능은 핵심 지표에 기여하지 않습니다”라고 냉정하게 판단을 내렸을 때.

그때마다 생각했습니다.

이 에이전트들이 없었다면, 이 일들은 전부 새벽에 저 혼자 하고 있었을 것이라고.

AI 코딩 도구의 진화 방향은 명확합니다. 더 빠르게, 더 오래, 더 넓은 범위를 자율적으로 처리하는 방향으로 갑니다. 지금 이 순간에도 모델은 좋아지고 있고, 에이전트의 판단력은 높아지고 있습니다.

이 흐름 속에서 가장 중요한 개발자의 역량은 코딩 속도가 아닙니다.

에이전트에게 무엇을 위임할지 판단하는 능력. 팀을 설계하고 방향을 잡는 능력. 에이전트의 실수에서 구조적 원인을 찾아 시스템을 개선하는 능력.

이것이 앞으로 1인 개발자가 갖춰야 할 핵심 역량입니다.

당신도 할 수 있습니다

이 책을 읽는 동안, 어쩌면 이런 생각이 들었을 수도 있습니다.

“30개 서비스, 24개 에이전트… 나는 아직 서비스 하나도 제대로 못 돌리는데.”

괜찮습니다. 저도 처음에는 3개 서비스로 시작했습니다. 에이전트도 처음에는 CLAUDE.md 한 파일이 전부였습니다. 크론도 처음에는 한 줄짜리 헬스체크 하나였습니다.

규모가 중요한 것이 아닙니다.

지금 운영 중인 서비스 하나에, 오늘 배운 것 중 하나만 적용해 보는 것. 그게 시작입니다.

CLAUDE.md를 처음 만들면, 에이전트가 처음으로 당신의 프로젝트 규칙을 기억하기 시작합니다. 첫 번째 크론을 설정하면, 처음으로 자고 있는 동안 에이전트가 일하기 시작합니다. 첫 번째 실패 로그를 남기면, 에이전트의 실수가 처음으로 자산이 됩니다.

그 첫 번째 경험이 생기는 순간, 무언가가 바뀝니다.

“아, 이게 되는구나.”

그 감각 하나가, 이후 모든 자동화의 출발점이 됩니다.

다음 행동

책을 덮기 전에, 딱 하나만 해주십시오.

지금 당장 첫 번째 에이전트를 만드십시오.

거창할 필요 없습니다. 현재 운영 중인 프로젝트 루트에 `CLAUDE.md` 파일을 만들고, 이 세 가지만 적으십시오.

프로젝트 규칙

기술 스택

- (사용 중인 언어/프레임워크 기재)

배포 규칙

- (배포 방법 기재)

코딩 컨벤션

- (팀이 지키는 규칙 기재)

이것이 전부입니다. 이 파일이 있는 것과 없는 것의 차이는, 직접 써보기 전까지는 설명이 안 됩니다.

그다음은 자연스럽게 이어집니다. 더 많은 규칙을 추가하고 싶어질 것입니다. 반복 작업을 에이전트에게 맡기고 싶어질 것입니다. 크론을 설정하고 싶어질 것입니다. 그렇게 팀이 만들어집니다.

이 책을 읽은 독자 분들과 경험을 나누고 싶습니다. 처음 만든 에이전트가 어떻게 작동했는지, 어떤 실수를 했고 어떻게 고쳤는지, 어떤 자동화가 가장 도움이 됐는지. 그 이야기들이 쌓이면, 다음 버전의 이 책이 더 나아질 수 있습니다.

마지막으로

이 책을 읽어주셔서 감사합니다.

200여 페이지 동안 함께한 것은, 단순히 도구 사용법이 아니었습니다. 혼자 일하는 방식에 대한 근본적인 재설계였습니다. 팀이 없어도 팀처럼 일할 수 있다는 가능성에 대한 탐험이었습니다.

AI가 개발자의 자리를 대체할 것인가라는 질문은, 이제 더 이상 흥미롭지 않습니다.

더 흥미로운 질문은 이것입니다.

AI를 팀으로 만든 개발자는, 무엇을 만들 수 있는가.

그 답을 직접 만들어 보시기 바랍니다.

당신의 첫 번째 에이전트가 오늘 밤 처음으로 혼자 일하기를 바랍니다.